ON FAIRNESS AND EFFICIENCY FOR DISTRIBUTED JOB EXECUTION

Shangyi Shao

College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

3 August, 2026

Date

Shangyi Shao

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

3 August, 2026

Date

Signature

Assoc. Prof. Xueyan Tang

Authorship Attribution Statement

This thesis contains material from 1 paper accepted at conferences in which I am listed as an author.

Chapter 4 is published as Shangyi Shao and Xueyan Tang. Task Scheduling and Transmission for Distributed Job Execution. Accepted to appear in Proceedings of the 46th IEEE International Conference on Distributed Computing Systems (ICDCS), June 2026.

The contributions of the co-authors are as follows:

- Assoc. Prof. Tang provided the initial project direction and edited the manuscript drafts.
- I prepared the manuscript drafts.
- I co-designed the algorithms.
- I implemented all the experimental simulations.

3 August, 2026

Date

Shangyi Shao

Acknowledgements

I wish to express my greatest gratitude to my advisor, Assoc. Prof. Tang Xueyan. Without his full support over the past few years, it would have been impossible for me to persevere in my research and graduate. His knowledge, patience and enthusiasm motivated me during my entire PhD career, and words fail to fully capture the depth of my admiration for Prof. Tang. Every time I struggled about the deviations in math deduction or experimental results and almost felt loss, Prof. Tang has always guided me with insightful suggestions, enlightening my path. Also, I would like to deliver my genuine thanks to Dr. Liu Kanghuai and Cheng Tong, for academic discussion and other assistance in daily life. Last but not the least, many thanks and love to my parents and girlfriend. Although distant, they have been supporting me in various aspects during these years, no matter what difficulties I experienced.

Looking back, my research journey has been fraught with challenges. It was a path marked by unexpected twists and turns, each challenging my underlying assumptions and forcing me to redirect my course. Yet, there is no denying that committing to scientific research demands immense resilience, unyielding patience, and unwavering dedication. Though I once suffered deeply from these recurring setbacks, I have never regretted my choice to pursue this path. To me, the clarity of thought and the resilience I have forged along the way remain the greatest rewards of all.

Shangyi Shao, 3 August, 2026

Abstract

The rise of computing-intensive workloads has firmly established geo-distributed architectures—spanning cloud platforms, interconnected data centers, and high-performance computing clusters—as the standard paradigm for modern large-scale data processing. To exploit data locality and minimize bandwidth consumption, these systems routinely distribute computing jobs into multiple localized tasks deployed across geographically distributed sites where the requisite input data reside. This framework has become highly prevalent across diverse application domains, such as big-data processing, cloud services, and large-scale machine learning. However, the distributed and heterogeneous nature of these environments introduces novel challenges in resource allocation and job scheduling. Due to the imbalanced distribution of workloads and varying site capacities, naive extensions of classical single-cluster scheduling algorithms (e.g., applying the algorithm to each site independently) fail in distributed environments. They often precipitate resource bottlenecks, system inefficiencies, and prolonged job starvation. More critically, a fundamental trade-off remains between improving job processing efficiency (i.e., minimizing average job response time) and guaranteeing fair resource allocation among competing jobs. Purely efficiency-targeted algorithms often compromise fairness, whereas strict fairness-oriented policies inevitably degrade job response times. Reconciling these competing objectives in distributed environments demands more sophisticated mechanism design. Furthermore, while geo-distributed sites are typically interconnected via well-provisioned networks, the strategic utilization of these network resources to transfer data and computational demands across sites remains a complex problem. To address these multifaceted challenges, this dissertation systematically investigates resource allocation and job scheduling in distributed job execution case. We propose a series of novel scheduling frameworks—ranging from network-aware data/demand migration policies to purely allocation-centric scheduling mechanisms—to synergistically optimize processing efficiency and enforce rigorous fairness objectives.

The first phase of this research focuses on redefining fair resource allocation in distributed execution environments by integrating network transmission capabilities. This dissertation proposes Dynamic Aggregate Max-min Fairness (DAMF), a novel resource allocation policy that leverages spare inter-site network bandwidth to dynamically transfer data and computation demands. DAMF mandates that the aggregate resource allocation for each job across all computational sites achieves max-min fairness over the entirety of the system's demand transmission and allocation solution space. This research mathematically proves that the DAMF policy satisfies three critical properties of fair allocation: Pareto efficiency, envy-freeness, and strategy-proofness. From a practical standpoint, the study develops a comprehensive algorithm and a fast approximation tailored to compute optimal demand transmissions and precise DAMF resource allocations subject to dynamic demand distributions and network capacity constraints. Extensive experimental evaluations demonstrate that DAMF significantly outperforms conventional baseline policies, successfully achieving superior fairness metrics and decreasing the average response time among competing jobs.

Building upon the integration of network resources, the second phase of this research transitions to the optimization of system efficiency, specifically targeting the minimization of average job response time in distributed environments. To schedule the tasks, we need to determine the execution order of jobs and a set of task transmissions across the system. This research introduces Workload-Aware Selection with Transmission (WAST), an integrated job scheduling algorithm designed to concurrently compute the global job execution order and optimal inter-site task transmissions. WAST systematically formulates the response-time minimization objective into a sequence of optimization problems, which are subsequently transformed into multiple minimum-cost flow problems solvable in polynomial time by standard operations research solvers. We also present a heuristic approximation to WAST which significantly reduces the computational overhead while maintaining comparable performance. Empirical validation, driven by real-world production cluster traces, confirms that both WAST and its heuristic counterpart compute near-optimal job execution orders and transmission strategies, drastically reducing average job response time, particularly under highly skewed spatial workload distributions.

The final phase of this research unifies the dual objectives by establishing a framework that successfully reconciles the inherent trade-off between job processing efficiency and long-term fairness. We present that the criterion of long-term fairness is that no job experiences a completion time later than it would under a baseline distributed Processor Sharing (PS) protocol. To achieve this, the dissertation formulates the distributed protective scheduling problem by constructing two mathematically rigorous slack systems under two perspectives: a global view and an independent view. Crucially, the analysis presented in this study reveals that, owing to the volatile nature of dynamic job arrivals and load distributions, no global order-based scheduling policy can maintain strict protectiveness. To overcome this theoretical limitation, a novel heuristic algorithmic framework, denoted as PFSP_x, is developed. PFSP_x acts as a hybrid scheduling engine that strictly enforces the protective fairness boundaries while aggressively optimizing the job execution orders to maximize efficiency. Trace-driven comparative analyses demonstrate that PFSP_x eliminates the violations observed in global order-based policies and efficiency-targeted policies. Ultimately, PFSP_x achieves the highest job processing efficiency among all protective policies evaluated, providing a low-cost, resilient scheduling paradigm for modern, large-scale geo-distributed computing systems.

Contents

[1]This chapter is based on our paper published in IEEE ICDCS 2026.

List of Figures

List of Tables

Chapter 1

Introduction

In the era of data-driven computing, geo-distributed service systems have emerged as a prevalent architecture for both large-scale technology enterprises and research institutions. To optimize data locality, these systems typically retain and process data at its generation site, which inherently requires dispatching the tasks of computation jobs across multiple geographically separated locations, depending on the spatial distribution of the requisite data chunks. By bridging compute and storage across wide-area networks (WANs), this architectural paradigm significantly enhances overall fault tolerance, reduces data access latency and minimizes cross-site bandwidth consumption [1, 2]. Consequently, this paradigm has expanded from big data analytics [3] and global cloud infrastructures (e.g., Google Cloud, Amazon Web Services) to a broader spectrum of applications, most notably machine learning [4] and large-scale neural network model training [5, 6].

Despite these architectural advantages, operating job execution frameworks across geographically distributed sites inherently introduces a suite of new challenges, particularly for improving job processing efficiency and providing fair resource allocation. On one hand, from the user's perspective, tasks should be processed as expeditiously as possible to achieve the minimum job response time (the time span from the job arrival to the job completion). On the other hand, from the system's perspective, favoring certain jobs with extra resources naturally inflicts a performance penalty on others. This dichotomy creates an inherent conflict between efficiency and fairness. Furthermore, fairness in instantaneous resource allocation

does not necessarily translate to long-term fairness in job response times, highlighting that the appropriate notion of fairness varies depending on the specific problem context. Additionally, finding the most efficient scheduling order in geo-distributed systems is known to be NP-hard, and the problem of ensuring fair allocation in such scenarios has not yet been widely studied. More importantly, the available bandwidth between different sites can be utilized to transfer tasks with required data chunks to improve either efficiency or fairness. However, how to properly schedule these network resources to achieve both objectives simultaneously introduces significant complexity to the problem.

This dissertation focuses on tackling the following three problems within the context of distributed job execution:

- **Problem 1: [Achieving fair instantaneous resource allocation with task transmission].** How do we achieve fair, instantaneous resource allocation across jobs running concurrently in the system, given that the processing time of each task is unknown and only a limited number of tasks can be transferred based on the network bandwidth constraint in the system?
- **Problem 2: [Achieving efficient job execution with task transmission].** How do we co-optimize the job scheduling and task transmission to maximize job processing efficiency, in scenarios where we are aware of the processing time of each task and a limited number of tasks can be transferred based on the network bandwidth constraint in the system?
- **Problem 3: [Achieving long-term fairness and efficiency trade-off].** How do we schedule jobs so that the job processing efficiency is enhanced, while simultaneously providing a fair performance bound (i.e., completion deadlines) guaranteed for every job?

1.1 Research Scope

In this thesis, we focus on the fairness and efficiency of distributed job execution with network resources and study three particular problems: *max-min fair resource allocation with task transmission*, *efficient job scheduling with task transmission*, and *protective scheduling for balancing the long-term fairness and efficiency trade-off*.

The first part of the thesis is concerned with fairness in distributed job execution case that tasks (i.e., job resource demands) can be transferred across sites with limited network resources. Providing fair resource allocation can prevent starvation and fulfill Quality of Service (QoS) requirements for all jobs, but it is a non-trivial problem in distributed job execution. In addition, transferring tasks across the distributed system can potentially improve fairness since it enables starving jobs to access resources from other sites. However, effectively leveraging this requires sophisticated mechanism design to determine optimal job selection, source site origination, and target site placement decisions. In this part, we focus on achieving max-min fair single resource allocation with task transmission in distributed job execution, where the network resource is modeled by an upper bound on the transmission demand at each site. The problem consists of two critical hurdles: determining the distributed resource allocation for each job at each site with varying resource demand distributions, and utilizing the limited network resources to transfer the demands of certain jobs across sites to achieve more fair allocation and better overall utility. We aim to present a novel fairness notion and examine whether the notion satisfies widely-accepted fairness properties. We also aim to present algorithms of computing the resource allocation and task transmissions satisfying the fairness.

The second part of this thesis focuses on developing job scheduling algorithms which utilize network resources to improve job processing efficiency in geo-distributed job execution. In such scenarios, the completion time of each job is determined by the completion time of its last task across all sites. Therefore, the scheduling of jobs within individual sites presents significant optimization opportunities, especially when the task distributions of jobs among different sites are imbalanced. How to achieve optimal job scheduling in distributed job execution remains an NP-hard problem. Furthermore, transferring tasks (along with their requisite data) to other sites can potentially accelerate the execution of tasks and thereby improve the job's response time. However, there exists complex interdependency between the execution order of jobs and the transmission of tasks. Even the transmission of a single task may change the optimal execution order of jobs, while different job execution orders may, in turn, require different task transmissions to minimize the average job response time. We present a novel job scheduling strategy, called Workload-Aware Selection with Transmission (WAST), designed to optimize average job response

time. WAST computes the global execution order of jobs and a set of task transmissions between different sites in an integrated manner, and iteratively schedules the job with the minimum response time until all jobs are scheduled. We also present a heuristic approximation algorithm that reduces the computational complexity without significantly compromising scheduling quality. Experimental results show that both algorithms can efficiently compute near-optimal job execution order and task transmissions, thereby significantly reducing job response times, especially in scenarios where the initial computation load distributions of jobs are highly skewed across sites.

The third topic of this thesis is concerned with the long-term fairness and efficiency trade-off in the context of distributed job execution. In this topic, we aim to provide a performance guarantee—specifically, a completion time deadline for each job—and design scheduling policies that improve the job processing efficiency based on it. Inspired by the notion of protective scheduling in the single site case, our long-term fairness criterion dictates that no job should complete later than it would under a distributed Processor Sharing (PS) protocol, which evenly shares the site capacity among all active jobs at each site. Building on this foundation, we formulate the protective scheduling problem under the distributed job execution by establishing the slack system with two types of views, a global view and an independent view. Based on these two views, we present a set of algorithms extended from the single site scenario. We prove that although violations can be rare, no globally order-based scheduling policy is protective due to dynamic job arrivals and load distribution. To further enhance the efficiency, we propose a novel heuristic algorithm framework PFSP_x, which not only guarantees protectiveness but also improves job processing efficiency compared to extension algorithms. We validate the proposed algorithms through real-world traces-driven evaluations. The results show that PFSP using the global slack scheme can effectively decrease the number of deadline violations while maintaining performance comparable to efficiency-targeted policies in terms of average job response time, and PFSP_x achieves the highest job processing efficiency among all protective policies.

1.2 Thesis Contributions

The remainder of this thesis is organized as follows:

In Chapter 2, we review the related work of fair resource allocation and efficient job scheduling in single-site and distributed job execution.

In Chapter 3, we study achieving max-min fair resource allocation with task (demand) transmission for distributed job execution. We present a fairness notion named Dynamic Aggregate Max-min Fairness (DAMF), which requires the aggregate resource allocation of each job to be max-min fair under all possible allocations and demand transmissions. We prove that DAMF satisfies important fairness properties, including Pareto Efficiency, envy-freeness and strategy-proofness. We further present an algorithm to search for DAMF allocation and a fast approximation method. The experiments using real-world job traces demonstrate that DAMF can effectively reduce the imbalance of the resource allocation among jobs while improving the average job response time.

In Chapter 4, we study improving the job processing efficiency by job scheduling and task transmission in the distributed job execution case. We start from optimizing the response time of a single job and model it as solving multiple Integer Linear Programming (ILP) problems. We then transform the ILP into a minimum-cost flow problem which can be efficiently solved in polynomial time. Next, we present the policy Workload-Aware Selection with Transmission (WAST), which computes the global execution order of jobs and a set of task transmissions across sites in an integrated manner. Specifically, WAST iteratively schedules the job with the minimum response time, updates its workload and task transmission to the system, repeats this process until all jobs are scheduled. We also present a fast approximation to WAST with lower complexity. Our experiments using real-world job traces show that WAST and its approximation significantly outperform the baseline policies in terms of average job response time.

In Chapter 5, we study the long-term fairness and efficiency trade-off in distributed job execution. We dictate that no job should complete later than it would under a distributed Processor Sharing (PS) protocol as the criterion of long-term fairness, and develop job scheduling policies that improve efficiency with respect to this deadline. We construct two types of slack systems—a global view and an independent view—that help identify whether any job violates its PS deadline. We also extend algorithms presented in the single site case based on the two types of slacks. We prove that no globally order-based scheduling policy can maintain this guarantee because of unpredictable job arrival and load distribution. To solve this

dilemma, we present an algorithm framework named PFSP_x, which both guarantees protectiveness and improves job processing efficiency. Experimental results demonstrate that PFSP_x achieves the lowest average job response time among all protective policies.

In Chapter 6, we conclude this thesis and outline possible future research directions.

Chapter 2

Literature Review

2.1 Fairness and Efficiency in Single-Site Systems

In the single-site scenario, the Shortest Remaining Processing Time (SRPT) policy is universally recognized as the optimal policy for maximizing job processing efficiency, whereas a multitude of notions exist for different types of fair resource allocation. The most widely used resource allocation policy in the single-site system is the Proportional Sharing (PS) policy, which allocates resources (i.e., system processing capacities) to jobs in proportion to their workload weights. If the system is not aware of the specific workload but only knows the resource demand of each job, the PS policy becomes the Max-min Fairness (MMF) allocation. Radunovic *et al.* [7] presents general algorithms for computing the MMF resource allocation in the single site system, which aims to maximize the most unsatisfied job. In addition, there are other notions of fairness such as the proportional fairness and weighted fairness. Mo *et al.* [8] presented a unified framework, namely α-Fairness, to harmonize various existing concepts of fairness and efficiency. When $\alpha = 0$, the α-Fairness degenerates to the efficiency-targeted which tries to maximize the total system utility; when $\alpha = 1$, it becomes the proportional fairness; when $\alpha \to \infty$, it becomes the MMF allocation. There are also studies working on the multi-resource allocation case. Cobb Ghodsi *et al.* [9] proposed Dominant Resource Fairness (DRF) which decides a user's allocation by its dominant share, i.e., the maximum proportional share that the user is allocated among all resource

types. Ghodsi *et al.* [10] addressed task placement constraints and presented Constrained Max-min Fairness (CMMF), which tries to meet the fairness target by recursively satisfying the users based on the order of dissatisfaction. Zahedi and Lee [11] solved the multi-resource allocation problem by proportionally allocating different resources based on the re-scaled Cobb-Douglas Utility, and named it Resource Elasticity Fairness. Gutman and Nisan [12] further extended DRF with respect to the bundle measure and utility functions. Fossati *et al.* [13] presents a distributed slice multi-resource allocation algorithm specialized for centralized 5G systems. Li *et al.* [14] studies the fair resource allocation based on various SLAs of users while improve the execution efficiency. Guo *et al.* [15] presents a fair multi-resource allocation mechanism in cloud computing system with elastic user demands. Multi-resource allocation holds wide applications in cloud servers [16], heterogeneous GPU clusters [17], 5G networks [18] and LLM inference [19]. In particular, Friedman *et al.* [20, 21] presents the notion of protective scheduling, which sets the criterion of fairness to be that no job would finish later than its completion time under the PS policy. Under this fairness guarantee, they present a series of algorithms to further improve the job processing efficiency.

2.2 Fairness and Efficiency in Distributed Job Execution Cases

The task (job) scheduling problem in geo-distributed environments has been widely studied from a variety of system scenarios and performance objectives [22–25]. Since the job load and the system capacity are distributed (mostly unevenly) across multiple sites, the problem becomes significantly more complex due to the additional dimension of site-specific resource constraints and inter-site communication overheads. Although the SRPT policy can be extended to such distributed job execution cases, it admits multiple algorithmic variations and they all no longer guarantees optimality. Hung *et al.* [26] presents SWAG, a workload-aware strategy which schedules jobs based on their estimated completion times under given task placements, but it is efficiency-targeted and neglects to utilize network resources to move tasks. Guan *et al.* [27], Zhao *et al.* [28] study the task assignment problem given that the input data of each task can be replicated across multiple geo-distributed data centers, and they do not consider using network resources

either. Hu *et al.* [29] and Pu *et al.* [30] study algorithms considering task transmissions, but they focus on improving the efficiency of a single job and do not study the scheduling strategy across jobs. Hung *et al.* [31] proposes Tetrium, an entire system that incorporates both compute and network resources to improve the efficiency of map-reduce job execution, but it focuses on the task assignment of each single job and simply applies SRPT to schedule multiple jobs. Convolbo *et al.* [32] and Li *et al.* [33] coordinates scheduling with transmission but focuses on minimizing the makespan of the entire system instead of the average response time of jobs. Li *et al.* [34] improves the MapReduce task scheduling efficiency by monitoring the idle containers in the geo-distributed system and solving the task assignment problem using the classical Hungarian algorithm. Some research focuses on utilizing the game theory to minimize the system entity cost by assuming that users in the system have different interests, and thus they form a game model [35, 36]. Recent studies have also adopted deep reinforcement learning to search for effective resource allocation strategies in fields like wireless networks [37], edge computing [38, 39], satellite networks [40, 41] as well as conventional distributed data centers [42]. However, learning-based approaches have mainly focused on improving the efficiency of resource allocation (such as system utility, revenue and profit) or achieving load balancing instead of achieving the fairness of resource allocation.

Many efforts aim to minimize the bandwidth usage or the transmission time/cost in the system. For example, Hsieh *et al.* [4] presents Gaia to minimize the transmission cost between machine learning clusters while maintaining the model's performance, Chen *et al.* [43] focuses on optimizing the transmission time of data along the given paths, and Shi *et al.* [44] aims to save the network bandwidth usage on both intra-DC and inter-DC links while ensuring the QoS of microservices. In addition, Zhao *et al.* [45] models the scheduling problem in geo-distributed systems as an extension to Hypergraph Partition-based Scheduling which aims to save the WAN data transmission amount and improves system makespan, namely HPS+.

In the context of fairness, Guan *et al.* [46] introduces a task assignment algorithm aimed at achieving max-min fairness in aggregate resource allocation among jobs in the distributed job execution. Similarly, Chen *et al.* [47] investigates achieving max-min fairness in terms of the job response times in geo-distributed systems by task transmission, but imposes strict constraints that the number of tasks is no

more than the number of computing slots in the system. Meskar *et al.* [48] presents a fair multi-resource allocation policy specialized for heterogenous servers in edge computing. Li *et al.* studies the similar problem with access constraint on each user to the servers [49]. In addition, there have also been researches investigating the job trace properties to design specialized scheduling or resource allocation algorithms [50, 51].

Chapter 3

Achieving Max-min Fairness in Distributed Job Execution with Task Transmissions

In this chapter, we study the problem of achieving max-min fair resource allocation in distributed job execution with task transmissions. The problem includes both computing the resource allocation and assigning the demand transmission across the system, so that the aggregate resource allocation is fair among jobs. We focus on the online setting where jobs arrive dynamically for execution. Our goal is to guarantee the fairness of resource allocation among jobs while improving the system utilization.

3.1 System Model

Consider a geo-distributed system composed of m separate sites: $S_1, S_2, ..., S_m$. Each site S_j holds compute resources with a capacity u_j. The capacity can be interpreted as the volume of computing slots that are available.

We would like to conduct fair resource allocations among jobs in an online environment where jobs arrive dynamically for execution. To do so, we start by studying a static problem of defining and deriving a fair instantaneous resource allocation among active jobs at a certain snapshot of the system. Suppose n jobs $J_1, J_2, ..., J_n$

are running in the system. Each job J_i consists of tasks that process distinct data partitions and could run in parallel. Due to the data locality consideration, initially each task is restricted to be served at a certain site where the input data is available. Therefore, a job's resource demand on a site S_j can be defined as the total demand of its tasks that can only run on S_j. For instance, if each task needs one computing slot for execution, the resource demand on site S_j is given by the number of tasks to run on S_j. We model the resource demands of all jobs as an $n * m$ demand matrix D_{n*m}, where each element d_{ij} denotes job J_i's demand on site S_j. Similarly, the instantaneous resource allocations to all jobs on all sites can be represented by an $n * m$ allocation matrix A_{n*m}, where each element a_{ij} denotes how much resource job J_i is allocated on site S_j. Throughout this dissertation, we shall use capital letters to stand for matrices and corresponding small letters to stand for the elements therein. For simplicity, we use $[x]$ to represent the set $\{1, 2, \ldots, x\}$ for a positive integer x.

Some basic constraints should be met to ensure that A_{n*m} is a sensible allocation. First, the summation of allocated resources on any site should not exceed that site's capacity:

$$\forall j \in [m], \quad \sum_{i=1}^{n} a_{ij} \leq u_j. \tag{3.1}$$

Allocations satisfying (3.1) are said *feasible*.

Second, any job's allocation on any site should not exceed its demand on that site so that no resources are wasted:

$$\forall i \in [n], j \in [m], \quad 0 \leq a_{ij} \leq d_{ij}. \tag{3.2}$$

Allocations satisfying (3.2) are said *rational*.

Suppose there are networks connecting the sites so that they can be used to transfer data and hence demand for compute resources from one site to another. We use an $n * m$ transmission matrix ΔD_{n*m} to represent the demand transmissions. Each element Δd_{ij} can be positive or negative: A positive Δd_{ij} describes how much demand of job J_i is transferred into site S_j, while a negative Δd_{ij} means some demand is transferred out of (removed from) site S_j, and $\Delta d_{ij} = 0$ represents no transmission. Apparently, we cannot transfer more demand out of a site than what

a given job has at that site:

$$\forall i \in [n], j \in [m], \quad d_{ij} + \Delta d_{ij} \geq 0, \tag{3.3}$$

and the overall demand of each job J_i should be kept unchanged after the transmissions:

$$\forall i \in [n], \quad \sum_{j=1}^{m} \Delta d_{ij} = 0. \tag{3.4}$$

We assume that the primary bandwidth bottleneck of the system lies in the link through which each site connects to the backbone network. Thus, to model the available network capacity, we use a count limit c_j for each site S_j to describe an upper bound on the total demand that can be transferred into and out of site S_j within a predefined short period of time T: (note that the data partition required by tasks of different jobs can be different, we add a weight w_i to each job denoting the relative data partition ratio between tasks of different jobs):

$$\forall j \in [m], \quad \sum_{i=1}^{n} |w_i \Delta d_{ij}| \leq c_j. \tag{3.5}$$

More specifically, suppose the average data partition size is 0.5 GB, the system time threshold $T = 10$ and the link bandwidth is 0.2 GB/s, then the count limit c_j at a site S_j is $0.2 * 10/0.5 = 4$. Furthermore, suppose three jobs J_1, J_2 and J_3 has data partition size 0.25 GB, 0.5 GB and 1 GB, respectively; Then, the data partition weight for each job is $w_1 = 0.25/0.5 = 0.5$, $w_2 = 0.5/0.5 = 1$, $w_3 = 1/0.5 = 2$, and the constraint is $0.5d_{1j} + d_{2j} + 2d_{3j} < 4$.

In big data processing frameworks, each task typically processes a data partition/split of the same size [52]. The transmission count limit c_j can be computed based on the data partition size, the amount of bandwidth currently available, and T.

Upon the transmissions, each job J_i's demand on site S_j becomes $d_{ij} + \Delta d_{ij}$, i.e., the demand matrix becomes $D + \Delta D$. Hence, constraint (3.2) is rewritten as:

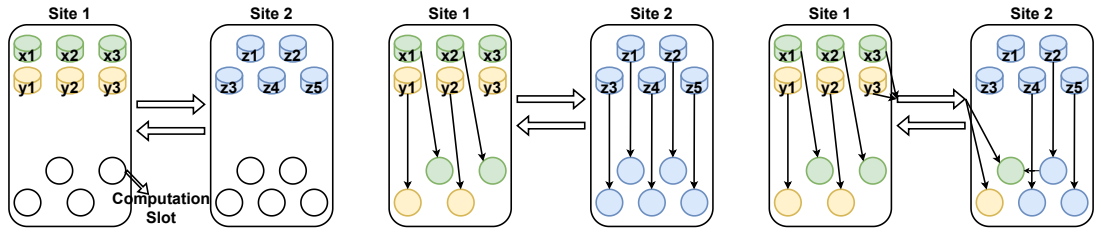
$$\forall i \in [n], j \in [m], \quad 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}. \tag{3.6}$$

We shall start with defining and deriving a fair resource allocation A_{n*m} among jobs satisfying constraints (3.1), (3.3), (3.4), (3.5) and (3.6), given the demands D_{n*m},

site capacities u_j's and transfer limits c_j's. Later, we shall discuss how to apply the solution to this static problem to online scenarios where jobs arrive dynamically. The key notations used in this paper are summarized in Table 3.1.

TABLE 3.1: Notation Table

Notation	Definition
$\mathcal{M} = \{S_1, S_2, ..., S_m\}$	Set of sites
$\mathcal{J} = \{J_1, J_2, ..., J_n\}$	Set of jobs
w_i	Job J_i's data partition weight
u_j	Site S_j's compute resource capacity
c_j	Site S_j's transmission count limit
D_{n*m}	Demand matrix
ΔD_{n*m}	Transmission matrix
A_{n*m}	Allocation matrix
$\mathcal{X}$	Set of all allocation matrices
X	Set of all aggregate allocation vectors



(A) Initial system state

(B) Fair allocation without transmission

(C) Fair allocation utilizing network resources

FIGURE 3.1: An intuitive example

Figure 3.1 provides an example of the static problem to illustrate that transferring job demands can help improve the fairness of resource allocation. Suppose a geo-distributed system has 2 sites S_1 and S_2 with capacities of 4 and 5 computing slots respectively. There are three active jobs x, y and z in the system. As shown in Figure 3.1a, jobs x and y have 3 tasks each, where the input data is available on site S_1; job z has 5 tasks, where the input data is available on site S_2. If no demand can be transferred between sites, the most balanced resource allocation among jobs x, y and z is 2, 2 and 5 computing slots as illustrated in Figure 3.1b, because jobs x and y do not have demands on site S_2. With a transfer limit of 2, we can transfer one task of job x and one task of job y from S_1 to S_2, and allocate them with computing slots on S_2. As a result, all jobs x, y and z are allocated 3 computing slots each as shown in Figure 3.1c, which is more balanced (fair) from the global perspective.

3.2 Preliminaries

3.2.1 Max-min Fairness

Max-Min Fairness (MMF) is one of the most common resource allocation strategies, which takes "serving the most unsatisfied user" as priority. Considering the resource allocation among n jobs as an n-dimensional vector, MMF can be defined as follows [7]:

Definition 3.1. Consider a set V of n-dimensional vectors. A vector $\vec{x} \in V$ is max-min fair on set V, if and only if for any $\vec{y} \in V$, if $y_s > x_s$ for an index $s \in \{1, 2, ..., n\}$, there must exist another index $t \in \{1, 2, ..., n\}$ such that $y_t < x_t \leq x_s$.

Based on the above definition, if we want to increase a component x_s in a max-min fair vector $\vec{x}$ while keeping the new vector staying in the vector set V, we must decrease another component x_t in $\vec{x}$ whose original value is less than or equal to that of x_s. The max-min fair vector is unique if it exists, because if there are multiple max-min fair vectors within a given vector set V, we can derive contradiction simply by applying the definition of MMF to them. Clearly, the max-min fair vector does not exist in every vector set. Previous work has proved sufficient conditions for the existence of the max-min fair vector [7].

Theorem 3.1. *If a set V is compact and convex, then it is max-min achievable.*

3.2.2 Aggregate Max-min Fairness

A naive idea of applying MMF to distributed job execution is to consider each site independently and find the MMF resource allocation among jobs on each site based on their local demands. This policy is known as Independent Max-min Fairness (IMF) [46]. However, IMF can be far away from providing fair services to jobs from the global perspective due to the possibly unbalanced distribution of job demands and compute capacities among sites.

Guan *et al.* [46] defined Aggregate Max-min Fairness (AMF) for distributed job execution and presented an algorithm to achieve AMF. AMF treats the distributed

system as a united resource pool and constructs an aggregate allocation vector to represent each job's total resource allocation across all distributed sites:

$$\vec{A} = \langle \sum_{j=1}^{m} a_{1j}, \sum_{j=1}^{m} a_{2j}, ..., \sum_{j=1}^{m} a_{nj} \rangle.$$

AMF requires this vector to be max-min fair among all possible aggregation vectors. While the AMF vector is unique if it exists, there may be multiple ways of resource allocation by sites to jobs producing this AMF vector since the vector only specifies each job's aggregate allocation across all sites.

AMF does not consider utilizing network resources to transfer job demands between sites. Transferring job demands has the potential to improve the fairness of compute resource allocation for jobs as well as the system utility.

3.3 Dynamic Aggregate Max-min Fairness

We propose a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) to improve the fairness of resource allocation by making use of network resources. Given the demand matrix, site capacities and transfer limits, there are a variety of possible transmission matrices and each would give rise to a number of feasible and rational allocations. Under this circumstance, DAMF requires the aggregate allocation vector to be max-min fair among all the vectors derived from the allocation matrices resulting from all possible transmission matrices. Let $\mathcal{X}$ be the set of all feasible and rational allocation matrices under all possible transmission matrices, and X be the set of all aggregate allocation vectors drawn from $\mathcal{X}$. DAMF can be formally defined as follows:

Definition 3.2. An allocation A_{n*m} satisfies Dynamic Aggregate Max-min Fairness, if and only if its aggregate allocation vector $\vec{A}$ is max-min fair over the set X.

We prove that DAMF exists by showing the compactness and convexity of set X.

Lemma 3.2. *The set X is compact.*

Proof. Consider any aggregate allocation vector $\vec{A} = \langle \sum_{j=1}^{m} a_{1j}, \sum_{j=1}^{m} a_{2j}, \ldots, \sum_{j=1}^{m} a_{nj} \rangle$ derived from some allocation matrix A under some demand matrix $D + \Delta D$ after transmission. According to constraints (3.1), (3.6) and (3.4), for the i-th component $\sum_{j=1}^{m} a_{ij}$, we have $0 \leq \sum_{j=1}^{m} a_{ij} \leq \sum_{j=1}^{m} \min\{u_j, d_{ij} + \Delta d_{ij}\} \leq \sum_{j=1}^{m} \min\{u_j, d_{ij} + c_j\}$. Thus, the set X is closed and bounded. Hence, it is compact. □

Lemma 3.3. *The set X is convex.*

Proof. Based on the definition of convexity, we need to prove that for any two vectors $\vec{A}, \vec{B} \in X$ and any $0 \leq \lambda \leq 1$, it holds that $\lambda\vec{A} + (1-\lambda)\vec{B} \in X$.

Let $\mathcal{T}$ be the set of all possible transmission matrices satisfying (3.3), (3.4) and (3.5). We first prove that $\mathcal{T}$ is convex. i.e., for any two transmission matrices $\Delta D, \Delta D' \in \mathcal{T}$ and any $0 \leq \lambda \leq 1$, $\lambda \cdot \Delta D + (1-\lambda) \cdot \Delta D' \in \mathcal{T}$.

Firstly, $\lambda \cdot \Delta D + (1-\lambda) \cdot \Delta D'$ satisfies constraint (3.3): since $\Delta D, \Delta D' \in \mathcal{T}$, for each job J_i and each site S_j, we have $d_{ij} + \Delta d_{ij} \geq 0$ and $d_{ij} + \Delta d'_{ij} \geq 0$, and hence,

$$\begin{aligned} & d_{ij} + \lambda \cdot \Delta d_{ij} + (1-\lambda) \cdot \Delta d'_{ij} \\ &= \lambda \cdot (d_{ij} + \Delta d_{ij}) + (1-\lambda) \cdot (d_{ij} + \Delta d'_{ij}) \\ &\geq \lambda \cdot 0 + (1-\lambda) \cdot 0 = 0. \end{aligned}$$

Similarly, it can be shown that $\lambda \cdot \Delta D + (1-\lambda) \cdot \Delta D'$ satisfies constraints (3.4) and (3.5). As a result, $\lambda \cdot \Delta D + (1-\lambda) \cdot \Delta D' \in \mathcal{T}$ and thus, $\mathcal{T}$ is a convex set.

Next, suppose the original demand matrix is D, and vector $\vec{A}$ and $\vec{B}$'s corresponding allocation and transmission matrices are $\{A, \Delta D^A\}$ and $\{B, \Delta D^B\}$, respectively. Since the set $\mathcal{T}$ is convex, we have $\lambda \cdot \Delta D^A + (1-\lambda) \cdot \Delta D^B \in \mathcal{T}$. By definition, $\lambda\vec{A} + (1-\lambda)\vec{B}$ is the aggregate allocation vector for the allocation matrix $\lambda A + (1-\lambda)B$. We prove that $\lambda A + (1-\lambda)B$ is a rational and feasible allocation matrix for the demand matrix $D + \lambda \cdot \Delta D^A + (1-\lambda) \cdot \Delta D^B$.

Since A and B are feasible, by constraint (3.1) we have

$$\forall j \in [m], \quad \sum_{i=1}^{n} a_{ij} \leq u_j, \quad \sum_{i=1}^{n} b_{ij} \leq u_j.$$

Thus,

$$\sum_{i=1}^{n}(\lambda a_{ij} + (1-\lambda)b_{ij}) \leq \lambda u_j + (1-\lambda)u_j = u_j.$$

Hence, $\lambda A + (1-\lambda)B$ is feasible. Similarly, it can be shown that $\lambda A + (1-\lambda)B$ is rational. Therefore, $\lambda A + (1-\lambda)B \in \mathcal{X}$. As a result, $\lambda\vec{A} + (1-\lambda)\vec{B} \in X$, and X is a convex set. □

Lemmas 3.2 and 3.3 together with Theorem 3.1 give rise to the following conclusion:

Theorem 3.4. *Dynamic Aggregate Max-min Fairness is achievable.*

3.4 Properties of DAMF

In this section, we study whether DAMF satisfies the properties of Pareto efficiency, envy-freeness and strategy-proofness, which are well-accepted characteristics of fairness.

3.4.1 Pareto Efficiency

Pareto efficiency means that no further improvement can be achieved which makes at least one individual better off without making any other individual worse off [9]. This includes two aspects [12]: first, no resource required by any consumer is left idle; second, no consumer is allocated any resource that it does not need.

In our distributed job execution context, suppose the DAMF vector and its corresponding allocation matrix are $\vec{A}$ and A_{n*m} respectively. Pareto efficiency means that for any other feasible and rational allocation matrix A'_{n*m} and its corresponding aggregate allocation vector $\vec{A'}$, if there is an index i such that the i-th component of $\vec{A}$ is larger than the i-th component of $\vec{A'}$, i.e., $\sum_{j=1}^{m} a'_{ij} > \sum_{j=1}^{m} a_{ij}$, there must be another index k such that the k-th component of $\vec{A}$ is smaller than the k-th component of $\vec{A'}$, i.e., $\sum_{j=1}^{m} a'_{kj} < \sum_{j=1}^{m} a_{kj}$.

Theorem 3.5. *DAMF is Pareto efficient.*

Proof. By definition, the DAMF vector is the max-min fair vector among all possible aggregate allocation vectors under all possible transmission matrices. For any other feasible and rational allocation matrix A'_{n*m} and its corresponding aggregate allocation vector $\vec{A'}$, if $\sum_{j=1}^{m} a'_{ij} > \sum_{j=1}^{m} a_{ij}$ for some job J_i, then based on Definition 3.1, there exists another job J_k such that $\sum_{j=1}^{m} a'_{kj} < \sum_{j=1}^{m} a_{kj} \leq \sum_{j=1}^{m} a_{ij}$. Hence, the theorem is proven. □

3.4.2 Envy-freeness

Envy-freeness means that no individual would envy another individual's allocation. More specifically, in our distributed job execution context, envy-freeness means that for any job J_l, its usable allocation will not increase if it is given any other job J_k's transmission and allocation.

Theorem 3.6. *DAMF is envy-free.*

Proof. Let $\vec{A}$ be the DAMF vector, and A_{n*m}, ΔD_{n*m} be its corresponding allocation and transmission matrices respectively. Assume on the contrary that they are not envy-free, that is, a job J_l will have its usable allocation increased if it is given another job J_k's transmission Δd_{kj}'s and allocation a_{kj}'s. Note that the transmission refers to the network bandwidth allocated, and J_l does not have to fully utilize the allocated bandwidth in order to optimize its usable allocation. Let r_{lj}'s be the actual transmission used by J_l, where for each site S_j, $|r_{lj}| \leq |\Delta d_{kj}|$ (transmission is within the allocated bandwidth), $d_{lj} + r_{lj} \geq 0$ (transmission is feasible), and $\sum_{j=1}^{m} r_{lj} = 0$ (the overall demand of J_l is unchanged after transmission). Then, the demands of J_l on all sites would become $d_{lj} + r_{lj}$'s. Thus, J_l's usable allocation on each site S_j is $\min\{d_{lj} + r_{lj}, a_{kj}\}$. If J_l's total usable allocation over all sites increases, it means that

$$\sum_{j=1}^{m} \min\{d_{lj} + r_{lj}, a_{kj}\} > \sum_{j=1}^{m} a_{lj}.$$

This implies two facts:

(i) By eliminating the min operator, we have

$$\sum_{j=1}^{m} a_{kj} > \sum_{j=1}^{m} a_{lj},$$

which means job J_k's aggregate allocation is larger than J_l's aggregate allocation in the DAMF vector.

(ii) There must be at least one site $S_{j'}$ such that $\min\{d_{lj'} + r_{lj'}, a_{kj'}\} > a_{lj'}$, which indicates $d_{lj'} + r_{lj'} > a_{lj'}$ and $a_{kj'} > a_{lj'}$. The former inequality means that J_l's demand on site $S_{j'}$ is not completely satisfied, and the latter inequality means that J_k's allocation on site $S_{j'}$ is larger than J_l's allocation on $S_{j'}$.

By (ii), from the DAMF allocation matrix A_{n*m}, we could increase $a_{lj'}$ by decreasing $a_{kj'}$ on site $S_{j'}$. Accordingly, J_l's aggregate allocation increases while J_k's aggregate allocation decreases. Note that by (i), J_k's aggregate allocation is larger than J_l's aggregate allocation in A_{n*m}. This implies that we can increase the l-th component of the DAMF vector $\vec{A}$ by decreasing its k-th component which is larger, and this contradicts the fact that $\vec{A}$ is the max-min fair vector among all possible aggregate allocation vectors. □

3.4.3 Strategy-proofness

Strategy-proofness means that no individual could achieve better allocation by lying about its demand. In our distributed job execution context, strategy-proofness requires that no job could get higher usable allocation by lying about its demand d_{ij}'s, even if this would cause the recomputation of the transmission matrix ΔD and the allocation matrix A.

Theorem 3.7. *DAMF is strategy-proof.*

Proof. Let $\vec{A}$ be the DAMF vector, and A_{n*m}, ΔD_{n*m} be its corresponding allocation and transmission matrices derived from the true demand matrix D_{n*m} (i.e., when all jobs are honest). Let D'_{n*m} be the misclaimed demand matrix when a job J_l lies about its demand, and A'_{n*m} and $\Delta D'_{n*m}$ be the resultant allocation matrix and transmission matrix respectively. Note that the transmission refers to the network bandwidth allocated. J_l does not have to fully utilize the allocated

bandwidth in order to optimize its usable allocation. Let r_{lj}'s be the actual transmission used by J_l, where for each site S_j, $|r_{lj}| \leq |\Delta d'_{lj}|$ (transmission is within the allocated bandwidth), $d_{lj} + r_{lj} \geq 0$ (transmission is feasible for J_l's true demand), and $\sum_{j=1}^{m} r_{lj} = 0$ (the overall demand of J_l is unchanged after transmission). For notational convenience, we define R as the transmission matrix actually used by all jobs, i.e., R replaces $\Delta d'_{lj}$'s with r_{lj}'s for job J_l and keeps all other elements unchanged in $\Delta D'$. Then, the real demand matrix after transmission is given by $D + R$. In particular, the real demands of J_l at all sites are $d_{lj} + r_{lj}$'s. Thus, J_l's usable allocation on each site S_j is $\min\{d_{lj} + r_{lj}, a'_{lj}\}$. We prove that J_l's total usable allocation over all sites cannot increase, i.e.,

$$\sum_{j=1}^{m} \min\{d_{lj} + r_{lj}, a'_{lj}\} \leq \sum_{j=1}^{m} a_{lj}.$$

We prove it by contradiction. Assume on the contrary that J_l's total usable allocation over all sites increases:

$$\sum_{j=1}^{m} \min\{d_{lj} + r_{lj}, a'_{lj}\} > \sum_{j=1}^{m} a_{lj}. \tag{3.7}$$

It follows that

$$\sum_{j=1}^{m} a'_{lj} > \sum_{j=1}^{m} a_{lj}. \tag{3.8}$$

Since J_l lies, it is possible that it gets excessive allocation than its real demand on some sites. Next, we construct a rational allocation matrix B' by shrinking J_l's excessive allocation (if any) in A' to its real demand:

(i) For each job $J_i \neq J_l$, let $b'_{ij} = a'_{ij}$. Since these jobs are honest about their demands, their allocations must be rational:

$$\forall j \in [m], \quad b'_{ij} = a'_{ij} \leq d_{ij} + \Delta d'_{ij} = d_{ij} + r_{ij}.$$

(ii) For job J_l, if its allocation a'_{lj} on site S_j does not exceed its real demand $d_{lj}+r_{lj}$, it is kept the same in B':

$$\forall j \in [m] \text{ where } a'_{lj} \leq d_{lj} + r_{lj}, \quad b'_{lj} = a'_{lj}.$$

(iii) For job J_l, if its allocation a'_{lj} on site S_j exceeds its real demand $d_{lj} + r_{lj}$, we shrink it to be the real demand in B':

$$\forall j \in [m] \text{ where } a'_{lj} > d_{lj} + r_{lj}, \quad b'_{lj} = d_{lj} + r_{lj}.$$

In short, B' records the allocations in A' that can be actually used by all jobs. It is clear that B' is a feasible and rational allocation matrix given the real demand matrix $D + R$. For simplicity, B' can be uniformly written as follows:

$$\forall i \in [n], j \in [m], \ b'_{ij} = \min\{d_{ij} + r_{ij}, a'_{ij}\}.$$

It then follows from (3.7) that

$$\sum_{j=1}^{m} b'_{lj} > \sum_{j=1}^{m} a_{lj}, \tag{3.9}$$

which means that J_l's aggregate allocation increases from A to B'.

By definition, A's aggregate allocation vector $\vec{A}$ is the max-min fair vector over all possible allocation matrices including those under the real demand matrix $D + R$. Based on the definition of max-min fairness and (3.9), there must be another job J_{i_1} whose aggregate allocation in A is no larger than J_l and whose aggregate allocation decreases from A to B':

$$\sum_{j=1}^{m} a_{lj} \geq \sum_{j=1}^{m} a_{i_1 j} > \sum_{j=1}^{m} b'_{i_1 j}.$$

Clearly, J_{i_1} is different from J_l. Thus, by the above construction, J_{i_1}'s allocations in B' and A' are the same. Hence, the inequality above can be rewritten as:

$$\sum_{j=1}^{m} a_{lj} \geq \sum_{j=1}^{m} a_{i_1 j} > \sum_{j=1}^{m} a'_{i_1 j}. \tag{3.10}$$

Next, we construct another allocation matrix B based on A assuming that the demand matrix is $D' + R$ (which is the demand matrix used to derive A'):

(i) For each job $J_i \neq J_l$, let $b_{ij} = a_{ij}$.

(ii) For job J_l, if its allocation a_{ij} on site S_j does not exceed the demand $d'_{lj} + r_{lj}$, it is kept the same in B: $b_{lj} = a_{lj}$.

(iii) For job J_l, if its allocation a_{ij} on site S_j exceeds the demand $d'_{lj} + r_{lj}$, we shrink it to be the latter demand in B: $b_{lj} = d'_{ij} + r_{ij}$.

In short, B is built by shrinking J_l's allocation in A exceeding the demand in $D' + R$ to the latter:

$$\forall i \in [n], j \in [m],\ b_{ij} = \min\{d'_{ij} + r_{ij}, a_{ij}\}.$$

Since J_{i_1} is different from J_l, J_{i_1}'s allocations in B and A are the same: $\sum_{j=1}^{m} a_{i_1 j} = \sum_{j=1}^{m} b_{i_1 j}$. Together with (3.10), it can be inferred that J_{i_1}'s aggregate allocation increases from A' to B, *i.e.*,

$$\sum_{j=1}^{m} b_{i_1 j} > \sum_{j=1}^{m} a'_{i_1 j}. \tag{3.11}$$

Note that both A' and B are based on the demand matrix $D' + R$ and are feasible and rational. By definition, the aggregate allocation vector of A' is the max-min fair vector over all possible allocation matrices including those under the demand matrix $D' + R$. Since J_{i_1}'s aggregate allocation increases from A' to B, based on the definition of max-min fairness, there must be another job J_{i_2} whose aggregate allocation in A' is no larger than J_{i_1} and whose aggregate allocation decreases from A' to B:

$$\sum_{j=1}^{m} a'_{i_1 j} \geq \sum_{j=1}^{m} a'_{i_2 j} > \sum_{j=1}^{m} b_{i_2 j}. \tag{3.12}$$

Clearly, J_{i_2} is different from J_{i_1}. In addition, by (3.8), (3.10) and (3.12), we have

$$\sum_{j=1}^{m} a'_{lj} > \sum_{j=1}^{m} a'_{i_1 j} \geq \sum_{j=1}^{m} a'_{i_2 j}. \tag{3.13}$$

Thus, J_{i_2} is also different from J_l. Hence, J_l, J_{i_1} and J_{i_2} are three distinct jobs.

By the construction of B, J_{i_2}'s allocations in B and A are the same. Moreover, by the construction of B', J_{i_2}'s allocations in B' and A' are also the same. Therefore, (3.12) implies that:

$$\sum_{j=1}^{m} b'_{i_2 j} > \sum_{j=1}^{m} a_{i_2 j}, \tag{3.14}$$

which means that J_{i_2}'s aggregate allocation increases from A to B'.

Now we can repeat the above analysis. Similar to the argument following (3.9), based on the definition of max-min fairness for A and (3.14), there must be another job J_{i_3} whose aggregate allocation in A is no larger than J_{i_2} and whose aggregate allocation decreases from A to B':

$$\sum_{j=1}^{m} a_{i_2 j} \geq \sum_{j=1}^{m} a_{i_3 j} > \sum_{j=1}^{m} b'_{i_3 j}.$$

Similarly, it can be shown that J_{i_3} is different from any of the jobs J_l, J_{i_1} and J_{i_2}. Since J_{i_3} is different from J_l, its allocations keep the same from A to B and from A' to B'. Therefore, J_{i_3}'s aggregate allocation increases from A' to B:

$$\sum_{j=1}^{m} b_{i_3 j} > \sum_{j=1}^{m} a'_{i_3 j}. \tag{3.15}$$

Then, similar to the argument following (3.11), based on the definition of max-min fairness for A' and (3.15), there must be another job J_{i_4} whose aggregate allocation in A' is no larger than J_{i_3} and whose aggregate allocation decreases from A' to B:

$$\sum_{j=1}^{m} a'_{i_3 j} \geq \sum_{j=1}^{m} a'_{i_4 j} > \sum_{j=1}^{m} b_{i_4 j}.$$

Similar to (3.13), it can be shown that J_{i_4} is different from any of the jobs J_l, J_{i_1}, J_{i_2}, J_{i_3}, and

$$\sum_{j=1}^{m} a'_{lj} > \sum_{j=1}^{m} a'_{i_1 j} \geq \sum_{j=1}^{m} a'_{i_2 j} > \sum_{j=1}^{m} a'_{i_3 j} \geq \sum_{j=1}^{m} a'_{i_4 j}. \tag{3.16}$$

Since J_{i_4} is different from J_l, its allocations keep the same from A to B and from A' to B'. Therefore, J_{i_4}'s aggregate allocation increases from A to B':

$$\sum_{j=1}^{m} b'_{i_4 j} > \sum_{j=1}^{m} a_{i_4 j}.$$

By repeating the above process continuously, we can obtain an infinite sequence of jobs $J_{i_1}, J_{i_2}, J_{i_3}, J_{i_4}, J_{i_5}, J_{i_6}, \ldots$ with non-increasing aggregate allocations in A'

like (3.16), which contradicts that there are a finite number of jobs. Hence, the theorem is proven. □

We now design an iterative algorithm (Algorithm 1) to compute the DAMF allocation, inspired by the max-min programming method from [7] which can find the max-min fair allocation vector for any max-min achievable set. The basic idea is to simultaneously increase the allocations of all jobs. Once a job's demand is completely satisfied, the algorithm fixes its allocation and continues to increase the allocations of other jobs. The algorithm terminates when all job demands are satisfied or all available resources are used up.

Each iteration of our algorithm consists of two stages. In stage 1, we define a unified lower bound T and maximize it by increasing the aggregate allocations of all jobs in set $\mathcal{J}$ simultaneously, where $\mathcal{J}$ is the set of jobs whose aggregate allocations have not been fixed yet. Initially, $\mathcal{J}$ contains all jobs (line 3). The maximum T value is derived by solving a linear program which models the constraints of resource allocation introduced in Section 3.1 (lines 5-12), where t_i is the aggregate allocation of a job J_i that has been fixed.

Let T_{max} denote the maximum T value obtained in stage 1. In stage 2, we identify jobs whose aggregate allocations are to be fixed. If a job J_l's total demand is equal to T_{max} amount of resources, it is fully satisfied. Thus, the algorithm fixes its aggregate allocation to be T_{max} and removes it from $\mathcal{J}$ (lines 15-17). Otherwise, the algorithm checks whether it is possible to increase J_l's aggregate allocation beyond T_{max} without violating other jobs' allocations. If J_l's aggregate allocation from stage 1 is already larger than T_{max}, or there exists a site on which J_l's demand is not fully satisfied and the site has spare capacity (i.e., the condition in line 18 is not satisfied), clearly J_l's aggregate allocation can be further increased. Otherwise, we change the aggregate allocation constraints for J_l and other jobs by replacing "$\geq$" with ">" (line 20) and "=" (line 21) respectively, and then check whether the new linear program has any feasible solution (lines 19-26). If so, it implies that J_l's allocation can be increased and J_l will be kept in set $\mathcal{J}$. Otherwise, J_l's aggregate allocation is fixed at T_{max} and J_l is removed from set $\mathcal{J}$ (lines 27-29). Note that this T_{max} value is the aggregate resource amount, which means that J_l's allocations on individual sites could vary in subsequent iterations as long as the aggregate amount is kept. The algorithm terminates when $\mathcal{J}$ becomes empty (line 4).

Algorithm 1: Computing DAMF Allocation

Input: A set of sites $\{S_1, S_2, ..., S_m\}$ with capacities $\{u_1, u_2, \ldots, u_m\}$ and transfer limits $\{c_1, c_2, \ldots, c_m\}$; a set of jobs $\{J_1, J_2, \ldots, J_n\}$ with data partition weights $\{w_1, w_2, \ldots, w_n\}$ and demand matrix D

Output: A DAMF allocation matrix A; a transmission matrix ΔD

$A \leftarrow \mathbf{0}$;
$\Delta D \leftarrow \mathbf{0}$;
$\mathcal{J} \leftarrow \{J_1, J_2, ..., J_n\}$;
while $\mathcal{J} \neq \emptyset$ **do**
  Solve the following linear program:;
    Maximize T, subject to:;
      $\forall J_i \in \mathcal{J} : \sum_{j=1}^{m} a_{ij} \geq T$;
      $\forall J_i \notin \mathcal{J} : \sum_{j=1}^{m} a_{ij} = t_i$;
      $\forall j \in [m] : \sum_{i=1}^{n} a_{ij} \leq u_j$;
      $\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;
      $\forall j \in [m] : \sum_{i=1}^{n} |w_i \Delta d_{ij}| \leq c_j$;
      $\forall i \in [n] : \sum_{j=1}^{m} \Delta d_{ij} = 0$;
  Let T_{max} be the outcome of the above linear program;
  for *each* $J_l \in \mathcal{J}$ **do**
    if $\sum_{j=1}^{m} d_{lj} = T_{max}$ **then**
      $t_l \leftarrow T_{max}$;
      $\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;
    else
      if $\sum_{j=1}^{m} a_{lj} = T_{max}$ *and* $(\forall j \in [m], a_{lj} = d_{lj} \vee \sum_{i=1}^{n} a_{ij} = u_j)$ **then**
        Check the feasibility of the following program:;
          $\sum_{j=1}^{n} a_{lj} > T_{max}$;
          $\forall J_i \in \mathcal{J} \wedge i \neq l : \sum_{j=1}^{m} a_{ij} = T_{max}$;
          $\forall J_i \notin \mathcal{J} : \sum_{j=1}^{m} a_{ij} = t_i$;
          $\forall j \in [m] : \sum_{i=1}^{n} a_{ij} \leq u_j$;
          $\forall i \in [n], j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;
          $\forall j \in [m] : \sum_{i=1}^{n} |w_i \Delta d_{ij}| \leq c_j$;
          $\forall i \in [n] : \sum_{j=1}^{m} \Delta d_{ij} = 0$;
        if *the above program is infeasible* **then**
          $t_l \leftarrow T_{max}$;
          $\mathcal{J} \leftarrow \mathcal{J} \setminus \{J_l\}$;
return $A_{n \times m}, \Delta D_{n \times m}$;

The linear program in Algorithm 1 has $2mn$ variables and $2m+2n+mn$ constraints, where m and n are the numbers of sites and jobs respectively. Since there can be up to n iterations and each iteration needs to solve up to n linear programs in stage 2, the worst-case time complexity of Algorithm 1 is $O(n^2 * LP(2mn, 2m+2n+mn))$, where $LP(2mn, 2m+2n+mn)$ is the complexity for solving a linear program with $2mn$ variables and $2m+2n+mn$ constraints.

3.4.4 A Fast Approximation

Algorithm 1 computes an exact DAMF allocation. However, its complexity could raise concern since solving large-scale linear programs can be computationally expensive [53]. In this section, we propose a heuristic algorithm to derive a near-DAMF allocation with significantly reduced time complexity to enhance the practicality.

The heuristic algorithm (Algorithm 2) proceeds in two steps. In step 1 (lines 4-9), we find a fast approximation to AMF (i.e., without transferring job demands between sites) in an iterative manner. Each iteration calculates the compute resource allocation for one site S_j, given the job demands and compute capacity on this site as well as the aggregate allocation vector $\vec{p}$ over the sites that have been processed in previous iterations (line 5). Suppose there are n jobs running in the system. Let $\vec{p} = \langle p_1, p_2, ..., p_n \rangle$ be the current aggregate allocation vector and $\langle d_{1j}, d_{2j}, ..., d_{nj} \rangle$ be the job demands on site S_j. We would like to compute an allocation $\langle a_{1j}, a_{2j}, ..., a_{nj} \rangle$ on S_j so that $\langle p_1 + a_{1j}, p_2 + a_{2j}, ..., p_n + a_{nj} \rangle$ is max-min fair among all possible allocations for the given $\vec{p}$. If the total demand of all jobs on site S_j is no larger than its capacity, i.e., $\sum_{i=1}^{n} d_{ij} \leq u_j$, we can simply satisfy the demands of all jobs by setting $a_{ij} = d_{ij}$ for all i's. Otherwise, we sort the jobs in increasing order of p_i, i.e., $p_1 \leq p_2 \leq ... \leq p_n$, and the resources on site S_j will be allocated to a prefix of this job sequence by the definition of max-min fairness. If resources are allocated to the first k jobs, the total resources allocated must be no more than

$$\sum_{i=1}^{k} \left(\min\{p_{k+1}, p_i + d_{ij}\} - p_i \right), \tag{3.17}$$

because the aggregate resource allocation of each job cannot exceed the given allocation p_{k+1} of the next job ($(k+1)$-th job). Since (3.17) is non-decreasing with k,

Algorithm 2: Heuristic to Approximate DAMF Allocation

Input: A set of sites $\{S_1, S_2, ..., S_m\}$ with capacities $\{u_1, u_2, \ldots, u_m\}$ and transfer limits $\{c_1, c_2, \ldots, c_m\}$; a set of jobs $\{J_1, J_2, \ldots, J_n\}$ with data partition weights $\{w_1, w_2, \ldots, w_n\}$ and demand matrix D;

Output: An allocation matrix A; a transmission matrix ΔD;

$A = \mathbf{0}$;
$\Delta D = \mathbf{0}$;
$\vec{p} = \mathbf{0}$;
Sort all sites in increasing order of the number of jobs with demands on each site (i.e., the cardinality of $\{J_i : a_{ij} > 0\}$);
for *each site* S_j *in the sorted order* **do**
    Compute max-min fair allocation $\langle a_{1j}, a_{2j}, ..., a_{nj} \rangle$ on S_j based on job demands $\{d_{1j}, d_{2j}, ..., d_{nj}\}$, capacity u_j of S_j, and current aggregate allocation vector $\vec{p} = \langle p_1, p_2, ..., p_n \rangle$ as described in the main text;
    for $i \in [n]$ **do**
        $p_i = p_i + a_{ij}$;
$\mathcal{S}_{spare} = \{S_j \mid \sum_{i=1}^{n} a_{ij} < u_j\}$;
$\mathcal{S}_{full} = \{S_j \mid \sum_{i=1}^{n} d_{ij} > u_j\}$;
$\mathcal{J} = \{J_i \mid \sum_{j=1}^{m} d_{ij} > \sum_{j=1}^{m} a_{ij}\}$;
while $\mathcal{S}_{spare} \neq \emptyset \wedge \mathcal{S}_{full} \neq \emptyset \wedge \mathcal{J} \neq \emptyset$ **do**
    Remove all S_l's from $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ that $c_l \leq \min_{1 \leq i \leq n} w_i$ or $\sum_{i=1}^{n} (d_{il} + \Delta d_{il}) = u_l$;
    while $\mathcal{J} \neq \emptyset$ **do**
        Select job $J_x \in \mathcal{J}$ with the least aggregate allocation;
        if $\forall S_j \in \mathcal{S}_{full}, d_{xj} + \Delta d_{xj} \leq a_{xj}$ **then**
            Remove J_x from $\mathcal{J}$;
        else
            Select a site $S_y \in \mathcal{S}_{full}$ where $d_{xy} + \Delta d_{xy} > a_{xy}$;
            Select a site $S_z \in \mathcal{S}_{spare}$;
            $\Delta d_{xy} = \Delta d_{xy} - 1$;
            $\Delta d_{xz} = \Delta d_{xz} + 1$;
            $a_{xz} = a_{xz} + 1$;
            $c_y = c_y - w_x$;
            $c_z = c_z - w_x$;
return $A_{n*m}, \Delta D_{n*m}$;

we can use a binary search to find out which jobs will receive resource allocations on S_j to make full use of its capacity u_j. After deciding k, we solve the equation $\sum_{i=1}^{k}(\min\{C, p_i + d_{ij}\} - p_i) = u_j$ to find out the maximum aggregate resource allocation C among these jobs to achieve max-min fairness. Then, the i-th job will be allocated $a_{ij} = \min\{C, p_i + d_{ij}\} - p_i$ amount of resources. Apparently, the iterating order of sites has an impact on the final allocation result. Our idea is to sort all sites in increasing order of the number of jobs with demands on the site (line 3). A site demanded by more jobs is considered to have higher potential in balancing the final aggregate allocation, so we process it in a later iteration.

In step 2 (lines 10-30), we utilize the network resources to improve the allocation derived in step 1. It is not difficult to see that there exists a DAMF allocation (represented by a pair of transmission and allocation matrices) such that for every demand unit (one task) transferred, (i) the task involved is allocated compute resources at the destination site; and (ii) the compute resources at the source site are fully allocated. In fact, if a demand unit transferred does not satisfy (i), the transfer can be canceled without affecting the allocation matrix. Similarly, the transfer of a demand unit that violates (ii) can also be canceled and the task can be satisfied by compute resources at the source site. This would increase the compute resource allocation of the job involved without affecting the compute resource allocation of any other job, thereby contradicting that the original allocation is max-min fair.

Based on the observation above, we define two sets $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ to record sites with spare capacities and with more job demands than capacities (lines 10-11), and a set $\mathcal{J}$ to record jobs whose demands are not fully satisfied (line 12). Then, we start a loop to transfer job demands from $\mathcal{S}_{full}$ sites to $\mathcal{S}_{spare}$ sites until either $\mathcal{S}_{spare}$, $\mathcal{S}_{full}$ or $\mathcal{J}$ becomes empty (which implies that no further improvement to the allocation can be made). To make effective use of network resources, we would like all job demands transferred to get compute resource allocations at destination sites. Intuitively, a site will not need to participate in transmissions if its bandwidth is used up or its updated local demand equals its capacity. Thus, within the loop, we first remove all such sites from $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ (line 14). As a heuristic to approximate max-min fairness, among all jobs whose demands are not completely satisfied, we pick the job J_x with the least aggregate allocation currently (line 16). If J_x's demands are satisfied on all sites in $\mathcal{S}_{full}$ (implying that J_x's unsatisfied demands locates at the sites that have been removed because no more bandwidth

is available), we discard J_x and examine the next job with the least aggregate allocation for demand transmission (lines 17-18). Otherwise, we select the source site and destination site for a transfer of J_x: the source site can be any site in $\mathcal{S}_{full}$ where J_x's demand is not fully satisfied (line 20), and the destination site can be any site in $\mathcal{S}_{spare}$ (line 21). After deciding a transmission, we update the related matrices and remaining transfer limits (lines 22-26).

The heuristic algorithm does not require solving linear programs and has much lower time complexity. In step 1, it solves m allocation problems separately (where m is the number of sites). In the water-filling-like strategy, sorting takes $O(n \log n)$ time (where n is the number of jobs), and the binary search takes $O(n \log n)$ time as computing (3.17) takes $O(n)$ time. So, the time complexity of each iteration is $O(n \log n)$, and the total time complexity of step 1 is $O(mn \log n)$. In step 2, the algorithm transfers at most $cm/2 = O(m)$ units of job demand (assuming the transfer limits of all sites are a constant c). Updating $\mathcal{S}_{spare}$ and $\mathcal{S}_{full}$ takes $O(m)$ time. If we arrange all unsatisfied jobs in $\mathcal{J}$ as a min-heap based on their aggregate allocations, selecting the unsatisfied job with the least aggregate allocation takes $O(1)$ time. Selecting the source and destination sites takes $O(m)$ time. After the demand transfer, updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. Thus, each unit of demand transfer takes $O(m + \log n)$ time. If a source site for the selected job can always be found, the time complexity of step 2 is $O(m(m + \log n))$. If the selected job fails to find a source site for transmission, the job is discarded and updating the min-heap of $\mathcal{J}$ takes $O(\log n)$ time. In the worst case, discarding all such jobs incur a time complexity of $O(n(m + \log n))$. Therefore, the worst-case time complexity of step 2 is $O((m + n)(m + \log n))$. Hence, the overall complexity of the algorithm is $O(mn \log n + m^2)$. In the experiments, we shall empirically evaluate and compare the computation times of Algorithms 1 and 2.

3.4.5 Optimizing Job Completion Times

In addition to providing fair resource allocation from the global perspective, we can further optimize the job completion times. The key idea is that DAMF fixes only the aggregate resource allocation for each job, while the particular allocations at individual sites can still be adjusted. In this case, the completion time of a job can be optimized by assigning the allocation proportionally to its demand at

each site. For example, if a job has 2 and 4 tasks at two sites respectively and its aggregate allocation by DAMF is 3 computing slots, then its allocations at the two sites can be set to 1 and 2 computing slots, so that the two sites can finish processing their tasks at the same time (to minimize job completion time) if all tasks have the same processing time. Meanwhile, we need to ensure that other jobs can also achieve their DAMF aggregate allocations. The details of our algorithm are shown in Algorithm 3.

Algorithm 3: Optimizing Job Completion Times

Input: A set of sites $\{S_1, S_2, ..., S_m\}$ with capacities; a set of jobs $\{J_1, J_2, ..., J_n\}$ with demand matrix D; the aggregate allocation e_i of each job J_i in a DAMF allocation with corresponding transmission matrix ΔD

Output: An optimized allocation matrix A'

1 **for** $k = 1, 2, \ldots, n$ **do**
2 Solve the following linear program:;
3 Maximize y_k, subject to:;
4 $\forall j \in [m], \sum_{i=1}^{n} a_{ij} \leq u_j$;
5 $\forall i < k, j \in [m] : a_{ij} = a'_{ij}$;
6 $\forall i \geq k, j \in [m] : 0 \leq a_{ij} \leq d_{ij} + \Delta d_{ij}$;
7 $\forall i \geq k : \sum_{j=1}^{m} a_{ij} = e_i$;
8 $\forall j \in [m] : a_{kj} = d_{kj} * y_k + z_{kj}$, $y_k \geq 0$, $z_{kj} \geq 0$;
9 **for** *each* $j \in [m]$ **do**
10 $a'_{kj} \leftarrow a_{kj}$;
11 **return** A'_{n*m};

Algorithm 3 optimizes the completion times of jobs iteratively from J_1 to J_n. In each iteration, the algorithm solves a linear program (lines 4-10) to optimize the completion time of a job J_k. The first constraint of the linear program (line 6) ensures that the allocations are feasible. The second constraint (line 7) retains the allocations of jobs already optimized. The third constraint (line 8) ensures that the allocations of remaining jobs are rational given the input transmission matrix. The fourth constraint (line 9) guarantees that the aggregate allocations of these jobs comply with DAMF. The last constraint (line 10) expresses J_k's allocation a_{kj} at each site S_j as a linear combination $d_{kj} * y_k + z_{kj}$, where y_k is a shared proportional coefficient for all sites and z_{kj} is an auxiliary variable. Thus, by maximizing y_k, the resource allocation a_{kj} across sites can be made as close to proportional as possible to the resource demand $d_{kj} + \Delta d_{kj}$ at each site, thereby minimizing job

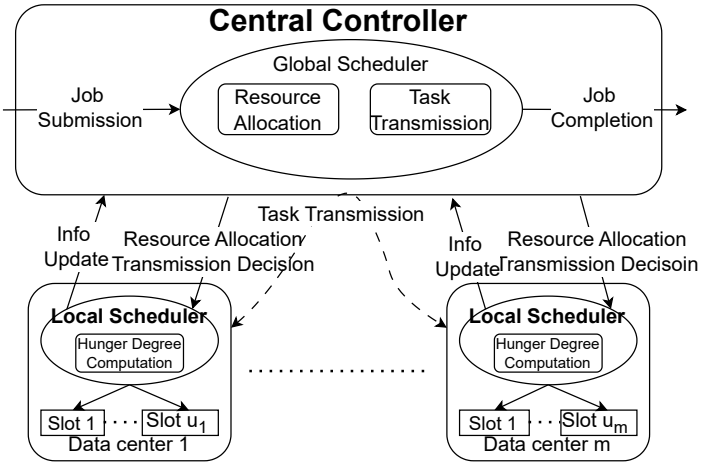


FIGURE 3.2: System Architecture

J_k's completion time. After solving the linear program, J_k's allocation at each site is decided and recorded in A' (lines 11-13). Finally, the algorithm returns the optimized allocation A' after all iterations. The set of jobs $\{J_1, J_2, ..., J_n\}$ input to the algorithm can be arranged in different orders based on job arrival times, job priorities, job demands, etc. In this paper, we sort the jobs based on their total demands from low to high so that jobs with smaller numbers of tasks are optimized earlier.

3.4.6 Deployment

Algorithms 1 and 2 in the above sections derive a DAMF or near-DAMF resource allocation among active jobs at a snapshot of the system. In this section, we discuss how to apply them to online scenarios where jobs arrive dynamically for execution. As shown in Figure 3.2, the algorithm for computing the DAMF allocation can be deployed at a central scheduler of the distributed job execution system. The scheduler tracks the real-time resource capacities of the sites and the distributed demands of each job on all sites. Once an allocation is derived, the scheduler propagates the transmission matrix ΔD and the allocation matrix A to relevant sites, where job demands (task input data) would be transferred based on ΔD. The compute resource allocation at each site would follow the allocation matrix A. If the demand of a job on a site is larger than its allocation on the site, some of its tasks will have to wait. When a computing slot becomes available on a site, the

TABLE 3.2: Dataset characteristics

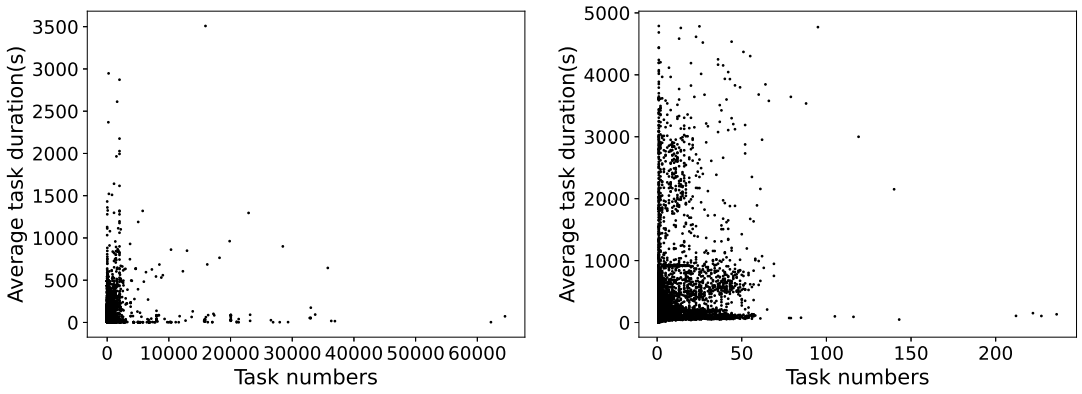
Dataset	Average task duration	Average task count per job
Alibaba	155.68 seconds	273.5 tasks
Google	512.25 seconds	4.5 tasks

site computes the "Hunger Degree" for jobs with pending tasks locally and selects the job with the lowest Hunger Degree to launch its task in the available slot. More specifically, the Hunger Degree of a job on a site is the ratio between the number of its tasks that are currently running (the number of computing slots occupied by this job now) and its DAMF allocation on this site (the number of computing slots that this job is supposed to occupy). A lower Hunger Degree indicates that the job is taking up less slots than expected and deserves more slot allocation. In particular, if a job's DAMF allocation at a site is zero, its Hunger Degree is set to infinity.

In distributed job execution, the demands of jobs change continuously because of job arrivals and task completions. In general, the completion of a single task would not significantly change the relative distribution of job demands and hence the resultant resource allocation [46]. Therefore, to reduce the computational overhead, allocations can be recomputed only when a new job arrives or all tasks of an existing job are completed on some site. Recall that in our model, there is a time threshold T which determines the limit c_j on the total demand that can be transferred into and out of each site S_j. This parameter T can be dynamically adjusted according to the job arrival rate. When new jobs arrive frequently, T can be shortened to align with the reduced time between recomputations; conversely, T can be lengthened if the job arrival rate decreases.

3.5 Experiments

Simulation experiments are conducted to compare the fairness and efficiency between various allocation policies.



(A) Alibaba Trace (B) Google Trace

FIGURE 3.3: Task characteristics of two datasets

3.5.1 Datasets

We use job traces from Alibaba [54] and Google [55]. The Alibaba trace contains information on approximately 16 million tasks collected from 1300 machines over a period of 12 hours in August 2017. The Google trace records events (e.g., a task is scheduled, completed or aborted) from Google's eight Borg cells over the month of May 2019. We pre-process the raw traces to extract key information, including the arrival time, task count and task durations of each job. From each trace, a continuous sequence of 15,000 jobs is randomly selected to serve as the dataset driving our simulations.

Figure 3.3 and Table 3.2 illustrate the task characteristics of both datasets, including the task number and mean task duration of each job, and the average task duration and task count of all jobs. It can be seen that the jobs in the Alibaba dataset generally have larger numbers of tasks and shorter average task durations, whereas the jobs in the Google dataset tend to have longer average task durations and significantly fewer tasks.

3.5.2 System Configuration

We simulate a distributed system of m sites, each with 32 computing slots. Each task is assumed to take up one computing slot to run and release it after completion. Due to the switching overhead consideration, we assume that there is no preemption during the execution of a task. For each job, the distribution of its tasks among

the sites is generated by a Zipf distribution: the numbers of tasks placed on the sites are proportional to $1, \frac{1}{2^\alpha}, \frac{1}{3^\alpha}, ..., \frac{1}{m^\alpha}$, where α is a Zipf parameter. A higher α implies a more skewed distribution (a higher proportion of tasks are concentrated at one site), whereas $\alpha = 0.0$ indicates a uniform distribution. In the experiments, we test four Zipf distributions with α set to 0.0, 0.5, 1.0 and 2.0 to simulate different workload situations. We shuffle the order of the sites when deploying each job's task distribution, so that the overall workload distribution of all jobs is roughly balanced among sites. We scale the job arrival times to simulate different levels of overall system workloads. We experiment with different numbers of sites and different workload levels, and observe similar performance trends. We implement the algorithms and run the experiments on a server with AMD EPYC 7H12 2.6GHz processor and 1024GB memory.

3.5.3 Resource Allocation Policies

Besides Algorithms 1 and 2 (called DAMF and DAMF_a for short), we also implement three baseline policies: IMF, AMF (both introduced in Section 3.2) and Dynamic Independent Max-min Fairness (DIMF). DIMF utilizes network resources on top of IMF: it first applies the IMF policy for resource allocation on each site, then transfers unsatisfied job demands to the sites with spare compute capacities to maximize the system utility (subject to the transfer limits), where jobs with lower aggregate allocations in the first step are assigned higher transmission priority in order to promote fairness. All the policies recompute the allocation matrix only when a new job arrives or an existing job completes all of its tasks on some site. We use CPLEX [56] as the program solver in the experiments. Since a task cannot be transferred fractionally, the transmission matrix ΔD derived to achieve the DAMF allocation may not be workable. To make DAMF applicable in practice, we enforce the elements of the transmission matrix ΔD in Algorithm 1 to be integers, i.e., replacing the linear program with a mixed-integer program. This would incur extra but acceptable computation time. We shall demonstrate the computation times of DAMF and DAMF_a later. Note that DAMF_a does not need to apply a similar modification because the transmission matrix ΔD derived from its heuristic is always integral.

As introduced in Section 3.1, transmission count limit c is computed based on the average data partition size S, available bandwidth B and time threshold T: $c = BT/S$. In the experiments, we set $T = 50$ seconds (which is small compared to the average task duration shown in Table 3.2), $S = 1$GB and $B = 160$Mbps or 480Mbps for all sites, thus $c = (160 \text{ or } 480) * 50/(1 * 1024 * 8) \approx 1$ or 3. Under these settings, a task transmission generally takes $50/(1 \text{ or } 3) = 50$ or 16.67 seconds to arrive at its destination. To emulate the bandwidth bottleneck being the connection between each site and the backbone network, task transmissions (potentially originating from different source sites) are received sequentially at each destination site. In recomputing the allocation and transmission matrices, the scheduler discards all pending transmissions (but retains the ongoing one) to release more bandwidth for the reallocation. For example, suppose $c = 3$ for all sites and a recomputation is triggered 30 seconds after the previous one. Suppose in the previous computation, it was decided that a site S_j would receive 3 tasks. In this case, the first task transmission would have arrived at site S_j 16.67 seconds after the previous computation, while the second task transmission is ongoing and the third one is pending. Then, in the recomputation, the third task transmission will be rescinded and the transmission count limit c_j available for S_j will be set to 2.

3.5.4 Performance Metrics

In Sections 3.3 and 3.4, we have proved that DAMF achieves several important fairness properties. Our experiments aim to further evaluate DAMF and DAMF_a's empirical performance from different perspectives. We study the following performance metrics: the deviation of the instantaneous resource allocation from ideal DAMF, the standard deviation of the instantaneous resource allocation among jobs, the average Jain's Index of instantaneous resource allocation, and the average job response time.

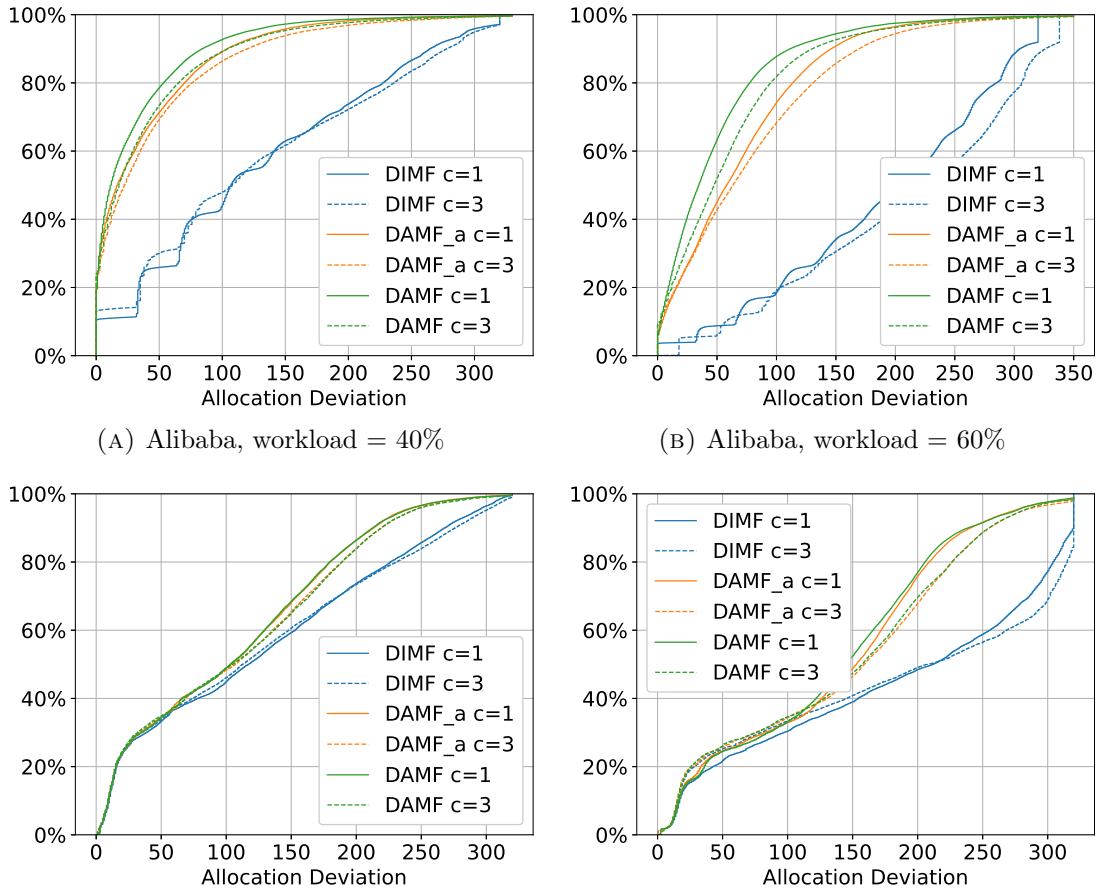
The deviation of the instantaneous resource allocation measures the instantaneous difference between the ideal DAMF allocation and the actual allocation of a policy. We continuously trace the number of computing slots actually occupied by each job throughout the simulation of different policies and compare it with the ideal DAMF allocation. More specifically, we first conduct the simulation using DAMF,

and at each recomputation within simulation, we record the output transmission and allocation matrices as well as the resultant aggregate allocation vector with timestamp, which is taken as the ideal DAMF allocation. We also track the actual aggregate allocation of DAMF by updating the actual aggregate allocation vector with timestamp when a computing slot is allocated to a job (to run a task) or released by a job (when a task is completed). Note that available computing slots are assigned by the Hunger Degree mechanism and there is no preemption. Thus, the actual instantaneous allocations (resp. aggregate allocations) may differ from the allocation matrices (resp. aggregate allocation vectors) output by recomputations. Next, we perform simulations of other policies and record their actual allocations as well. Finally, we compute and trace the allocation deviation between an actual allocation and the ideal DAMF allocation over the simulated time span. Given two aggregate allocation vectors $\vec{A} = \langle a_1, a_2, ..., a_n \rangle$. and $\vec{B} = \langle b_1, b_2, ..., b_n \rangle$, the allocation deviation is computed as $\sum_{i=1}^{n} |a_i - b_i|$. The allocations of completed jobs are treated as 0. A lower deviation indicates a closer instantaneous allocation to the ideal DAMF allocation. We plot the cumulative distribution of the deviation over the simulated time span. If the distribution curve is closer to the upper left corner, the allocation policy is considered to have better instantaneous fairness, because it has a lower deviation for a higher percentage of time.

The standard deviation of the instantaneous resource allocation among jobs is computed from the records of the actual aggregate allocation vectors of jobs. We exclude jobs that have completed or not yet arrived from the computation of standard deviation. A lower standard deviation naturally indicates a more balanced resource allocation among jobs. We plot the cumulative distribution of the standard deviation over the simulated time span for comparison. Again, allocation policies whose distribution curves are closer to the upper left corner are considered superior in terms of instantaneous fairness.

The Jain's Index is a quantitative measure used to assess the fairness or equity of resource allocation among n competing users or entities. The range of Jain's Index is $[\frac{1}{n}, 1]$, and a higher index indicates a more balanced allocation. Suppose each user i gets x_i resources, then Jain's Index can be computed as follows:

$$f(x_1, x_2, \ldots, x_n) = \frac{(\sum_{i=1}^{n} x_i)^2}{n * \sum_{i=1}^{n} x_i^2}.$$



(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%

(C) Google, workload = 40%

(D) Google, workload = 60%

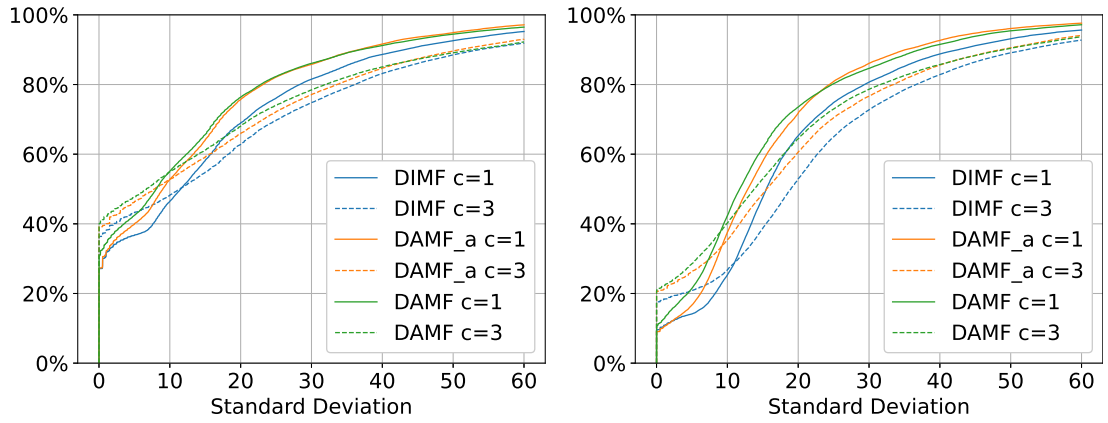
FIGURE 3.4: Cumulative distribution of allocation deviation, Zipf parameter $\alpha = 2$.

We compute the Jain's Index from the records of the actual aggregate allocation vectors of jobs, and calculate the average Jain's Index over the simulated time span for comparison.

The response time of a job is the time duration from its arrival to its completion. We use the average job response time to evaluate the long-term impact of allocation policies. A lower average job response time suggests a higher system efficiency.

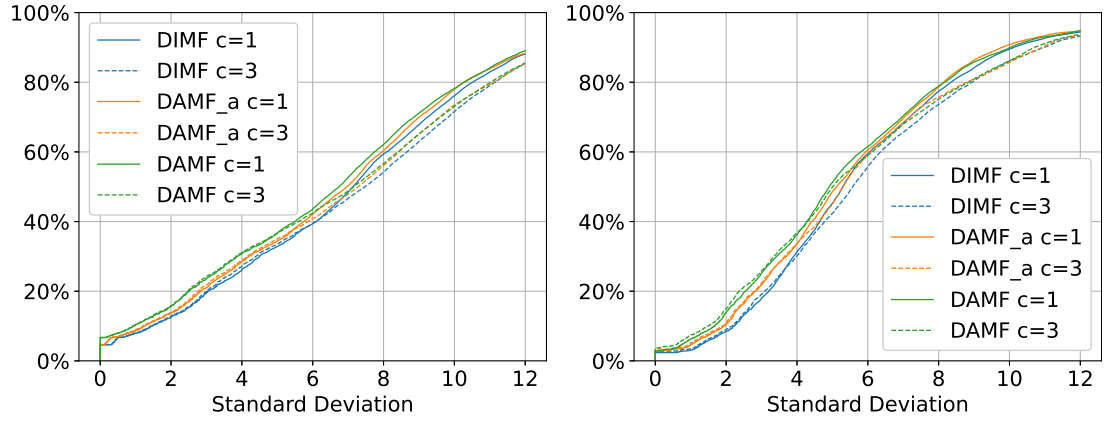
3.5.5 Experimental Results

Figure 3.4 shows the cumulative distribution of the allocation deviation from the ideal DAMF. Due to space limitations, we only present the results for the Zipf distribution of $\alpha = 2$. The results for other Zipf distributions show similar trends. Comparing DAMF and the baseline DIMF under the same transfer limit, it can be



(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%



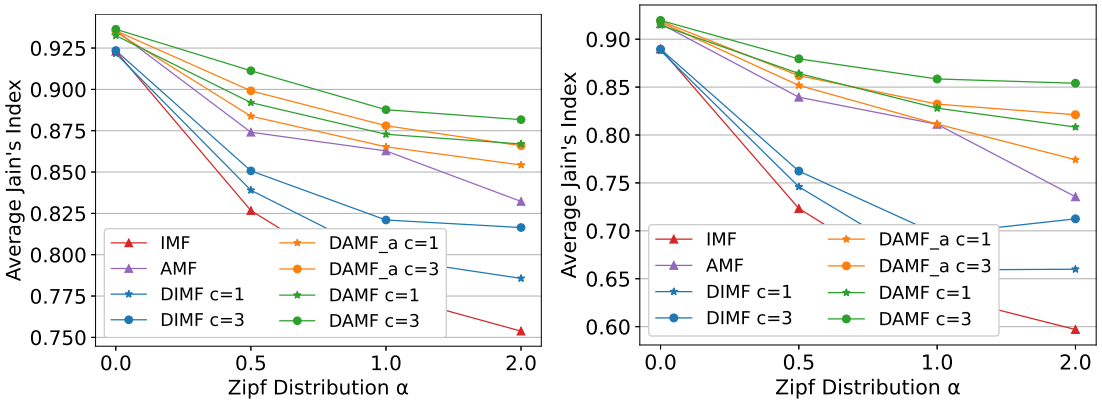
(C) Google, workload = 40%

(D) Google, workload = 60%

FIGURE 3.5: Cumulative distribution of standard deviation, Zipf parameter $\alpha = 2$

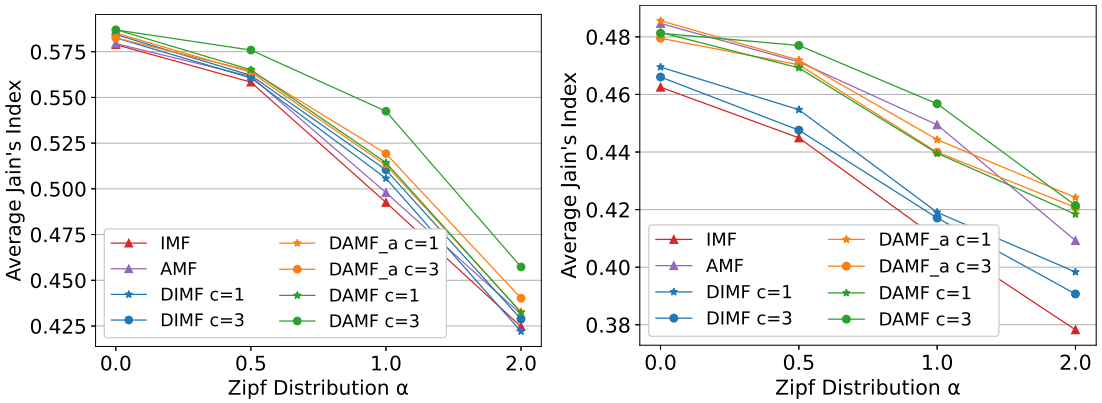
seen that DAMF's distribution curves are closer to the upper left corner, which indicates that DAMF achieves better instantaneous fairness than DIMF. In addition, the distribution curves of DAMF and DAMF_a are rather close to each other under the same transfer limit, demonstrating that DAMF_a is a close approximation of DAMF. We also record and compare the average computation time for recomputing allocation matrices during simulations. As shown in Table 3.3 and 3.4, DAMF_a reduces the average computation time by four orders of magnitude compared to DAMF (from the order of 10^{-1} seconds down to 10^{-5} seconds).

Comparing the results for the Alibaba and Google datasets under the same transfer limit, it can be seen that the Google dataset has much higher allocation deviations than the Alibaba dataset. This is mainly due to task durations: the Google dataset has much longer task durations than the Alibaba dataset as shown in Table 3.2. Since there is no preemption, an updated allocation matrix after recomputation



(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%



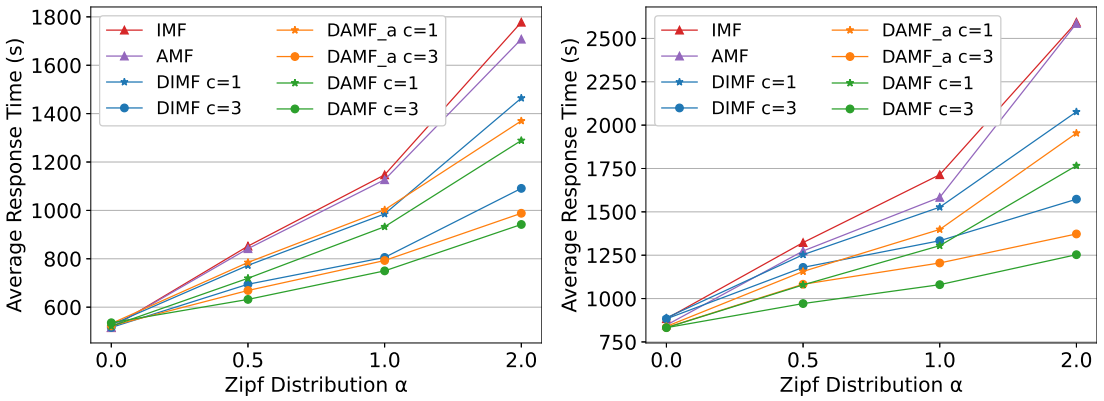
(C) Google, workload = 40%

(D) Google, workload = 60%

FIGURE 3.6: Average Jain's Index

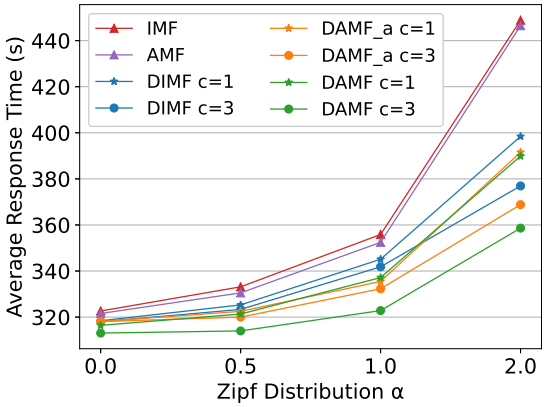
starts to take effect only after the computing slots are released (when the currently running tasks are completed) and reallocated to the intended jobs. Longer task durations make it slower for the actual allocation to converge to the allocation matrix computed.

Figure 3.5 shows the cumulative distribution of the standard deviation of the instantaneous resource allocation among jobs. Due to space limitations, we again only present the results for the Zipf distribution of $\alpha = 2$. Similar to the results in Figure 3.4, it can be seen that DAMF's distribution curves are closer to the upper left corner, which indicates that DAMF achieves more balanced instantaneous resource allocations than DIMF. Comparing DAMF and DAMF_a under the same transfer limit, we can see that their distribution curves are quite close to each other, which again shows that DAMF_a is a close approximation of DAMF. It can also be observed from Figure 3.5 that the standard deviation tends to increase with the transfer limit. This is because in DAMF, small-demand jobs are prioritized to

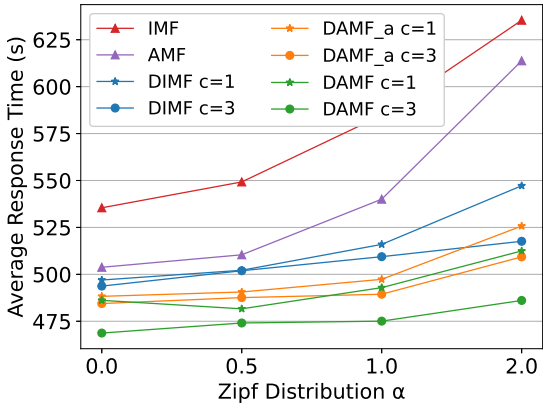


(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%



(C) Google, workload = 40%



(D) Google, workload = 60%

FIGURE 3.7: Average job response time

have their demands fully satisfied at their original sites, while large-demand jobs are more likely to have unsatisfied demands and tend to exploit idle compute resources at other sites by leveraging network resources. Therefore, transmissions often help large-demand jobs to obtain more resources while keeping the resources allocated to small-demand jobs unchanged, which leads to the rise of the standard deviation of the instantaneous resource allocation with increasing transfer limit.

Figure 3.6 shows the average Jain's Index. It can be seen that allocation policies that allow task transmissions (DAMF, DAMF_a, DIMF) have higher Jain's Indexes compared with their counterparts which do not (AMF, IMF). More importantly, DAMF achieves the highest index while DAMF_a is close to it, demonstrating that they effectively reduce the imbalance of resource allocation among jobs.

Figure 3.7 shows the average job response time for different datasets, system workloads and Zipf distributions. It can be seen that policies utilizing network resources

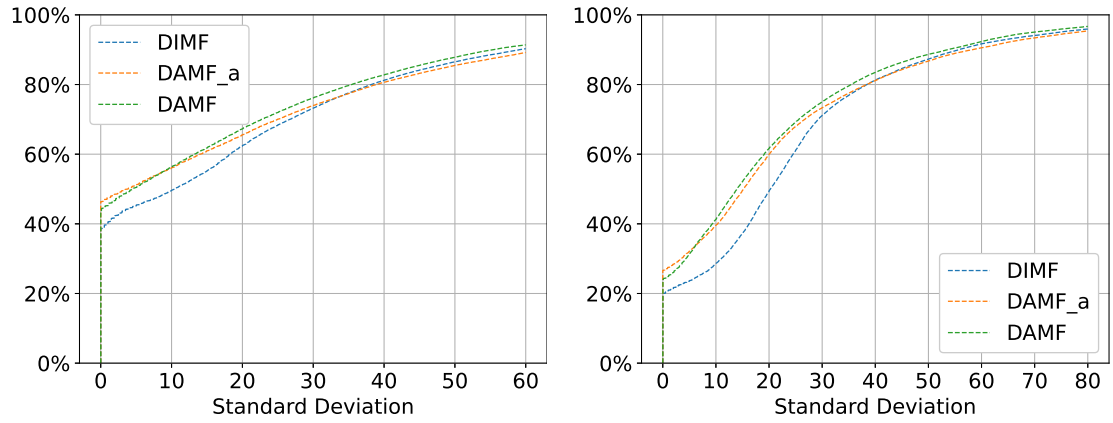
Table 3.3: Average computation time (s), Alibaba

		workload = 40%		workload = 60%	
c	α	DAMF	DAMF_a	DAMF	DAMF_a
1	0.0	0.0747	$1.005 * 10^{-5}$	0.1287	$1.402 * 10^{-5}$
	0.5	0.0792	$9.223 * 10^{-6}$	0.1661	$1.440 * 10^{-5}$
	1.0	0.0793	$9.409 * 10^{-6}$	0.1764	$1.453 * 10^{-5}$
	2.0	0.0841	$8.205 * 10^{-6}$	0.1709	$1.401 * 10^{-5}$
3	0.0	0.0709	$8.749 * 10^{-6}$	0.1497	$1.690 * 10^{-5}$
	0.5	0.0765	$8.395 * 10^{-6}$	0.1571	$1.351 * 10^{-5}$
	1.0	0.0761	$8.853 * 10^{-6}$	0.1551	$1.424 * 10^{-5}$
	2.0	0.0837	$1.051 * 10^{-5}$	0.1733	$1.316 * 10^{-5}$

Table 3.4: Average computation time (s), Google

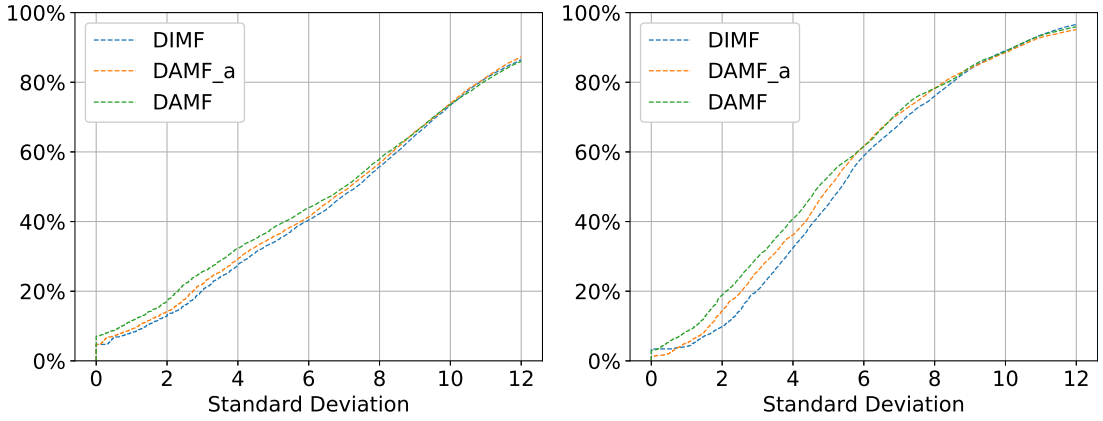
		workload = 40%		workload = 60%	
c	α	DAMF	DAMF_a	DAMF	DAMF_a
1	0.0	0.1029	$1.710 * 10^{-5}$	0.3783	$6.346 * 10^{-5}$
	0.5	0.1247	$1.836 * 10^{-5}$	0.3771	$6.084 * 10^{-5}$
	1.0	0.1298	$1.610 * 10^{-5}$	0.3952	$5.071 * 10^{-5}$
	2.0	0.1164	$1.686 * 10^{-5}$	0.3708	$3.817 * 10^{-5}$
3	0.0	0.1180	$1.508 * 10^{-5}$	0.4670	$7.205 * 10^{-5}$
	0.5	0.1400	$1.583 * 10^{-5}$	0.5175	$5.482 * 10^{-5}$
	1.0	0.1489	$1.496 * 10^{-5}$	0.5289	$5.178 * 10^{-5}$
	2.0	0.1796	$1.798 * 10^{-5}$	0.5465	$3.670 * 10^{-5}$

achieve lower average job response time, compared with IMF and AMF. The improvement typically increases with the transfer limit. Furthermore, since DAMF views the whole distributed system as a united resource pool and requires the aggregate allocation vector to be max-min fair among all possible transmissions and allocations, it outperforms DIMF and other allocation policies in most cases. The improvement is substantial even with only one bandwidth unit available, and it becomes more significant when the system workload is higher or the task distribution of jobs is more skewed among sites. In addition, DAMF_a has similar performance of average job response time to DAMF, demonstrating that it closely approximates the original DAMF.



(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%



(C) Google, workload = 40%

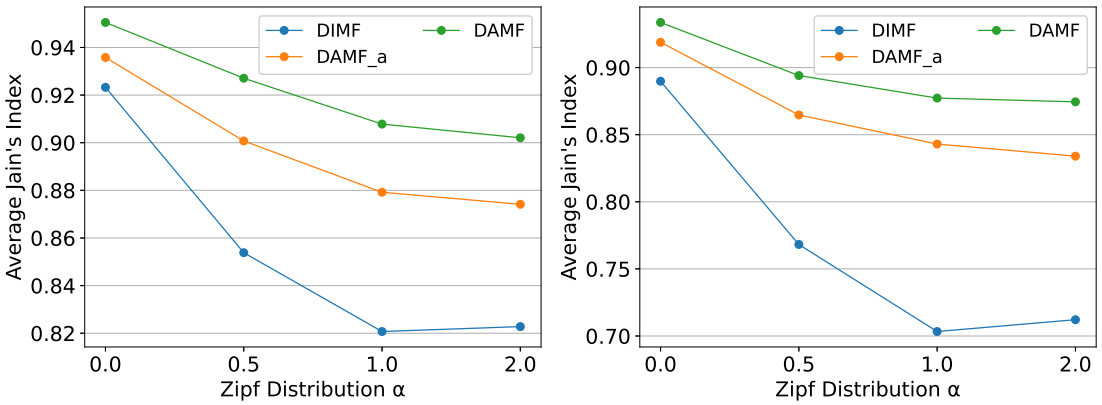
(D) Google, workload = 60%

FIGURE 3.8: Cumulative distribution of standard deviation under fluctuating bandwidth, Zipf parameter $\alpha = 2$

3.5.6 Evaluation on Robustness and Scalability

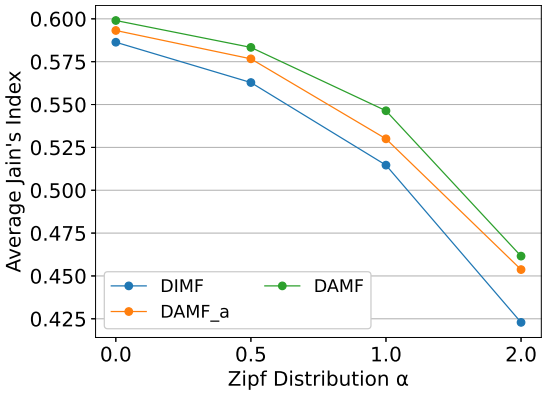
We also conduct experiments to evaluate the allocation policies under fluctuating bandwidth conditions. Specifically, we initialize an integer sequence randomly generated from a uniform distribution between 1 and 5 to simulate the changes of the transfer count limit c (which reflects the available bandwidth) over time. In the experiments, once the i-th job arrives, the value of c for all sites is changed to the i-th element in the sequence for recomputation. For fair comparison, the same sequence is used for evaluating all allocation policies. As shown in Figures 3.8, 3.9 and 3.10, the performance trends of different policies (that utilize network resources) remain similar to previous experiments. DAMF and DAMF_a again significantly outperform the baseline policy DIMF.

In addition, we evaluate the scalability of DAMF_a under larger numbers of sites

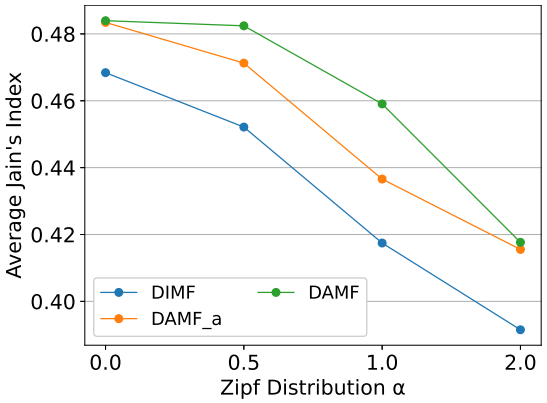


(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%



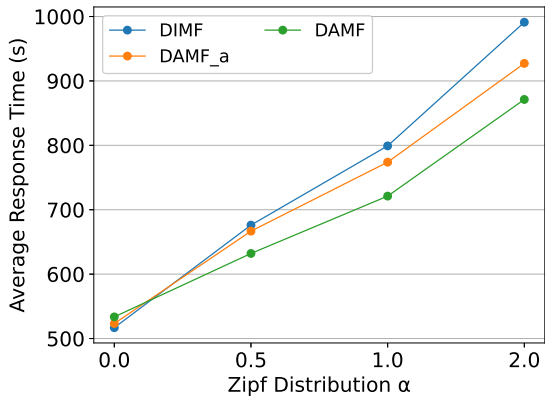
(C) Google, workload = 40%



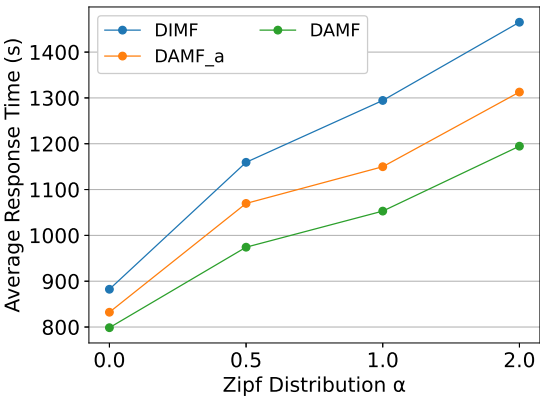
(D) Google, workload = 60%

FIGURE 3.9: Average Jain's Index under fluctuating bandwidth

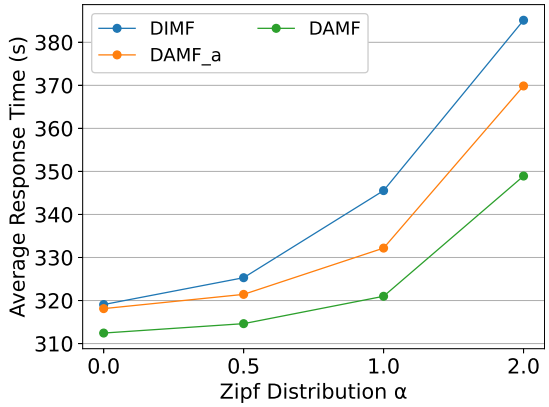
m. We set $m = 20$, 50 and 100 and conduct experiments for DAMF_a and DIMF. Since the average task count per job of the Google dataset is low (4.5 tasks per job), it is not suitable for evaluation with a large number of sites. Thus, we test with the Alibaba dataset only. Due to space limitations, for performance metrics other than the average computation time, we present only the results for the Zipf distribution of $\alpha = 1$ in Figures 3.11, 3.12, 3.13 and 3.14 (those for other α values show similar trends). These results again demonstrate that DAMF_a outperforms the baseline DIMF in terms of both resource allocation fairness and system utility. Table 3.5 shows the average computation times for different site numbers and Zipf distributions. These results demonstrate that DAMF_a maintains low computational overhead, thereby validating its potential for large-scale geo-distributed systems.



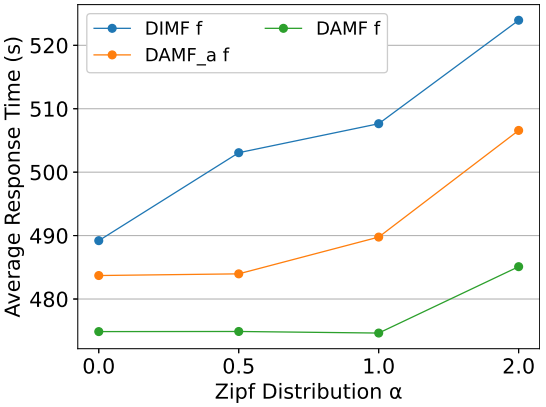
(A) Alibaba, workload = 40%



(B) Alibaba, workload = 60%

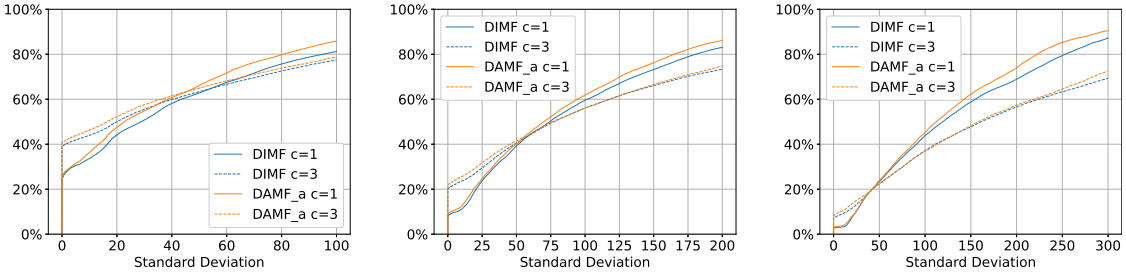


(C) Google, workload = 40%



(D) Google, workload = 60%

FIGURE 3.10: Average job response time under fluctuating bandwidth

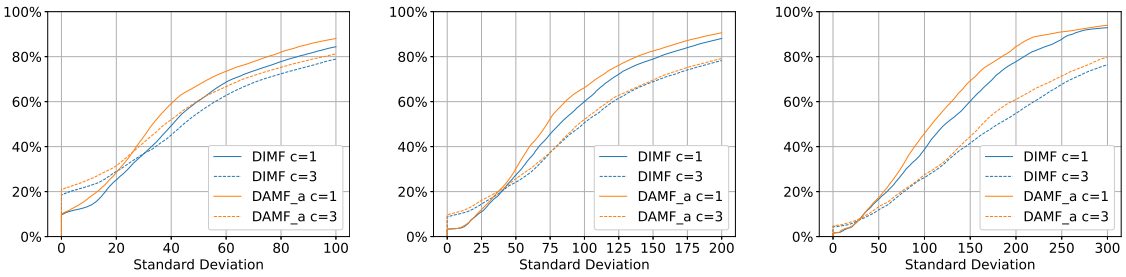


(A) $m = 20$ (B) $m = 50$ (C) $m = 100$

FIGURE 3.11: Cumulative distribution of standard deviation, Alibaba, workload = 40%, Zipf parameter $\alpha = 1$

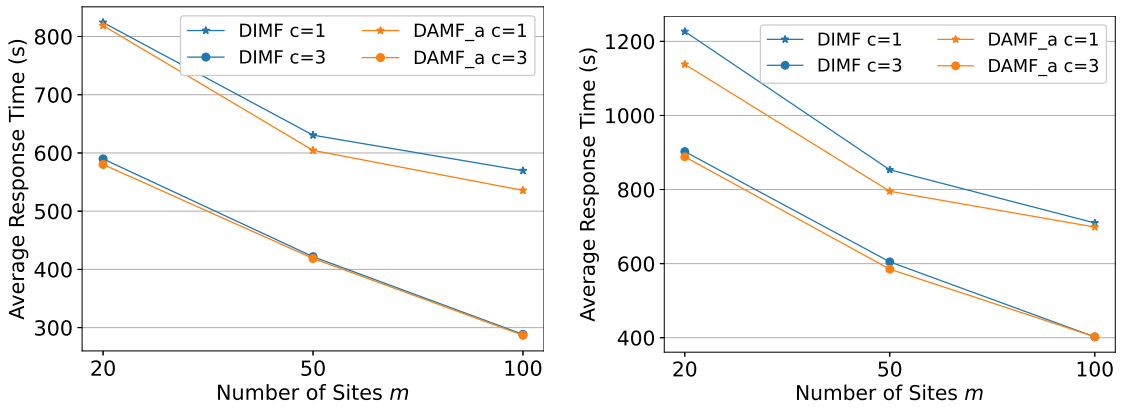
3.6 Summary

In this chapter, we have defined a new resource allocation policy called Dynamic Aggregate Max-min Fairness (DAMF) in distributed job execution with task transmissions. We prove that DAMF satisfies several widely accepted properties, including Pareto efficiency, envy-freeness and strategy-proofness. We present an



(A) $m = 20$ (B) $m = 50$ (C) $m = 100$

FIGURE 3.12: Cumulative distribution of standard deviation, Alibaba, workload = 60%, Zipf parameter $\alpha = 1$



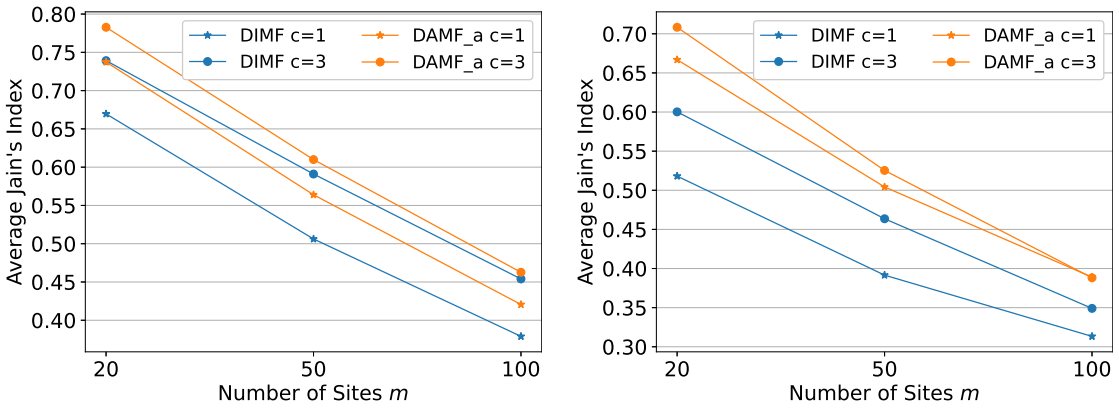
(A) Alibaba, workload = 40% (B) Alibaba, workload = 60%

FIGURE 3.13: Average job response time, Zipf parameter $\alpha = 1$.

TABLE 3.5: Average computation time (s) of DAMF_a

c	α	Alibaba, workload = 40%			Alibaba, workload = 60%		
		$m = 20$	$m = 50$	$m = 100$	$m = 20$	$m = 50$	$m = 100$
1	0.0	2.035×10^{-5}	4.908×10^{-5}	8.455×10^{-5}	3.732×10^{-5}	7.321×10^{-5}	1.594×10^{-4}
	0.5	2.231×10^{-5}	4.491×10^{-5}	7.954×10^{-5}	3.339×10^{-5}	6.835×10^{-5}	1.592×10^{-4}
	1.0	1.645×10^{-5}	4.084×10^{-5}	7.899×10^{-5}	2.517×10^{-5}	6.675×10^{-5}	1.439×10^{-4}
	2.0	1.586×10^{-5}	3.561×10^{-5}	8.723×10^{-5}	2.243×10^{-5}	5.335×10^{-5}	1.219×10^{-4}
3	0.0	2.169×10^{-5}	4.678×10^{-5}	9.114×10^{-5}	3.644×10^{-5}	7.452×10^{-5}	1.639×10^{-4}
	0.5	2.195×10^{-5}	5.027×10^{-5}	1.062×10^{-4}	3.089×10^{-5}	7.945×10^{-5}	2.174×10^{-4}
	1.0	1.836×10^{-5}	5.408×10^{-5}	1.137×10^{-4}	3.035×10^{-5}	9.177×10^{-5}	2.428×10^{-4}
	2.0	1.935×10^{-5}	5.155×10^{-5}	1.443×10^{-4}	2.786×10^{-5}	9.714×10^{-5}	2.868×10^{-4}

algorithm for computing the DAMF allocation and a fast approximation. Experimental evaluations using real job traces from Alibaba and Google demonstrate that DAMF could effectively enhance the fairness of resource allocation among jobs and improve the system utility.



(A) Alibaba, workload = 40%

(B) Alibaba, workload = 60%

FIGURE 3.14: Average Jain's Index, Zipf parameter $\alpha = 1$.

Chapter 4

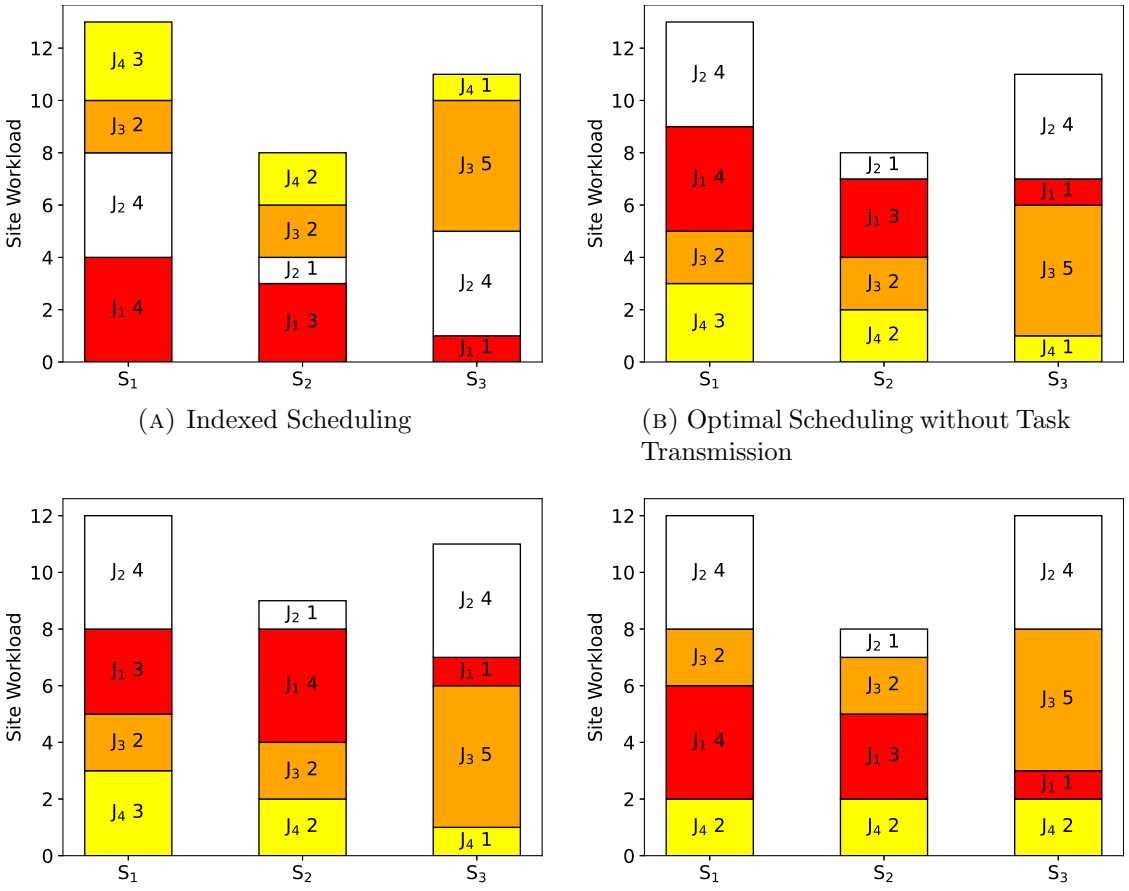
Task Scheduling and Transmission for Distributed Job Execution[1]

In this chapter, we study the task scheduling and transmission for improving job processing efficiency in distributed job execution. The problem includes two primary hurdles: how to compute the global job execution order so that the jobs in the system are executed efficiently, and how to assign task transmissions across the system based on the bandwidth constraints to further improve performance. We focus on the online setting where jobs arrive dynamically for execution. Our goal is to minimize the average job response time, which is the average time span from every job's arrival to its completion.

4.1 A Motivating Example

Fig. 4.1 provides a motivating example of the problem with four jobs J_1, J_2, J_3 and J_4 in three geo-distributed data centers (sites) S_1, S_2 and S_3. All jobs are submitted at time 0. For simplicity, we assume that each task of any job requires one computing slot and takes one time unit to process, and each site has one computing slot only. Fig. 4.1a shows the initial task distribution of each job among the three sites. The response time of a job is decided by the completion time of its last task across all sites. Obviously, different execution orders of the jobs can change the completion times of their tasks and result in different job response times.

[1]This chapter is based on our paper published in IEEE ICDCS 2026.



(A) Indexed Scheduling

(B) Optimal Scheduling without Task Transmission

(C) Optimal Transmission under Execution Order of Fig. 4.1b

(D) Optimal Scheduling with Task Transmission

FIGURE 4.1: Motivating Example

If the jobs are simply scheduled in the order of their indices (which, for example, represents their submission order), jobs J_1, J_2, J_3 and J_4 will complete at time 4, 8, 10 and 13, respectively, so the total job response time is $4+8+10+13 = 35$ time units. The optimal job execution order without task transmission is $\langle J_4, J_3, J_1, J_2 \rangle$ as shown in Fig. 4.1b, and the resultant total job response time is $3+6+9+13 = 31$ time units, which is 4 time units better than the first schedule.

Suppose that with utilization of network resources, the input data of one task is allowed to be transferred between sites and it takes one time unit to arrive at the destination. Under the job execution order of Fig. 4.1b, the optimal transmission can decrease the total job response time to 29 time units. For example, we can transfer one task of J_1 from site S_1 to S_2, as shown in Fig. 4.1c. This task will arrive at site S_2 at time 1 when the slot of S_2 is processing a task of J_4 (and J_4 is scheduled earlier than J_1 in the execution order), so the task transferred can be

treated as a local task of J_1 at site S_2 initially. Consequently, J_4, J_3, J_1 and J_2 will complete at time 3, 6, 8 and 12, respectively, and the total job response time is $3+6+8+12=29$ time units. In our example, the optimal solution is to transfer one task of J_4 from site S_1 to S_3 while modifying the job execution order to be $\langle J_4, J_1, J_3, J_2\rangle$, as shown in Fig. 4.1d. This results in the total job response time of $2+6+8+12=28$ time units, which is even better than the schedule in Fig. 4.1c.

The above example illustrates some interesting phenomena. First, job execution order and task transmission have mutual influence. The optimal execution order of jobs can change after task transmissions are performed: both execution orders in Figs. 4.1b and 4.1d achieve the optimal total job response times given the respective task transmissions, but J_1 and J_3 are swapped in the execution order after a task is transferred in Fig. 4.1d. Vice versa, different job execution orders have different optimal task transmissions: the optimal transmission changes after we modify the job execution order from $\langle J_4, J_3, J_1, J_2\rangle$ (Fig. 4.1c) to $\langle J_4, J_1, J_3, J_2\rangle$ (Fig. 4.1d). Second, the transmission in the optimal solution is from site S_1 to S_3, while the site with the lowest load is S_2. This indicates that the transmissions that minimize the total job response time can differ from the one that focuses on load balancing, so traditional load balancing techniques may not be preferable in this efficiency-targeted scenario.

4.2 System Model

Consider a system consisting of m separate geo-distributed sites: $\mathcal{S} = \{S_1, S_2, ..., S_m\}$. Each site S_j has compute resources with capacity u_j, which can be interpreted as the volume of computing slots available. A set of n jobs $J_1, J_2, ..., J_n$ are submitted to run in the system. Each job J_i consists of tasks that require distinct data partitions and can run in parallel. For simplicity, we assume that each task takes one computing slot to process. In big data processing frameworks, tasks of the same job typically process data partitions of the same size [57, 58], so they would take similar times to process. We denote the processing time of a task in each job J_i as l_i, which can be estimated by the historical execution records of the job or sampling-based learning, which proactively samples a small fraction of the tasks in the job to make an estimate [59]. A job is completed when all of its tasks are completed.

TABLE 4.1: Summary of Notations

Notation	Definition
$\mathcal{S} = \{S_1, S_2, \ldots, S_m\}$	Set of sites
$U = \{u_1, u_2, \ldots, u_m\}$	Compute resource capacities of individual sites
$\{J_1, J_2, \ldots, J_n\}$	Set of jobs
$D_{n\times m}$	Compute resource demands
$B_{m\times m}$	Network resource availability
$\{l_1, l_2, \ldots, l_n\}$	Task processing times of individual jobs
$\{r_1, r_2, \ldots, r_n\}$	Data partition sizes of individual jobs

The data partitions required by the tasks of each job are originally distributed among the sites. Due to data locality consideration, each task is initially restricted to be executed at a certain site where its input data is available. Therefore, the computational demand of a job J_i on site S_j can be modeled as the number of its tasks that can run only on S_j initially. We represent the computational demands of all jobs across all sites as a matrix $D_{n\times m}$, where each element d_{ij} denotes job J_i's computational demand on site S_j.

Suppose the sites in the system are connected by networks, which can be used to transfer data and hence tasks from one site to another. The network resources in the system is denoted by a matrix $B_{m\times m}$, where each element b_{ij} represents the bandwidth available on the link from site S_i to site S_j. b_{ij} is set to 0 if there is no direct link between S_i and S_j. Let r_k denote the size of a data partition of job J_k, then a task of J_k takes r_k/b_{ij} time to transfer from site S_i to site S_j (if $b_{ij} = 0$, this time is defined to be infinite). For simplicity, we assume that the transmissions on each link are carried out sequentially. That is, if there is an ongoing transmission on the link, a new transmission has to wait and its arrival time at the destination site will be delayed.

We study the problem of deriving an execution order of jobs and a set of task transmissions between sites, so that the total job response (completion) time of all jobs is minimized. The key notations used in this paper are summarized in Table 4.1. For simplicity, we use $[x]$ to represent the set $\{1, 2, \ldots, x\}$ for a positive integer x. Note that although the above model is formulated in a static setting, it can be easily extended to address online scenarios by dynamically capturing snapshots of job demand distributions and system resources.

4.3 Workload-Aware Selection with Transmission

4.3.1 Optimizing the Completion Time of a Single Job

We start by considering the subproblem of how to utilize network resources in the system to minimize the completion time of a single job. This subproblem can be formulated as follows: Given some workloads and task transmissions already scheduled in the system, and a job J_k with task processing time l_k, data partition size r_k and initial demand distribution $d_{k1}, d_{k2}, \ldots, d_{km}$ across the m sites, we would like to determine a set of task transmissions for J_k so that its job completion time can be minimized.

We use a sequence of arrays $W_1, W_2, \ldots, W_m$ to represent the workload already scheduled for each computing slot at each site, where the workload of a slot refers to the total processing time of all tasks assigned to it (i.e., the time when this slot will become available). Each W_i is an array whose length is the number of computing slots at site S_i, and each element w_{ij} in W_i represents the scheduled workload of the j-th slot at S_i. In addition, we use a matrix $T_{m\times m}$ to represent the usage of network resources by the task transmissions already scheduled, where t_{ij} records the time when the link from site S_i to site S_j will complete the scheduled transmissions and become available.

To avoid the complex interaction between task transmissions and executions and simplify the problem model, we assume that every transferred task will arrive at its destination site before any computing slot at that site becomes available. This ensures that the transferred tasks can be processed continuously at their destination sites together with the tasks originally available there, which simplifies the modeling of job completion time. Based on this, we model the completion time of job J_k as follows. Suppose a matrix $X_{m\times m}$ represents the task transmissions of job J_k, where each element x_{ij} refers to how many tasks of J_k are to be transferred from site S_i to site S_j. Naturally, each x_{ij} is greater than or equal to 0. In particular, for each site S_i, we define $x_{ii} = 0$. We use an array Y of length m to represent the computational demands at the m sites after all the transmissions in X are performed. Each element y_i refers to the number of tasks to be executed at site S_i

after the transmissions, and is given by:

$$\forall i \in [m], \quad y_i = d_{ki} + \sum_{j=1}^{m} x_{ji} - \sum_{j=1}^{m} x_{ij}. \tag{4.1}$$

Recall that we assume each task transmission to arrive at its destination site before any computing slot at that site becomes available. Thus, all the y_i tasks of job J_k to be executed at site S_i can be scheduled by iteratively assigning each task to the slot of the lowest workload in S_i. Let $w_{ij}^{(t)}$ denote the workload of the j-th slot in site S_i after t tasks of J_k are assigned to the slots at S_i. $w_{ij}^{(t)}$ can be computed iteratively as follows:

$$\forall j \in [u_i], \;\; w_{ij}^{(t)} = \begin{cases} w_{ij}, & t = 0 \\ w_{ij}^{(t-1)} + l_k, & j = \text{argmin}_{p \in [u_i]}\, w_{ip}^{(t-1)}, \\ w_{ij}^{(t-1)}, & j \neq \text{argmin}_{p \in [u_i]}\, w_{ip}^{(t-1)}, \end{cases} \tag{4.2}$$

where in each iteration, a task is assigned to the slot of the lowest workload ($\text{argmin}_{p \in [u_i]}\, w_{ip}^{(t-1)}$) and its workload then increases by l_k, while other slots are kept unchanged. Ties are broken arbitrarily if multiple slots have the same lowest workload. Based on this, the time when all y_i tasks complete execution at site S_i can be estimated as a function $f(W_i, y_i)$:

$$f(W_i, y_i) = \begin{cases} 0, & y_i = 0, \\ \min_{j \in [u_i]}(w_{ij}^{(y_i - 1)}) + l_k, & y_i > 0, \end{cases}$$

which gives the completion time of the last task at S_i. Note that function f's inputs and outputs cannot be captured by any closed-form expression. The objective of optimizing the job completion time of J_k (i.e., $\max_{i \in [m]} f(W_i, y_i)$ — the completion time of its last task across all sites) can be achieved by searching for the best task transmissions X between sites, which decide the computational demands Y at different sites and hence job J_k's completion time.

It is essential to determine the bounds of X before starting the search. An excessively tight bound can limit the potential benefit of task transmissions in reducing the job completion time, whereas an excessively loose bound may give rise to task transmissions that fail to reach the destination sites in time, thereby introducing

additional delays in the job completion time. Suppose x_{ij} tasks are to be transferred from site S_i to site S_j. Then, the completion time of the last task will be at least $t_{ij} + x_{ij} \cdot r_k/b_{ij} + l_k$ (when it is processed immediately after arrival), and we require this last task to finish before the job completion time:

$$\forall i \in [m],\ j \in [m],\ t_{ij} + \frac{x_{ij} \cdot r_k}{b_{ij}} + l_k \leq \max_{i \in [m]} f(W_i, y_i). \tag{4.3}$$

In addition, the number of tasks transferred out of a site cannot exceed the number of tasks it holds initially, which imposes another upper bound on $\sum_{j=1}^{m} x_{ij}$ at each site S_i:

$$\forall i \in [m], \quad 0 \leq \sum_{j=1}^{m} x_{ij} \leq d_{ki}. \tag{4.4}$$

Constraints (4.3) and (4.4) collectively define the upper bounds of the elements in X. Therefore, we can define a program to search for task transmissions X under constraints (4.1), (4.3) and (4.4) to optimize the job completion time $\max_{i \in [m]} f(W_i, y_i)$. If multiple transmission matrices X's produce the same minimum job completion time, we choose the X with the least total number of task transmissions. The rationale is to minimize the network resource usage and make more network resources available to other jobs.

4.3.2 Transformation into ILP

The program in Section 4.3.1 cannot be solved directly by popular optimization solvers owing to the discrete function f, which cannot be captured by any closed-form expression. In order to make the program solvable, we transform it into solving multiple Integer Linear Programs (ILP) that are supported by popular solvers. To facilitate presentation, we refer to the completion time of the last task at a site S_i as "the completion time at site S_i". We compute and record the completion times $f(W_i, y_i)$ at each site S_i using the method introduced in Section 4.3.1, for increasing number of tasks y_i scheduled at site S_i starting from $y_i = 0$. The calculation ends at the lowest y_i value where the resulting $f(W_i, y_i)$ at site S_i exceeds the job completion time when all tasks are executed locally without transmission, i.e., $\max_{i \in [m]} f(W_i, d_{ki})$.

Note that these $f(W_i, y_i)$ values include all possible completion times at each site that J_k can produce, and they are non-decreasing with respect to y_i since adding a new task can never reduce the completion time at a site. With these $f(W_i, y_i)$ values, given any target job completion time z, the maximum number of tasks c_i that each site S_i can process (to keep its completion time within this target) can be derived as $c_i = \max\{p \mid f(W_i, p) \leq z\}$. Thus, the target z can be converted to an array of upper bounds $c_1, c_2, \ldots, c_m$ for the elements $y_1, y_2, \ldots, y_m$ in Y. Hence, the problem of determining "whether the job J_k can achieve a target job completion time z using task transmissions" can be transformed to determining whether task transmissions can be performed to bound the elements in $y_1, y_2, \ldots, y_m$ by $c_1, c_2, \ldots, c_m$. The latter can be formulated as an ILP, because its constraints are linear and the elements in X and Y are integers. Furthermore, the minimum job completion time that J_k can achieve must be among all the $f(W_i, y_i)$ values. Therefore, we can sort all values $f(W_1, 0)$, $f(W_1, 1), \ldots, f(W_1, c_1), f(W_2, 0)$, $f(W_2, 1), \ldots, f(W_2, c_2), \ldots, f(W_m, 0), f(W_m, 1), \ldots, f(W_m, c_m)$ to form an array ψ. Then, the minimum job completion time of J_k can be found by a binary search in ψ and solving multiple ILPs.

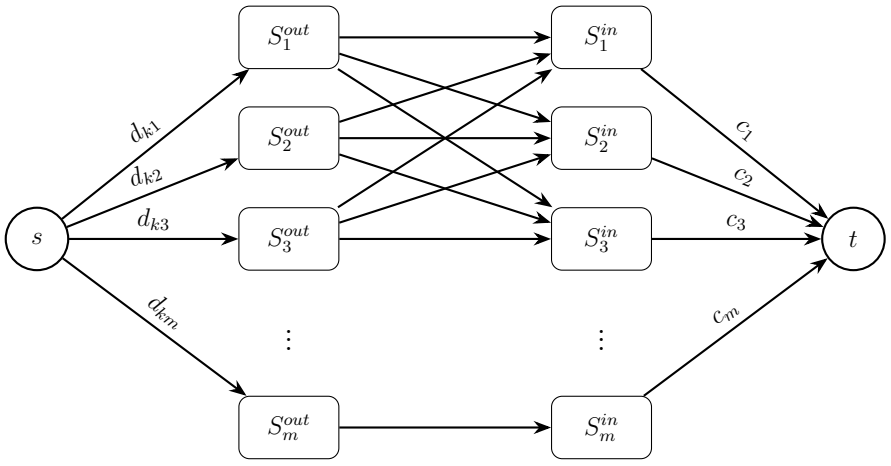


FIGURE 4.2: The flow network

Algorithm 4 presents the details of optimizing the job completion time of job J_k. $\psi.size$ represents the number of elements in the array ψ. The algorithm first computes and records the possible completion times $f(W_i, y_i)$ at each site S_i using the method introduced in Section 4.3.1. These values are then merged and sorted to form a new array ψ which records the job completion times that J_k can possibly achieve. Next, a binary search is applied to the array ψ to find the optimal job completion time in ψ as well as the transmissions required. The parameters *low*

Algorithm 4: Optimizing the Completion Time of a Single Job

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B; a job J_k to be scheduled with task processing time l_k, data partition size r_k and computational demand distribution $d_{k1}, d_{k2}, \ldots, d_{km}$; current workload $W_1, W_2, \ldots, W_m$ and network usage T;

Output: J_k's optimal job completion time and task transmissions;

Compute and record $f(W_i, y_i)$'s for each site S_i with increasing y_i until $f(W_i, y_i) \geq \max_{i \in [m]} f(W_i, d_{ki})$, using the method in Section 4.3.1;
Merge and sort all $f(W_i, y_i)$ values to form an array ψ;
$low = 1$; $high = \psi.size$; $opt = 0$;
while $low < high$ **do**
    $middle = \lfloor (low + high)/2 \rfloor$;
    $c_i = \max\{p \mid f(W_i, p) \leq \psi[middle]\}$, for each $i \in [m]$;
    Minimize $\sum_{i=1}^{m} \sum_{j=1}^{m} x_{ij}$, subject to:;
        $\forall i \in [m], j \in [m],\ t_{ij} + \frac{x_{ij} * r_k}{b_{ij}} + l_k \leq \psi[middle]$;;
        $\forall i \in [m],\ 0 \leq \sum_{j=1}^{m} x_{ij} \leq d_{ki}$;;
        $\forall i \in [m],\ y_i = d_{ki} + \sum_{j=1}^{m} x_{ji} - \sum_{j=1}^{m} x_{ij}$;;
        $\forall i \in [m],\ y_i \leq c_i$;;
        all x_{ij}'s and y_i's are non-negative integers;;
    if *the above ILP is feasible and admits a solution* **then**
        $opt = \psi[middle]$;
        $X \leftarrow x_{ij}$'s in the solution of the above ILP;
        $high = middle - 1$;
    else
        $low = middle + 1$;
return opt, X;

and *high* in the algorithm refer to the boundary indices of the binary search. In each iteration, the *middle* index and bounds $c_1, c_2, \ldots, c_m$ are updated, and the feasibility of an ILP is checked to decide whether the job completion time $\psi[middle]$ is achievable.

4.3.3 Transformation into Minimum-cost Flow Problem

Algorithm 4 solves the problem, but its computational complexity could raise concerns. ILP is NP-hard in general. If the solution space becomes huge (e.g., the job has thousands of tasks unevenly spread across tens of geo-distributed sites), solving the ILP can incur intolerable computational overhead. To improve the algorithm's

efficiency, we further transform the ILP problem in lines 8-13 of Algorithm 4 to a minimum-cost flow problem in a bipartite flow network shown in Fig. 4.2. The flow network has a source node s, a sink node t, a set of m nodes on the left side and a set of m nodes on the right side. To facilitate exposition, we use S_i^{out} and S_i^{in} to represent the i-th node on the left and right sides, respectively. A set of m edges with cost 0 connects the source node s to the m nodes on the left, and the capacity of each edge (s, S_i^{out}) is set to d_{ki}. In addition, a set of $m \times m$ edges connects the m nodes on the left to the m nodes on the right. The capacity of each edge (S_i^{out}, S_j^{in}) where $i \neq j$ is set to the upper bound of x_{ij} given in line 9 of Algorithm 4 (where $\psi[middle]$ is the target job completion time) and the cost of the edge is set to 1. The capacity of each edge connecting each pair of nodes with the same index (S_i^{out}, S_i^{in}) is set to d_{ki} (the initial computational demand at site S_i) and the cost of the edge is set to 0. Finally, a set of m edges with cost 0 connect the m nodes on the right to the sink node t, and the capacity of each edge (S_i^{in}, t) is set to c_i (the upper bound of y_i for the target job completion time). The problem is to decide whether the bipartite flow network can transfer all $\sum_{i=1}^{m} d_{ki}$ units of flows from s to t under the given edge capacity constraints while minimizing the total cost.

Each edge (s, S_i^{out}) represents the initial number of tasks d_{ki} at site S_i. Each edge (S_i^{out}, S_j^{in}) where $i \neq j$ denotes the task transmissions from site S_i to site S_j. Due to the flow conservation constraint, the sum of flows going out of each node S_i^{out} is d_{ki}, ensuring that the number of task transmissions out of site S_i does not exceed d_{ki}. Each edge (S_i^{out}, S_i^{in}) denotes the number of local tasks that are not transferred out of site S_i, so it refers to no transmission and the cost is set to 0. Each edge (S_i^{in}, t) represents the final number of tasks to be executed at site S_i after transmissions, which is required to fall below the upper bound c_i. It is easy to verify that this minimum-cost flow problem is equivalent to the ILP problem in Section 4.3.2. Since all the edge capacities in the flow network are integers, based on the Integral Flow Theorem [60], the solution (if it exists) is made up of integral units of flows on all edges. Existing algorithms (e.g., Successive Shortest Path Algorithm, Cost Scaling Algorithm [61]) for solving the minimum-cost flow problem in bipartite graphs have polynomial time complexities. They can solve the problem more efficiently than ILP.

4.3.4 Optimizing the Completion Times of All Jobs

We have solved the problem of minimizing the completion time of a single job given the workloads and task transmissions already scheduled. We now consider how to deal with the problem when multiple jobs are submitted to run in the system. Since how a single job will utilize the network resources has been solved, the key problem that remains is how to decide an efficient job execution order. Inspired by the work from [26], we develop a Workload-Aware Selection with Transmission (WAST) algorithm, which aims to minimize the total job completion time by scheduling job execution and task transmission progressively across the system. In each iteration, WAST traverses all unscheduled jobs, searches for task transmissions for each job to minimize its completion time (accounting for the workloads and task transmissions already scheduled), and selects the job with the earliest possible completion time to add to the execution order.

Algorithm 5: Workload-Aware Selection with Transmission

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B; a set of jobs $J_1, J_2, \ldots, J_n$ with task processing times $l_1, l_2, \ldots, l_n$ and data partition sizes $r_1, r_2, \ldots, r_n$; a computational demand matrix D;

Output: Job execution order and task transmissions;

$\mathcal{J} = \{J_1, J_2, ..., J_n\}$;
Scheduled workload $W_i = \mathbf{0}$, for each $i \in [m]$;
Network usage $T = \mathbf{0}$;
while $\mathcal{J} \neq \emptyset$ **do**
    for *each job* $J_k \in \mathcal{J}$ **do**
        Call Algorithm 4 on J_k given W and T to obtain J_k's optimal completion time and the corresponding task transmissions X^k;
    Find the job $J_{min} \in \mathcal{J}$ of the earliest completion time;
    Append J_{min} to the execution order and save its task transmissions X^{min};
    $t_{ij} = t_{ij} + x_{ij}^{min} \cdot r_{min}/b_{ij}$, for each $i, j \in [m]$;
    $y_i = d_{ki} + \sum_{j=1}^{m} x_{ji}^{min} - \sum_{j=1}^{m} x_{ij}^{min}$, for each $i \in [m]$;
    for *each* $i \in [m]$ **do**
        for *each count* $\in [y_i]$ **do**
            $j = \text{argmin}_{p \in [u_i]} w_{ip}$;
            $w_{ij} = w_{ij} + l_{min}$;
    $\mathcal{J} = \mathcal{J} \setminus \{J_{min}\}$;
return Job execution order and each job's task transmissions;

Algorithm 5 shows the details of WAST. Initially, there is no scheduled workload or task transmission, and the set of unscheduled jobs $\mathcal{J}$ contains all jobs (line 3). In each iteration, the algorithm computes the optimal completion time of each job in $\mathcal{J}$ using Algorithm 4 with ILP transformed into solving the minimum-cost flow problem (lines 5-6). Then, the job J_{min} with the earliest completion time is selected and added to the job execution order, and its task transmissions X^{min} are saved (lines 7-8). Meanwhile, the scheduled workload W and network usage T are updated based on those arising from J_{min}, and J_{min} is removed from $\mathcal{J}$ (lines 9-14). The algorithm terminates when all jobs are scheduled and returns the job execution order and task transmissions of each job.

Additional optimizations can be applied to enhance the algorithm's computational efficiency. We can reduce the number of binary search operations for each job by decreasing the number of elements to consider in the array ψ (the job completion times that the job can possibly achieve). First, duplicate elements can be removed from ψ. Next, suppose that infinite bandwidth is available on the links between all site pairs so that any task transmission is instant. In this case, the best strategy that minimizes job completion time is to treat all m sites as one aggregate site and iteratively assign each task to the slot with the lowest workload globally. The job completion time derived in this way is guaranteed to be no worse than the optimal solution in the case of finite bandwidth, so it is a lower bound on the optimal job completion time. As a result, all elements in ψ below this lower bound can be eliminated from consideration. Furthermore, in each iteration, suppose that job J_{min} has the minimum completion time t^* among all jobs examined before examining an unscheduled job J_i. If the above lower bound of J_i's completion time is higher than or equal to t^*, there is no need to examine J_i in this iteration because it cannot achieve an earlier job completion time than t^*. Otherwise, if the lower bound of J_i's completion time is lower than t^*, t^* can be set as an upper bound when searching in J_i's array ψ of possible job completion times, since we are only concerned about whether J_i can achieve an earlier job completion time than t^*. In summary, when examining an unscheduled job J_i, we can remove all elements in ψ that fall below the lower bound of its completion time or exceed t^*.

Algorithm 6: Recursive Approximation of Optimizing the Completion Time of a Single Job

Input: A set of sites $\mathcal{S}$ with compute capacities U and network resources B; a job J_k to be scheduled with task processing time l_k, data partition size r_k and computational demand distribution $d_{k1}, d_{k2}, \ldots, d_{km}$; current workload $W_1, W_2, \ldots, W_m$ and network usage T;

Output: J_k's job completion time and transmissions;

Copy current workload: $W'_i = W_i$, for each $i \in [m]$;
$X = \mathbf{0}$;
$q = 0$;
for *each* $i \in [m]$ **do**
    for *each count* $\in [d_{ki}]$ **do**
        $j = \text{argmin}_{p \in [u_i]} w_{ip}$;
        $w_{ij} = w_{ij} + l_k$;
        $q = \max\{q, w_{ij}\}$;
while *true* **do**
    Find a task whose completion time $= q$;
    Suppose the task is currently assigned to the a-th slot of site S_g;
    if *site* S_g *has ever received* J_k*'s tasks* **then**
        return q, X;
    $q^* = q$, $h = -1$;
    for *each* $i \in [m]$ ***and*** $i \neq g$ **do**
        if $t_{gi} + r_k / b_{gi} \leq \min_{p \in [u_i]} w_{ip}$ ***and*** $\min_{p \in [u_i]} w_{ip} + l_k < q^*$ **then**
            $q^* = \min_{p \in [u_i]} w_{ip} + l_k$;
            $h = i$;
    if $h == -1$ **then**
        return q, X;
    else
        $x_{gh} = x_{gh} + 1$;
        $t_{gh} = t_{gh} + r_k / b_{gh}$;
        $w_{ga} = w_{ga} - l_k$;
        $v = \text{argmin}_{p \in [u_h]} w_{hp}$;
        $w_{hv} = w_{hv} + l_k$;
        $q = \max_{i \in [m]} f(W'_i, d_{ki} + \sum_{j=1}^{m} x_{ji} - \sum_{j=1}^{m} x_{ij})$;

4.3.5 Recursive Approximation

Algorithm 5 attempts to minimize the average job completion time in a step-by-step manner by solving multiple minimum-cost flow problems. We also present an alternative recursive approximation method called WAST_a which provides better computational efficiency. The outer loop of WAST_a is the same as WAST, and we only present the inner loop in Algorithm 6 which optimizes the completion time of a single job J_k. For simplicity, WAST_a assumes that, a site can either receive J_k's tasks from other sites or send J_k's tasks to other sites, but not both. Under this assumption, each task will be transferred at most once, because the destination site of any transmission will not send any task out. WAST_a first computes the completion time of J_k when all its tasks are executed locally without transmission as $q = \max_{i\in[m]} f(W_i, d_{ki})$ (line 8). Then, WAST_a starts a loop, where in each iteration it locates a task whose completion time is equal to q and tries to transfer it to another site to bring its completion earlier. The transmission is again required to arrive at the destination site before any computing slot at that site becomes available. In the case that multiple sites satisfy the requirement, the algorithm selects the one where the transferred task can be completed the earliest as the destination site (lines 14-18). After the task transmission is identified, the scheduled workload W, network usage T and J_k's job completion time q and task transmissions X are updated accordingly (lines 22-27). Then, the algorithm moves onto the next iteration to locate another task whose completion time is equal to the new job completion time and look for its possible transmission. If the task whose completion time equals q is at a site that has received J_k's tasks in previous iterations (line 12) or cannot find a destination site to have its completion time reduced (line 19), the algorithm terminates and returns all the transmissions identified in previous iterations and the job completion time q (lines 13 and 20). Note that this algorithm is not guaranteed to produce the optimal completion time for job J_k because it searches for task transmissions one at a time. However, it is easier to implement and takes much less time to compute the task transmissions.

4.3.6 Complexity Analysis

The outer loop of the WAST algorithm (Algorithm 5) has n iterations, in which it computes the optimal job completion times of all remaining jobs in $\mathcal{J}$ (which

decreases by 1 in each iteration) using Algorithm 4. Therefore, Algorithm 5 calls Algorithm 4 for a total of $n(n+1)/2$ times. With ILP transformed into a minimum-cost flow problem, Algorithm 4 solves multiple minimum-cost flow problems to derive a job's optimal completion time, and the number of problems to be solved depends on the job's task number, task distribution, data partition size as well as network resources available. Consider the worst case that a job J_k's tasks concentrate at a single site initially and the network resources are adequate to ensure that these tasks can be transferred to any other site in time. In this case, all the tasks can be executed at any site, and the total number of possible job completion times for J_k is $F \cdot m$ (where F is the number of tasks in J_k), which requires $\log(F \cdot m)$ rounds in the binary search. The time complexity to solve the minimum-cost flow problem depends on the algorithm applied. Our flow network has $2m+2$ nodes and m^2+2m edges. The Successive Shortest Path Algorithm (using a binary heap) has the worst-case computational complexity of $O(m^3 \log m)$ [61]. Therefore, assuming that all jobs have the same task number F, the worst-case computational complexity of Algorithm 5 is $O(n(n+1)/2 \cdot \log(F \cdot m) \cdot m^3 \log m) = O(n^2 m^3 \log F \log^2 m)$.

The outer loop of the approximation algorithm WAST_a is the same as that of Algorithm 5. In each iteration, WAST_a calls Algorithm 6 to optimize the completion time of every remaining job in $\mathcal{J}$. Algorithm 6 first places the tasks of the job locally, and then iteratively searches for destination sites to transfer tasks whose completion times are equal to the current job completion time. In the worst case, all tasks of the job are to be transferred to other sites to improve the job completion time (for example, all tasks of the job concentrate at one site initially whose scheduled workload is already the highest among all sites), and each task traverses all sites to find the best destination site, so the worst-case computational complexity is $O(F \cdot m)$. Therefore, if Algorithm 5 calls Algorithm 6 instead of Algorithm 4, the worst-case complexity is $O(n(n+1)/2 \cdot F \cdot m) = O(n^2 F m)$. Since no minimum-cost flow problem needs to be solved, this computational complexity is much lower than using Algorithm 4. We shall empirically evaluate the performance of these two algorithms in the experiments.

4.3.7 Deployment

In this section, we discuss how to apply the algorithms introduced above to online scenarios where jobs arrive dynamically for execution. The algorithm can be deployed at a central scheduler of the geo-distributed system. The scheduler keeps track of the system's resource capacities (including the compute resources and the network resources) as well as the demand distribution of each job across all sites. Once the job execution order and the task transmissions of each job are derived, the scheduler propagates them to relevant sites. Whenever a computing slot becomes available at a site S_i, S_i selects the first job in the job execution order that has pending tasks locally and assigns the slot to one task of this job. For each link from a source site to a destination site, the task input data of jobs are transferred based on the task transmission decisions. The transmissions follow the job execution order, so that the tasks of the jobs to be executed earlier are transferred earlier. The central scheduler recomputes the job execution order and the task transmissions of jobs whenever a new job arrives. To do so, it takes a snapshot of the pending task distributions of all jobs as their computational demand distributions. The recomputation derives a new job execution order and new task transmissions for operation. Existing scheduled transmissions not yet completed are withdrawn, and the network usage T is reset to $\mathbf{0}$ for each recomputation.

4.4 Experiments

Simulation experiments using read-world job traces are conducted to evaluate the performance of applying our scheduling algorithms to online scenarios as discussed in Section 4.3.7.

4.4.1 Datasets

We use cluster job traces sourced from the Alibaba Cluster Trace Program [54], which contains cluster traces of different years from real production. We pre-process the raw trace files of years 2017 and 2018, extract the key information including the arrival time, task number, average memory consumption and task processing times of each job. A continuous sequence of 10,000 jobs is randomly selected from

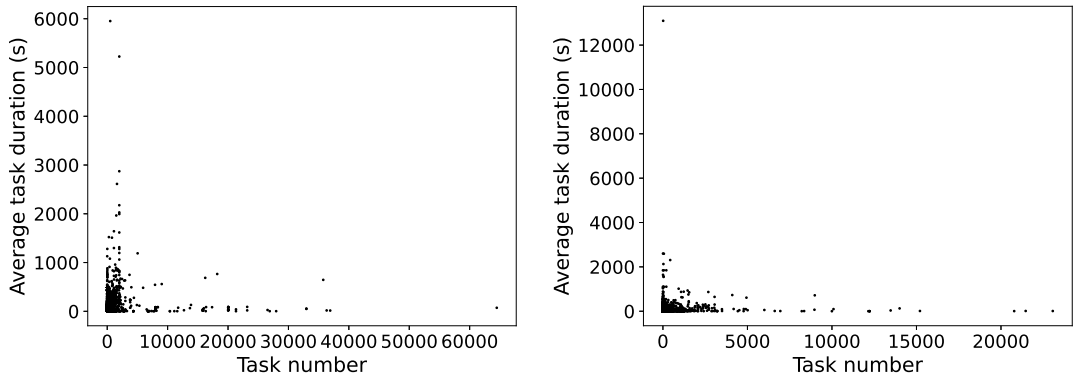
Characteristics	Alibaba 2017	Alibaba 2018
Average task number per job	266.01	94.70
Average task processing time (s)	129.14	82.41
Average data partition size (MB)	661.15	453.31

TABLE 4.2: Job characteristics of two datasets

each trace to serve as the dataset driving our simulations. The job characteristics of selected sequences are shown in Fig. 5.1 and Fig. 4.4. Both sequences exhibit a heavy-tailed distribution, in which a small subset of jobs accounts for a disproportionately large share of the total computation load. We take the average memory consumption of each job as its data partition size. The average memory consumption records in the data traces are normalized values with respect to the memory capacity of the servers in the cluster. We derive the exact data partition size assuming that each server has 128GB RAM, under which the largest data partition is about 4GB. The average task number per job, average task processing time and average data partition size of all jobs are shown in Table 4.2.

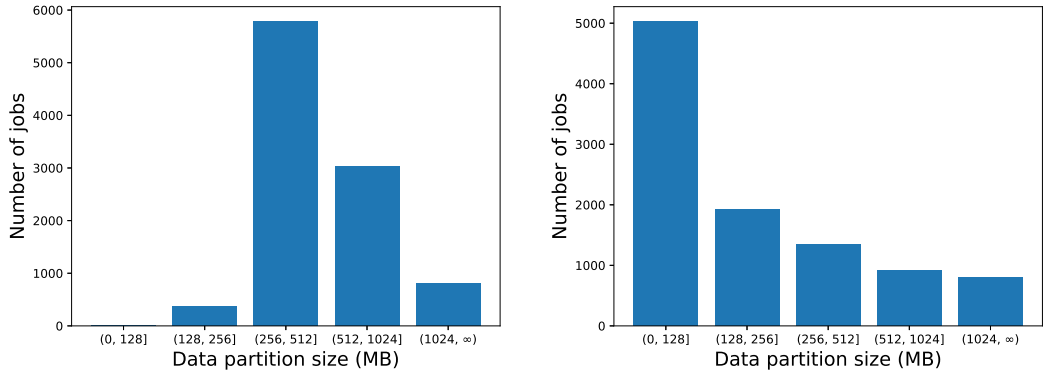
4.4.2 System Configuration

We simulate a distributed system of m sites, each with the same number of computing slots. Each task is assumed to take up one computing slot to run and release it after completion. Due to the switching overhead consideration, we assume that the system does not allow preemption during task execution. For each job, the initial distribution of its tasks among the sites is generated by a Zipf distribution: the



(A) Alibaba 2017 (B) Alibaba 2018

FIGURE 4.3: Job characteristic distribution of two datasets



(A) Alibaba 2017 (B) Alibaba 2018

FIGURE 4.4: Data partition size distribution of two datasets

numbers of tasks placed at the m sites are proportional to $1, \frac{1}{2^\alpha}, \frac{1}{3^\alpha}, ..., \frac{1}{m^\alpha}$, where α is a Zipf parameter. A higher α implies a more skewed distribution (a higher proportion of tasks are concentrated at fewer sites), whereas $\alpha = 0$ indicates a uniform distribution. In the experiments, we test four Zipf distributions with α set to 0, 0.5, 1 and 2 to simulate different task distributions of jobs. After the task distribution is generated, we shuffle the order of the sites when deploying each job's task distribution, so that the total workload at each site is roughly balanced. We scale the arrival times of the jobs in the datasets to simulate different levels of overall system utilization. We assume that the system is fully connected, i.e., a direct link exists between any two sites. The available bandwidth is set to 24Mbps, 48Mbps or 96Mbps for all links. We experiment with different numbers of sites, different numbers of computing slots per site, and different levels of system utilization. The results show similar performance trends. Due to space limitations, we focus on presenting the results for $m = 10$ sites, 32 computing slots per site, and system utilization = 50%, 60%, 70% and 80%.

4.4.3 Scheduling Policies

In Section 4.3, we have introduced the WAST algorithm and its fast approximation WAST_a. For comparison, we also simulate the following baseline algorithms.

- **First-Come-First-Serve (FCFS)**: FCFS simply schedules jobs in the order of their arrival times. When a computing slot becomes available, it is assigned to the job that arrives the earliest and has pending tasks.

- **Shortest Remaining Processing Time (SRPT)**: SRPT schedules jobs based on their estimated completion times in the case that each job runs alone in the system. More specifically, it estimates the completion time of a job J_k as $\max_{j\in[m]}\lceil d_{kj}/u_j\rceil \cdot l_k$, and executes pending jobs based on this estimation from low to high.

- **Workload-Aware Greedy Scheduling (SWAG)** [26]: SWAG greedily schedules jobs iteratively by estimating their completion times based on scheduled workloads without using network resources to transfer tasks between sites. Our WAST algorithm degenerates to SWAG when network resources are not available in the system.

- **GEODIS** [32]: GEODIS models job scheduling in geo-distributed data centers as a joint optimization of task placement and data access. It employs a heuristic scheduling algorithm which is triggered when any data center completes its scheduled workload. The objective of scheduling is to minimize the makespan of job execution (the latest job completion time) by selectively migrating data over faster network links when local servers are overloaded.

In the experiments, we use OR-Tools [62] to solve the minimum-cost flow problems in WAST. All algorithms (except FCFS) take the average task processing time of each job as input for scheduling and assume that the processing times of all tasks of the job are equal to the average. The simulation, on the other hand, uses the real processing times of individual tasks recorded in the trace to derive the job response time.

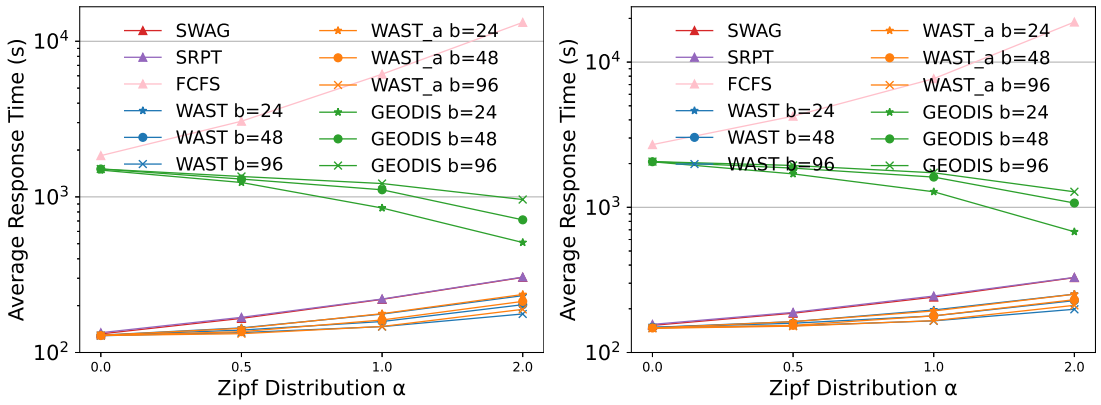
4.4.4 Experimental Results

Fig. 4.5 shows the performance of all algorithms in terms of average job response time. Due to space limitations, we only present the results for the Alibaba 2018 dataset (the results for the Alibaba 2017 dataset are similar). Note that the y-axis is plotted on a logarithmic scale. The results indicate that the average job response times of FCFS and GEODIS are one to two orders of magnitude higher than the other algorithms. This is because FCFS does not optimize job execution order and GEODIS is primarily designed to minimize the makespan.

Figs. 4.6 and 4.7 zoom into the performance of the remaining algorithms for both datasets. Compared to SRPT and SWAG, the average job response time of WAST and WAST_a is reduced by up to 53.3% for the Alibaba 2017 trace and 42.1% for the Alibaba 2018 trace. The improvement becomes more pronounced when the task distribution of jobs across sites is more skewed and when higher bandwidth is available, as more network resources can be effectively utilized to mitigate the imbalance in the task distribution of jobs under these conditions. Moreover, our WAST algorithm performs better for the Alibaba 2017 trace than the Alibaba 2018 trace. This is because the higher average task number per job in Alibaba 2017 leads to more tasks accumulated at some particular sites under highly uneven task distributions, resulting in increased response times if task transmission is not conducted. This, in turn, allows the WAST algorithm to offload more tasks from peak-load sites, enhancing the advantage of WAST. In addition, Figs. 4.6 and 4.7 show that although WAST_a compromises the quality of scheduling and performs slightly worse than WAST, it still significantly outperforms the baseline algorithms.

Figs. 4.8 and 4.9 show the 30th, 60th, 90th percentiles and the highest job response times for both datasets, under the Zipf distribution with $\alpha = 1$. For each percentile, the longest job response time among algorithms is normalized to 1, and the job response times of other algorithms are expressed relative to it. It can be seen that jobs in the lower percentiles have little improvement when the WAST and WAST_a algorithms are applied, since these jobs are small and have already achieved nearly best response times under the SRPT and SWAG strategies which favor small jobs. On the other hand, WAST and WAST_a can significantly enhance the performance of large jobs whose response times are relatively longer by transferring their tasks to mitigate their imbalanced task distributions. The highest job response time can be improved by more than 70%, as shown in Fig. 4.8.

Tables 4.3 and 4.4 show the average computation time per recomputation of the SWAG, WAST and WAST_a algorithms under various bandwidth settings, slot numbers per site, Zipf parameters α and system utilization. Due to space limitations, we present the results for the Alibaba 2018 trace under 60% and 80% utilization only. Both tables demonstrate that WAST_a effectively reduces the computation time compared with WAST, which can be up to 87.3%. Furthermore, it can be seen that the average computation time of WAST is about an order of magnitude higher than that of SWAG, while the average computation



(A) utilization = 50%

(B) utilization = 60%

(C) utilization = 70%

(D) utilization = 80%

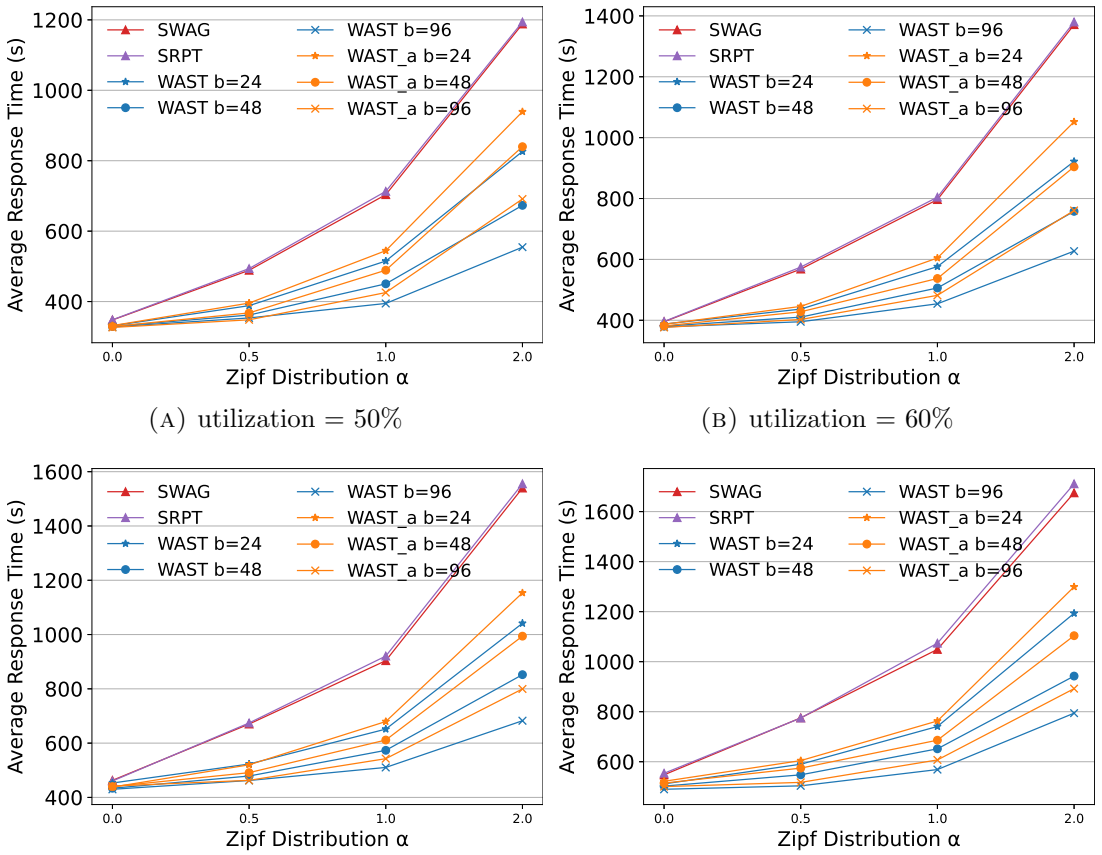
FIGURE 4.5: Average Job Response Time Compared with GEODIS, Alibaba 2018

time of WAST_a is of the same order of magnitude as SWAG, demonstrating the effectiveness of WAST_a in practice.

4.5 Discussion

Heterogeneous tasks. In real-world workloads, a job may consist of tasks with heterogeneous processing times and data sizes. Our proposed model and algorithms are capable of handling heterogeneous tasks by replacing the parameters l_k and d_k with task-specific values. Although this complicates the modeling of computation and transmission process, it does not compromise the effectiveness of our proposed algorithms.

Limited network resources. Inter-datacenter bandwidth is often limited and costly compared to intra-datacenter links. Therefore, the scheduler must avoid



(A) utilization = 50%

(B) utilization = 60%

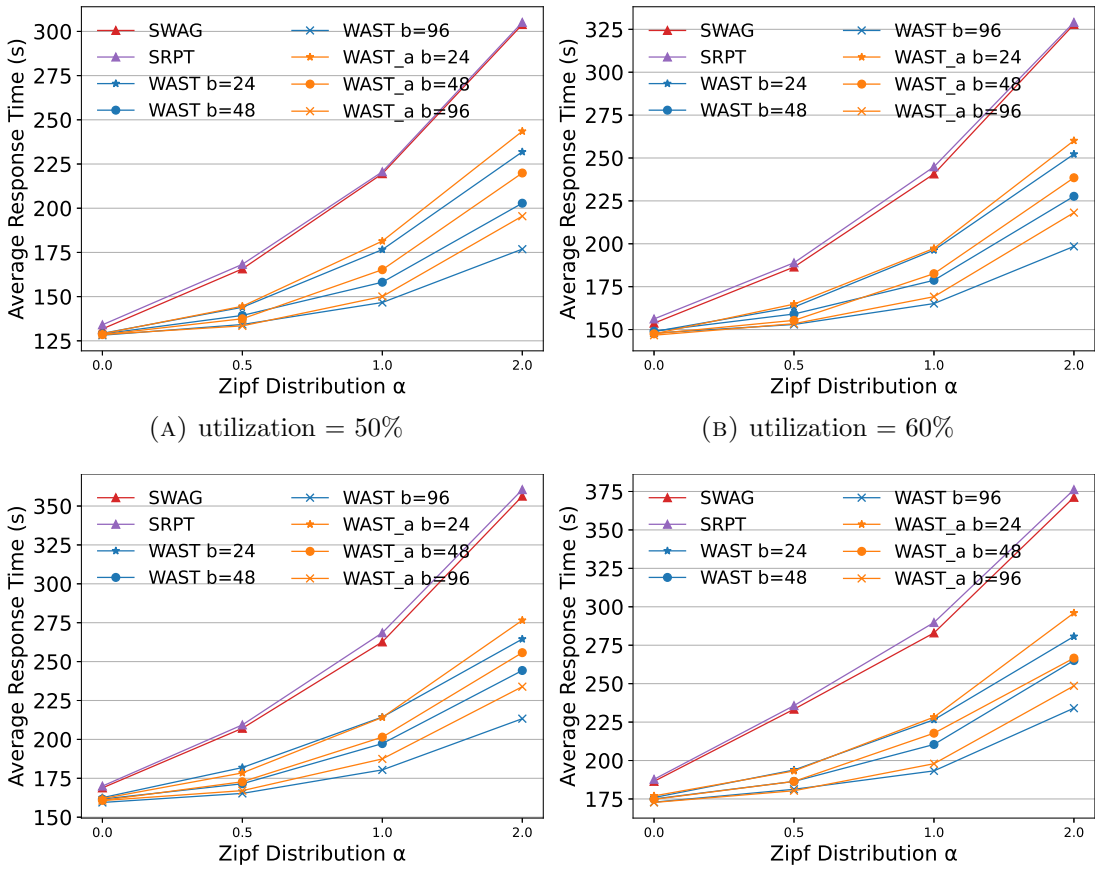
(C) utilization = 70%

(D) utilization = 80%

FIGURE 4.6: Average Job Response Time, Alibaba 2017

excessive network resource consumption. We suggest that the algorithm should exclusively utilize the spare network bandwidth, which is identified based on current system conditions. The specific method for quantifying this spare bandwidth via network monitoring is beyond the scope of this paper.

Replicated data inputs. Cloud-based environments often replicate data chunks across multiple geo-distributed servers in order to reduce service response latency and data transfer overhead [27, 28]. Our method can be integrated with algorithms that determine task assignments under replicated data inputs. Specifically, we first execute the task assignment algorithm to determine an initial execution site for each task, and then employ our proposed algorithm(s) to derive the job execution order and task transmissions. Notably, if a destination site is found to already host a replica of the required data partition, the corresponding transmission is not needed, and the task can be processed locally at the destination.



(A) utilization = 50%

(B) utilization = 60%

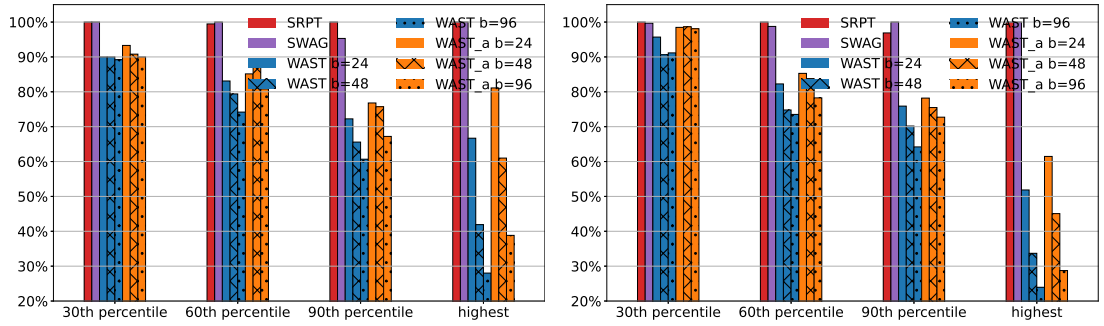
(C) utilization = 70%

(D) utilization = 80%

FIGURE 4.7: Average Job Response Time, Alibaba 2018

4.6 Summary

In this chapter, we have proposed a WAST algorithm and a fast approximation to improve job scheduling efficiency in geo-distributed systems with the usage of network resources. WAST constructs a near-optimal job execution order with task transmissions progressively. In each round, WAST searches for the job with the minimum completion time among all unscheduled jobs given the workloads and task transmissions scheduled in previous rounds. The sub-problem of minimizing a single job's completion time is converted into solving multiple minimum-cost flow problems, which can be computed efficiently. We also present a fast approximation to WAST, which optimizes each job's completion time by transferring its tasks of the latest completion time to other sites recursively. We have evaluated WAST and its approximation using real job traces from Alibaba. The experimental results show that WAST can effectively reduce the average job response time, especially when the task distributions of jobs are highly skewed.



(A) utilization = 50%

(B) utilization = 60%

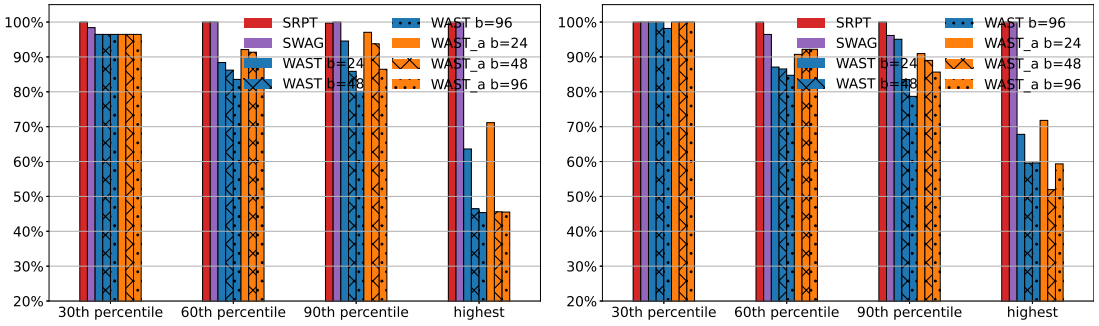
(C) utilization = 70%

(D) utilization = 80%

FIGURE 4.8: Job Response Time at Different Percentiles, Zipf Distribution $\alpha = 1$, Alibaba 2017

TABLE 4.3: Average computation time (second), Alibaba 2018 trace, 60% utilization

			b = 24Mbps		b = 48Mbps		b = 96Mbps	
slot	α	SWAG	WAST	WAST_a	WAST	WAST_a	WAST	WAST_a
32	0.0	0.0157	0.1209	0.0201	0.1127	0.0199	0.1005	0.0198
	0.5	0.0184	0.1323	0.0323	0.1167	0.0297	0.1023	0.0274
	1.0	0.0253	0.1805	0.0453	0.1327	0.0468	0.1301	0.0424
	2.0	0.0475	0.3014	0.0756	0.2283	0.0872	0.1936	0.0877
64	0.0	0.0306	0.2293	0.0378	0.2310	0.0399	0.2283	0.0376
	0.5	0.0365	0.2797	0.0585	0.2467	0.0653	0.2330	0.0563
	1.0	0.0502	0.3646	0.0883	0.3281	0.0781	0.2867	0.0845
	2.0	0.0885	0.6217	0.1666	0.5365	0.1299	0.4369	0.1482
128	0.0	0.0772	0.6290	0.0899	0.6169	0.0928	0.6057	0.0922
	0.5	0.0876	0.7265	0.1127	0.7205	0.1228	0.6601	0.1341
	1.0	0.1302	0.9427	0.1459	0.9012	0.1565	0.8432	0.1781
	2.0	0.2068	1.5751	0.2247	1.4975	0.2317	1.3203	0.2493



(A) utilization = 50%

(B) utilization = 60%

(C) utilization = 70%

(D) utilization = 80%

FIGURE 4.9: Job Response Time at Different Percentiles, Zipf Distribution $\alpha = 1$, Alibaba 2018

TABLE 4.4: Average computation time (second), Alibaba 2018 trace, 80% utilization

			b = 24Mbps		b = 48Mbps		b = 96Mbps	
slot	α	SWAG	WAST	WAST_a	WAST	WAST_a	WAST	WAST_a
32	0.0	0.0326	0.2950	0.0411	0.2782	0.0398	0.2543	0.0393
	0.5	0.0402	0.3212	0.0785	0.2879	0.0616	0.2549	0.0547
	1.0	0.0521	0.4283	0.1078	0.3196	0.1100	0.2513	0.0905
	2.0	0.0964	0.4826	0.1503	0.5172	0.1785	0.4501	0.1746
64	0.0	0.0667	0.5337	0.0863	0.5363	0.0891	0.5340	0.0818
	0.5	0.0842	0.6638	0.1378	0.5871	0.1483	0.5706	0.1266
	1.0	0.1078	0.8532	0.2476	0.7262	0.1848	0.6623	0.2028
	2.0	0.1816	1.2361	0.3728	1.1003	0.2484	0.9437	0.2913
128	0.0	0.1665	1.3935	0.1963	1.4098	0.2024	1.4332	0.2087
	0.5	0.1947	1.7591	0.2664	1.6433	0.2899	1.6000	0.3193
	1.0	0.2715	2.1885	0.3210	2.0776	0.3473	1.8640	0.3722
	2.0	0.4260	3.9332	0.4998	3.6373	0.5136	3.0193	0.5459

Chapter 5

Protective Scheduling for distributed Job Execution

In this chapter, we focus on developing scheduling policies that improve job processing efficiency in terms of average job response time, while guaranteeing a certain criterion of long-term fairness for online distributed job execution, where each job's workload is distributed across multiple sites. Inspired by the concept of protective scheduling in the single site system introduced by Friedman *et al.* [20, 21], our long-term fairness criterion dictates that no job should complete later than it would under a distributed Processor Sharing (PS) protocol, which evenly shares the site capacity among all active jobs at each site. Building on this foundation, we aim to develop job scheduling algorithms that tackle the dual challenges of enhancing processing efficiency while strictly adhering to this fairness criterion.

5.1 Preliminaries

5.1.1 Slack System in Single-Site Case

We introduce a slack system that provides an intuitive protectiveness assessment of the current system state, identifying whether any jobs are projected to exceed their completion times under the Processor Sharing (PS) policy. It is defined as follows [21]:

Consider a single-site system with compute resources, where the total resource capacity is assumed to be 1 for simplicity. During execution, the system hypothetically runs a Processor Sharing (PS) policy to obtain each job's PS completion time, which serves as its deadline. Every job in the system has two types of job loads, namely the real load and the PS load, which represent the remaining load of a job in the current system and the hypothetical PS execution respectively. Initially, the real load and the PS load of a new job are both equal to its initial load upon its arrival. The PS load then continuously decreases according to a simulated execution of the PS policy, where processing capacity is evenly shared by all jobs with positive loads, while its real load remains unchanged or decreases according to the actual scheduling policy. Let the current real loads and PS loads for n jobs in the system be $w_1, w_2, \ldots, w_n$ and $v_1, v_2, \ldots, v_n$, respectively. For simplicity, the job sequence is sorted in ascending order of their PS loads, i.e., $v_1 \leq v_2 \leq \cdots \leq v_n$ (note that the PS load order is identical to the PS completion time order of the jobs). The slack of a job J_i is computed as follows:

$$s_i = \sum_{l=1}^{i} v_l + (n-i)v_i - \sum_{l=1}^{i} w_l. \tag{5.1}$$

Specifically, the slack s_i consists of two parts: (i) the PS completion time of J_i, given by $\sum_{l=1}^{i} v_l + (n-i)v_i$, and (ii) the time required to complete the real loads of the first i jobs, given by $\sum_{l=1}^{i} w_l$. Consequently, s_i quantifies the maximum permissible delay for J_i and its preceding jobs to complete before violating the deadline set at J_i's PS completion time.

5.1.2 Two Types of Events in Slack System

We now explain how the slack system will change under various events. Friedman *et al.* [21] discussed the changes of the slack system under two types of events. First, if the system serves some job J_i for a duration of Δt, the slacks of jobs preceding J_i in the PS load order will decrease by Δt:

$$s_l = s_l - \Delta t, \quad \forall l \in \{1, \ldots, i-1\}. \tag{5.2}$$

Second, if a new job arrives in the system and joins in the job sequence at the i-th position based on its PS load (suppose its initial load is x), it will cause the slacks

of jobs preceding it to increase by x:

$$s_l = s_l + x, \quad \forall l \in \{1, \ldots, i-1\}. \tag{5.3}$$

For both events, the slacks of subsequent jobs $J_{i+1}, J_{i+2}, \ldots, J_n$ remain unchanged.

5.1.3 Servable and Completable jobs

Friedman *et al.* [21] proved that a scheduling policy is protective if and only if the slack of each job remains non-negative at all times. In other words, once the slack of any job becomes negative, then regardless of subsequent scheduling decisions (in the absence of new job arrivals), at least one job will inevitably complete later than its completion time under the PS policy. Therefore, the analysis of the first type of event above (serving jobs) indicates that it may not be true that all jobs can be prioritized for processing immediately if the system aims to maintain protectiveness. This fundamental restriction introduces the concept of servability and completability.

Definition 5.1. A job J_i is **servable** if and only if $s_l > 0$ for all $l \in \{1, \ldots, i-1\}$.

Definition 5.2. A job J_i is **completable** if and only if $s_l \geq w_i$ for all $l \in \{1, \ldots, i-1\}$.

Note that the first job in the PS load order is always servable and completable because the index set $\{1, \ldots, i-1\}$ is empty for $i = 1$. Furthermore, a job that has actually completed its actual workload (its real load drops to 0) but remains in the system—owing to an uncleared Processor Sharing (PS) load—is defined as both servable and completable, with its slack set to infinity. This convention ensures the correctness of the system's calculations regarding servable and completable jobs.

5.1.4 Fair Sojourn Protocol (FSP) and Its Variants

Friedman *et al.* [21] proposed three protective scheduling policies in the single-site case, namely Fair Sojourn Protocol (FSP), Optimistic FSP (OFSP) and Pessimistic FSP (PFSP). FSP schedules jobs based on the PS load order (PS completion time order). OFSP and PFSP prioritize the servable job and the completable job with

the least positive real load, respectively. Specifically, OFSP serves the servable job of the least real load until it completes or becomes not servable, and then repeats the computation of servable jobs to identify the next job to serve. Recall from Section 5.1.2 that when a new job arrives, the slacks of existing jobs either increase or remain unchanged. The term "optimistic" implies that although the job chosen by OFSP may not be able to serve to completion, the policy expects that there will be new job arrivals during its processing to further extend the PS completion time (and hence the slack) of the job, thus making it completable. On the contrary, the term "pessimistic" indicates that PFSP serves the job that is already completable (it stays completable if any new job arrives), which is more conservative and stable than OFSP.

The following example illustrates how protective scheduling policies like FSP maintain long-term fairness and prevent job starvation compared to efficiency-targeted policies. Consider two jobs, J_x and J_1, arriving simultaneously at $t = 0$ with initial loads $w_x = 2$ and $w_1 = 1$, respectively. Thereafter, a new job J_{a+1} with an initial load of 1 arrives at each subsequent time $t = a$ for $a = 1, 2, \ldots, 100$.

If the system adopts an efficiency-targeted scheduling policy like Shortest Remaining Processing Time (SRPT), it will serve the jobs in the order of $J_1, J_2, \ldots, J_{101}$, and finally J_x. This is because each job J_a has a smaller load than J_x and arrives exactly when the previous job J_{a-1} completes. Consequently, J_x will complete at $t = 103$, suffering from severe starvation.

Adopting FSP, the system execution will proceed as follows:

- $t = 0$**:** The system has J_x and J_1 with initial loads of 2 and 1, respectively. The system prioritizes J_1 for processing.
- $t = 1$**:** J_1 finishes its real execution, and J_2 arrives. In the hypothetical PS execution, the PS loads are $v_x = 1.5$, $v_1 = 0.5$, and $v_2 = 1$. Since J_1 completes and J_2 has smaller PS load than J_x, the system chooses to serve J_2.
- $t = 2$**:** J_2 finishes its real execution, and J_3 arrives. In the hypothetical PS execution, the PS loads are updated as $v_x \approx 1.167$, $v_1 \approx 0.167$, $v_2 \approx 0.667$, and $v_3 = 1$. The system then chooses to serve J_3.
- $t = 3$**:** J_3 finishes its real execution, and J_4 arrives. Notably, during the interval $[2, 3]$, J_1's PS load also drops to 0 and is hence removed from the

system. Therefore, the remaining PS loads become $v_x \approx 0.889$, $v_2 \approx 0.389$, $v_3 \approx 0.722$, and $v_4 = 1$.

At time $t = 3$, there are four jobs remaining in the system and two of them still have real loads (J_x and J_4). The system calculates that J_x has a lower PS load than J_4 ($0.889 < 1$), so FSP chooses to serve J_x, preventing it from indefinite starvation.

5.2 System Model

Consider a distributed computing environment comprising m distinct sites denoted by $\mathcal{S} = \{S_1, S_2, \ldots, S_m\}$. Each site S_j has compute resources with capacity u_j. A set of n jobs $\mathcal{J} = \{J_1, J_2, ..., J_n\}$ are ready to run the system. Each job J_i consists of workloads distributed across multiple sites that can be processed in parallel, and its overall completion time is determined by the workload that finishes last. We describe the real load distribution of the jobs using a load matrix $W_{n\times m}$. Specifically, each element w_{ij} represents the processing load of job J_i at site S_j. For example, if $u_2 = 2$ and $w_{12} = 4$, processing J_1's load at S_2 will require $w_{12}/u_2 = 2$ time units to complete. Furthermore, the system simulates a Processor Sharing (PS) policy at each site to estimate both the site-specific and global PS completion times for every job. Under this simulated policy, each site equally shares its capacity among all jobs with positive loads on the site. To track this process, we introduce a virtual load matrix $V_{n\times m}$, where each element v_{ij} denotes the virtual load of job J_i at site S_j under the PS policy, henceforth referred to as the PS load. The key notations used in this chapter are summarized in Table 5.1. We aim to design scheduling policies that achieve the highest possible efficiency in terms of average job response time, while ensuring that every job can complete before its completion time under the PS policy. Note that although the model is formulated in a static setting, it can be extended to online scenarios by capturing snapshots of dynamic job load distributions.

5.3 Distributed Slack System

In this section, we discuss how to build a slack system in the distributed job execution case that assists us in the protectiveness assessment of the system and

TABLE 5.1: Notation Table

Notation	Definition
$\mathcal{M} = \{S_1, S_2, \ldots, S_m\}$	Set of sites
$\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$	Set of jobs
$u_1, u_2, \ldots, u_m$	Each site's compute resource capacity
$W_{n\times m}$	Real load matrix
$V_{n\times m}$	PS load matrix

designing protective scheduling algorithms. There are two types of views in constructing the slack system: a global view and an independent view.

5.3.1 Slack System in Distributed Job Execution Case: A Global View

Since a job's global PS completion time is determined by the maximum of its site-specific PS completion times across all sites, an intuitive design of the slack system is to adopt each job's global PS completion time as the sorting criterion, which we refer to as the Global Slack (GS) scheme.

In the single-site case, computing the PS completion time requires jobs to be sorted in ascending order of their PS loads. However, in the distributed job execution case, the PS load distribution V may give rise to different job sequences at different sites. For notational simplicity, let $P_j(i)$ denote the set of jobs that precede (and include) job J_i when sorted in ascending order of their PS loads on site S_j:

$$P_j(i) = \left\{ J_k \in \mathcal{J} \;\middle|\; (v_{kj}, k) \leq (v_{ij}, i) \right\} \tag{5.4}$$

where $\leq$ denotes the standard lexicographical order on the tuples. In addition, different sites S_j's in the system may have different resource capacities u_j's. Thus, the site-specific PS completion time of a job J_i at a site S_j can be expressed as

$$\frac{1}{u_j}\left(\sum_{J_l \in P_j(i)} v_{lj} + (n - |P_j(i)|)v_{ij}\right). \tag{5.5}$$

The global PS completion time PS_i of a job J_i is the maximum site-specific PS completion time of J_i across m sites:

$$PS_i = \max_{1 \leq j \leq m} \frac{1}{u_j} \left(\sum_{J_l \in P_j(i)} v_{lj} + (n - |P_j(i)|) v_{ij} \right). \tag{5.6}$$

Based on this, we can sort jobs in ascending order of their global PS completion times, i.e., $PS_1 \leq PS_2 \leq \cdots \leq PS_n$. After sorting, the slack of a job J_i at site S_j under the GS scheme can be defined as:

$$s_{ij} = PS_i - \frac{1}{u_j} \sum_{l=1}^{i} w_{lj}, \tag{5.7}$$

which is the difference between J_i's global PS completion time and the time required to complete the real loads of the first i jobs at site S_j. We refer to this slack as the GSslack. The GS slack quantifies the maximum permissible delay for J_i and its preceding jobs to complete before violating the deadline set at J_i's global PS completion time. Under the GS scheme, a scheduling policy is protective if and only if all s_{ij}'s are non-negative during the system execution. Similar to the single-site case (introduced in Section 5.1.2), serving job J_i at site S_j for a duration of Δt reduces the GS slacks of all preceding jobs $(P_j(i) \setminus J_i)$ at S_j by Δt, while a new job arrival will only increase the slacks of jobs in $P_j(i) \setminus J_i$ at S_j. The definitions of servable and completable jobs can also be applied to individual sites in the distributed job execution case. We define that a job is **servable** or **completable** globally if and only if it is servable or completable at all sites. Note that the completion of a job's real load at a particular site does not affect its global servability or completability. In addition, the first job in the global PS order is always servable and completable globally because the index set $\{1, \ldots, i-1\}$ is empty for $i = 1$.

5.3.2 Conflict of Global Order-based Scheduling to Protectiveness

An ideal job schedule naturally desires a global order that is consistent across all sites, which is intuitive, simple and guarantees efficiency (obviously policies that prioritize different jobs for processing at different sites will lead to worse

job completion times). In protective scheduling, this necessitates an algorithm that generates a global job order in which every job is globally completable when scheduled for processing. However, we discover that under the GS scheme, the protectiveness cannot be guaranteed if the system schedules jobs using a global job order. The primary reason is that during the execution, while the GS scheme ensures that the deadline derived from the global PS completion time of each job can be satisfied, the site-specific PS completion time of some particular job J_i at some particular site S_j can be violated. Recall that the global PS completion time of a job is given by the maximum of its site-specific PS completion times across all sites. Since new jobs may arrive and new job arrivals can extend the site-specific PS completion times of jobs, it is possible that after some new jobs arrivals, this violated site S_j decides the global PS completion time of job J_i, thereby making the slack s_{ij} negative and violating protectiveness. We construct a concrete example below to illustrate the above.

Consider a distributed system of two sites, where the capacities of both sites are assumed to be 1 for simplicity. Suppose two jobs J_1, J_2 are submitted to the system with initial load distribution:

$$W = \begin{bmatrix} 2a & 2a+e \\ 2a+2e & 2a-e \end{bmatrix}, \tag{5.8}$$

where $a \gg e > 0$, J_1 has loads $2a$ and $2a+2e$ on sites S_1 and S_2 respectively, and J_2 has loads $2a+e$ and $2a-e$ on sites S_1 and S_2 respectively. The site-specific PS completion time can be computed as:

$$\begin{bmatrix} 4a & 4a \\ 4a+2e & 4a-2e \end{bmatrix}. \tag{5.9}$$

Therefore, the global PS completion time of the two jobs are $4a, 4a+2e$, respectively, which introduces the global PS order $J_1 \prec J_2$.

We compute the slack system based on the GS scheme: J_1 is the first job in the global PS order, so its GS slacks are $[4a-2a = 2a, 4a-(2a+e) = 2a-e]$; J_2 is the second job in the global PS order, so its GS slacks are $[4a+2e-2a-(2a+2e) =$

$0, 4a + 2e - (2a + e) - (2a - e) = 2e]$. Therefore, the slack matrix is

$$S = \begin{bmatrix} 2a & 2a - e \\ 0 & 2e \end{bmatrix}. \tag{5.10}$$

In this case, the first job J_1 in the global PS order is globally servable and completable, while the second job J_2 is not completable because $s_{11} = 2a < w_{21} = 2a + 2e$. Therefore, any scheduling policy that wants to derive a global job order while keeping the system protective will serve J_1 to completion first. Meanwhile, if we treat S_2 as an independent single site, another type of slacks can be calculated based on the discussion in Section 5.1.1 (we define it as "local slack" temporarily to differentiate it from the GS slack): the site-specific PS completion times of J_1 and J_2 at S_2 are $4a$ and $4a - 2e$, respectively. Thus, the local PS load order at S_2 is $J_2 \prec J_1$ and the local slacks of J_1 and J_2 at S_2 are $4a - (2a - e) - (2a + e) = 0$, and $4a - 2e - (2a - e) = 2a - e$, respectively. This indicates that J_2 is locally completable at site S_2 while J_1 is not, and serving J_1 will make J_2's local slack negative.

Suppose the system adopts the global PS order $J_1 \prec J_2$ to execute jobs. At time $t = 2a$, the real load distribution and PS load distribution will be

$$W = \begin{bmatrix} 0 & e \\ 2a + 2e & 2a - e \end{bmatrix}, \quad V = \begin{bmatrix} a & a + e \\ a + 2e & a - e \end{bmatrix}, \tag{5.11}$$

respectively. The GS slacks of jobs do not change because the system serves the first job in the global PS order, and it is still protective. Meanwhile, serving J_1 continuously decreases the local slack of J_2 at S_2 and it becomes negative ($2a - e - 2a = -e$) already.

Suppose at time $t = 2a$, a new job J_3 arrives with initial loads 0 and $a - 2e$ on sites S_1 and S_2 respectively. Then, the real load distribution and PS load distribution become

$$W = \begin{bmatrix} 0 & e \\ 2a + 2e & 2a - e \\ 0 & a - 2e \end{bmatrix}, \quad V = \begin{bmatrix} a & a + e \\ a + 2e & a - e \\ 0 & a - 2e \end{bmatrix}, \tag{5.12}$$

respectively. Therefore, the site-specific PS completion time is

$$\begin{bmatrix} 2a & 3a-2e \\ 2a+2e & 3a-4e \\ 0 & 3a-6e \end{bmatrix}. \tag{5.13}$$

Since $a \gg e$, the global PS completion times of all three jobs are decided by site S_2, and the global PS order becomes $J_3 \prec J_2 \prec J_1$, and the GS slacks of jobs are

$$S = \begin{bmatrix} a-4e & 0 \\ a-6e & -e \\ 3a-6e & 2a-4e \end{bmatrix}, \tag{5.14}$$

where s_{22} is negative. This implies that J_2 will never be able to complete its load at site S_2 before its global PS completion time. Therefore, the protectiveness is violated at the arrival of J_3.

As demonstrated by the above example, global order-based scheduling policies fail to preserve the protectiveness in distributed environments under the GS scheme. Furthermore, if the local slack of any job J_i at a site S_j becomes negative, one can systematically construct a worst-case scenario involving the simultaneous arrival of a job set $\{J_{(1)}, J_{(2)}, \ldots, J_{(z)}\}$, each containing a load $x \leq v_{ij}$ at S_j (v_{ij} is the PS load of J_i at S_j) and zero load at all other sites. Under such conditions, the site-specific PS completion time of J_i at S_j can be arbitrarily delayed while its local slack keeps negative (i.e., the second type of events 2 in Section 5.1.2). For a sufficiently large z, this prolonged site-specific PS completion time will become the global PS completion time of J_i, thereby rendering its GS slack at S_j negative. This fundamental limitation highlights strciter constraints on the slack system to maintain protectiveness, which we will introduce next.

5.3.3 Slack System in the Distributed Job Execution Case: An Independent View

Another natural approach to extend the slack system to distributed environments is to compute the slack at each site independently, which we refer to as the Independent Slack (IS) scheme. Under the IS scheme, we do not derive the global PS

order, and each site performs slack calculations independently. Using the notation $P_j(i)$ defined in equation 5.4 for the set of jobs preceding (and including) J_i based on their PS loads at S_j, the slack of job J_i at site S_j under the IS scheme is defined as:

$$s_{ij} = \frac{1}{u_j}\left(\sum_{J_l \in P_j(i)} v_{lj} + (n - |P_j(i)|)v_{ij} - \sum_{J_l \in P_j(i)} w_{lj}\right). \tag{5.15}$$

We refer to this slack as the IS slack. Similar to the GS scheme, a protective scheduling algorithm in the IS scheme requires that every s_{ij} to be non-negative during the system execution, i.e., the actual completion time of each job at every site does not exceed its site-specific PS completion time. Compared with the GS scheme, the IS scheme appears to be a stricter constraint since the completion deadline is set for each site rather than for the system as a whole. Consequently, the global completion time of each job is also bounded by its global PS completion time, and protectiveness is guaranteed. The potential drawback of the IS scheme is that scheduling policies following it may prioritize different jobs for processing at different sites (i.e., a global order of job execution may not exist), leading to poor efficiency in terms of average job response time.

5.4 PFSP_x: A Trade-off Algorithm Framework

Through the above analysis, we have shown a dilemma in distributed protective scheduling: adopting the GS scheme can achieve higher efficiency by deriving global job processing orders, but it may violate the protectiveness. On the other hand, using the IS scheme can guarantee the protectiveness, but may lead to poor efficiency because it may prioritize different jobs for processing at different sites. The key to solving this dilemma is to use the IS scheme and keep the IS slack of each job at each site non-negative while adjusting the schedule to approximate the global job processing order that is optimized for efficiency.

5.4.1 PFSP_x

We propose a novel heuristic algorithm framework PFSP_x which combines the Independent Slack scheme with any global order-based scheduling policy to enhance

Algorithm 7: PFSP_x Algorithm at Individual Sites

Input: A set of jobs $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ with real loads $w_1, w_2, \ldots, w_n$ and PS loads $v_1, v_2, \ldots, v_n$, hint order $\mathbf{h}$.

Output: Job execution order.

1 Let jobs be sorted in ascending order of PS load, i.e., $v_1 \leq v_2 \leq \cdots \leq v_n$;
2 $\sigma \leftarrow 0$;
3 **for** $i = 1$ **to** n **do**
4 $\quad \sigma \leftarrow \sigma + v_i - w_i$;
5 $\quad s_i \leftarrow \sigma + (n - i)v_i$;
6 $\quad r_i \leftarrow \begin{cases} \infty, & \text{if } i = 1; \\ \min(r_{i-1}, s_{i-1}), & \text{otherwise;} \end{cases}$
7 **while** $\mathcal{J} \neq \emptyset$ **do**
8 $\quad \mathcal{C} \leftarrow \{J_i \in \mathcal{J} \mid r_i \geq w_i\}$;
9 $\quad J_y \leftarrow \operatorname{argmin}_{J_i \in \mathcal{C}} \operatorname{pos}(J_i, \mathbf{h})$;
10 $\quad$ Append J_y to the job execution order;
11 $\quad \mathcal{J} \leftarrow \mathcal{J} \setminus \{J_y\}$;
12 $\quad$ **for** $i = 1$ **to** $y - 1$ **do**
13 $\quad\quad s_i \leftarrow s_i - w_y$;
14 $\quad s_y \leftarrow \infty$;
15 $\quad$ **for** $i = 1$ **to** n **do**
16 $\quad\quad r_i \leftarrow \min(r_{i-1}, s_{i-1})$;
17 **return** Job execution order;

efficiency while maintaining protectiveness. Specifically, PFSP_x takes as input a global job processing order derived by any efficiency-targeted scheduling policy like SRPT or Workload-Aware Greedy Selection (SWAG) [26], which is treated as the hint order for job processing and tries to approximate to it in the schedule. PFSP_x computes the IS slack of each job at each site S_j independently, from which it calculates the set of locally completable jobs and schedules the job with the highest priority in the hint order for processing, updates the IS slacks until all jobs at site S_j are scheduled. Algorithm 7 illustrates the details of the procedure at one site. The algorithm runs at each site based on the site-specific information, i.e., capacity, real loads and PS loads.

At every single site, PFSP_x sorts jobs in ascending order of their PS loads, computes each job's IS slack s_i and the minimum slack r_i of jobs preceding each job (lines 3-6). Because computing the IS slack requires the cumulative PS load and

real load of preceding jobs, a running variable σ is used to record the sum and optimize computing complexity. Note that a job J_i is completable if and only if its real load w_i is smaller or equal to the IS slacks of all its preceding jobs, which is equivalent to $r_i \geq w_i$. Next, the algorithm calculates the set of completable jobs $\mathcal{C}$ (line 8) by judging whether $r_i \geq w_i$ for each unscheduled job J_i. Then, it schedules the job with the highest priority in the hint order (line 9). Here, we define $\text{pos}(J_i, \mathbf{h})$ as the position (index) of job J_i within $\mathbf{h}$, where $\mathbf{h}$ is the hint order (permutation) of jobs given as input. A lower position index indicates a higher priority. Then, the algorithm appends the selected job J_y to the job processing order of the local site (line 10) and removes J_y from consideration (line 11). After scheduling J_y, the algorithm updates all s_i's and r_i's (lines 12-16). For each job before J_y in the PS load order, its IS slack decreases by w_y; the IS slakc s_y of job J_y becomes infinity; for each job after J_y in the PS load order, its slack remains unchanged. After updating all the IS slacks, the algorithm updates r_i's. The scheduling repeats iteratively until all jobs are scheduled. Finally, the algorithm returns the job execution order of the local site. In this way, PFSP_x achieves both the protectiveness guarantee by following the IS scheme and the efficiency improvement by approximating the hint order derived from any efficiency-targeted scheduling policy.

5.4.2 Complexity Analysis

In Algorithm 7, sorting n jobs in ascending order of their PS loads takes $O(n \log n)$ time. To efficiently compute the slack of each job, we maintain and reuse a running cumulative sum σ, which aggregates the differences between the PS loads and real loads of jobs iteratively. Consequently, evaluating all slacks s_i and the minimum preceding slacks r_i can be achieved in $O(n)$ time. Thus, the total time complexity for the initial slack calculation of all n jobs is $O(n \log n)$. Next, the algorithm iterates n times, in which it computes the set of completable jobs $\mathcal{C}$ and searches for the completable job with the highest priority in the hint order $\mathbf{h}$. The former operation requires checking whether $r_i \geq w_i$ for each unscheduled job J_i, which takes $O(n)$ time. Selecting the job with the highest priority in $\mathbf{h}$ also takes $O(n)$ time. After scheduling the selected job, updating the slack s_i's and minimum preceding slack r_i's of all jobs takes $O(n)$ time. Therefore, the overall time complexity of Algorithm 7 is $O((n \log n + n^2)) = O(n^2)$. The algorithm runs at each site.

Since the runs at different sites are entirely independent, they can be parallelized in real-world deployments.

5.4.3 Deployment

The PFSP_x algorithm is highly amenable to deployment in real-world distributed computing environments, either via a centralized scheduler or through distributed execution. Initially, the system requires collecting the real load and PS load distributions of each job across sites, and running an efficiency-targeted scheduling policy to derive the hint job order **h**. If the system deploys PFSP_x at the central scheduler, the scheduler will compute the job processing orders of all sites by running Algorithm 7 for each site separately and then propagate the job processing orders to respective sites .Alternatively, in a distributed setup, the central scheduler only broadcasts the hint order **h** to all sites. Each site independently performs its local IS slack calculations and the scheduling logic, reducing network communication overhead and allowing for efficient scheduling in large distributed systems. Re-computation only needs to be triggered upon new job arrivals, while completed jobs are excluded from computation.

5.5 Experiments

This section evaluates the performance of our proposed protective scheduling algorithms in online scenarios through comprehensive simulations using real-world job traces, aligning with the deployment framework outlined in Section 5.4.3.

5.5.1 Datasets

To validate our approach under realistic production conditions, we utilize industrial cluster data from the Alibaba Cluster Trace Program [54]. We extract essential scheduling telemetry from the raw log files of the 2017 and 2018 releases, capturing the arrival timestamp, task count, and processing duration of each job. From these pre-processed traces, a contiguous segment of 10,000 jobs is randomly sampled from each year to serve as the benchmark driving our simulations. As visualized in Fig.

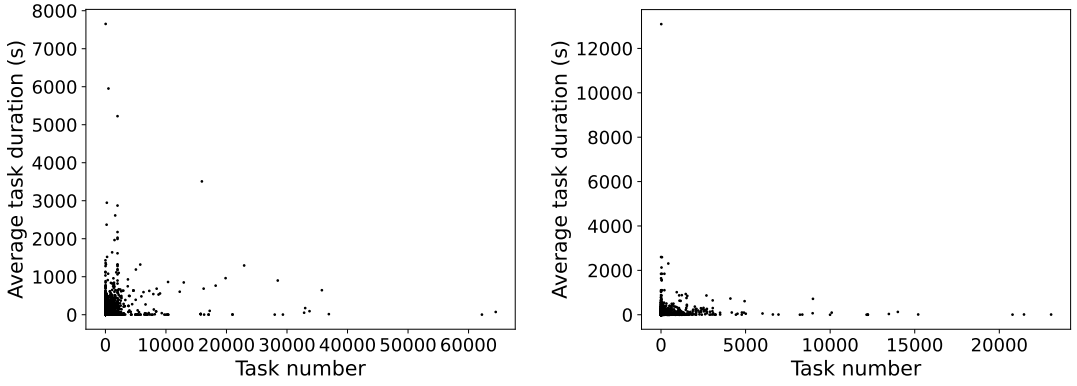
TABLE 5.2: Job characteristics of segments sampled from two datasets

Characteristics	Alibaba 2017	Alibaba 2018
Average task number per job	260.11	94.70
Average task processing time (s)	209.18	82.41

5.1, both segments sampled feature a heavy-tailed profile, where a tiny fraction of jobs dominates the vast majority of the total computational demand. Finally, the macro-level characteristics of the segments sampled for experimentation—including mean task counts and average task processing times—are summarized in Table 5.2.

5.5.2 System Configuration

We model a distributed environment comprising m identical sites, where each site is provisioned with an equal capacity. To evaluate our algorithms under varying degrees of skewness in workload distribution, we initialize the initial load distribution of each job by deploying its tasks among the sites based on the Zipf distribution and summing up their task processing times at each site. Specifically, the task allocation across sites follows the proportional weights $1, 2^{-\alpha}, 3^{-\alpha}, \ldots, m^{-\alpha}$, where the skewness parameter α controls the load imbalance. A higher α value implies that tasks are more heavily concentrated on a small subset of sites, while $\alpha = 0$ indicates a uniform distribution. We evaluate four distinct load distributions by varying $\alpha \in \{0.0, 0.5, 1.0, 2.0\}$. To prevent global load hotspots, we shuffle the



(A) Alibaba 2017 (B) Alibaba 2018

FIGURE 5.1: Job characteristic distributions of segments sampled from two datasets

order of the sites when deploying each job's task distribution. Additionally, the job arrival times from the trace are linearly scaled to emulate different system utilizations. While we examined various combinations of site numbers, site capacities, and load levels—all yielding consistent performance trends-we focus on presenting the results for a representative setup of $m = 10$ sites, capacity $u = 32$ for each site, and 50%, 60%, 70%, and 80% system utilizations.

5.5.3 Baseline Policies

We implemented the following baseline policies in the experiments for comparison. The first two baselines are efficiency-targeted policies, namely Shortest Remaining Processing Time (SRPT) and Workload-Aware Greedy Scheduling (SWAG) [26]. SRPT executes jobs based on their estimated completion time in the case that each job runs alone in the system. In our distributed environment, the estimated completion time of a job J_i is computed as $\max_{j\in[m]} w_{ij}/u_j$ where w_{ij} is the remaining real load of J_i at site S_j and u_j is the capacity of S_j. SRPT derives the job execution order by sorting these estimated completion times from low to high. SWAG schedules jobs in a greedy and iterative manner by estimating their completion times based on scheduled workloads in the system. After a job is scheduled, SWAG updates the scheduled workloads at all sites, and estimates the completion times of unscheduled jobs based on the updated workloads. We evaluate PFSP_SWAG which uses the job execution order derived by SWAG as the hint order $\mathbf{h}$.

In addition, we extend the protective scheduling algorithms in the single-site case (FSP, PFSP and OFSP) to the distributed job execution case based on both of the slack schemes. Under the IS scheme, we have FSP_i, OFSP_i and PFSP_i, which runs FSP, OFSP and PFSP independently at each site. Similarly, under the GS scheme, FSP_g derives the global job execution order based on the global PS completion times, while OFSP_g and PFSP_g prioritize the globally servable or completable job that finishes the fastest if served first, respectively. Since the first job in the global PS order is always servable and completable, OFSP_g and PFSP_g are feasible. However, they cannot perfectly guarantee protectiveness, as in the counterexample of Section 5.3.2.

The following example further illustrates how FSP_g, OFSP_g and PFSP_g work in the distributed job execution case. Consider a distributed system of two sites S_1 and S_2, both with capacity 1. Suppose at the beginning $t = 0$, two jobs J_1 and J_2 are submitted to the system, with the initial load distribution

$$W = \begin{bmatrix} 8 & 9 \\ 10 & 7 \end{bmatrix}, \tag{5.16}$$

where J_1 has loads 8 and 9 on S_1 and S_2, respectively, and J_2 has loads 9 and 7 on sites S_1 and S_2 respectively. The site-specfic PS completion times of the jobs are

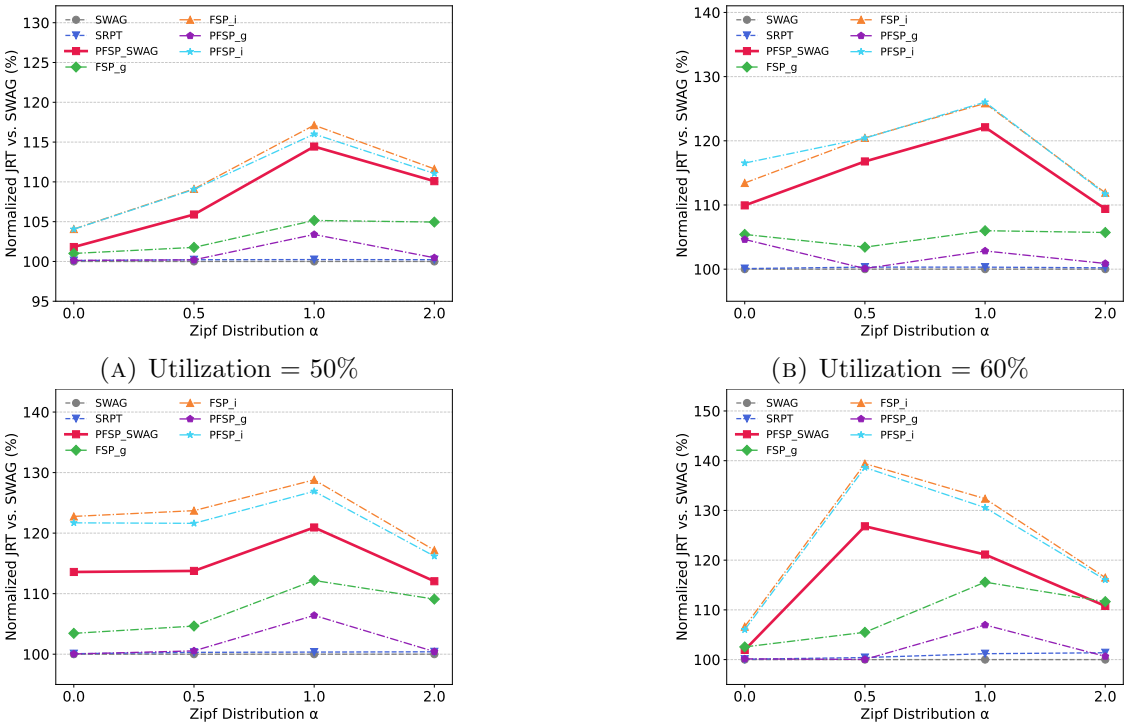
$$\begin{bmatrix} 16 & 16 \\ 18 & 14 \end{bmatrix}. \tag{5.17}$$

Thus, the global PS completion times of J_1 and J_2 are 16 and 18, respectively, and the global PS order is $J_1 \prec J_2$. Therefore, FSP_g will serve J_1 first then J_2 at all sites, and the average job response time is $(9 + 18)/2 = 13.5$.

We compute the slack system based on the GS scheme: J_1 is the first job in the global PS order, so its GS slacks are $[16 - 8 = 8, 16 - 9 = 7]$; J_2 is the second job and its GS slacks are $[18 - 10 - 8 = 0, 18 - 7 - 9 = 2]$, resulting in

$$S = \begin{bmatrix} 8 & 7 \\ 0 & 2 \end{bmatrix} \tag{5.18}$$

In this case, J_1 is globally servable and completable, while J_2 is not completable because $s_{11} = 8 < w_{21} = 10$. Therefore, PFSP_g will behave the same as FSP_g, i.e. serve J_1 to completion, and then J_2. Meanwhile, J_2 is servable since its preceding job J_1 has positive slacks $s_{11} = 8 > 0, s_{12} = 7 > 0$ at all sites. J_2 will finish at time $t = 10$ if it is served first, while J_1 will finish at time $t = 9$ if it is served first. Therefore, although both jobs are servable, OFSP_g will also first serve J_1 to completion since it finishes earlier than J_2, and the average job response time is also 13.5.



(A) Utilization = 50%

(B) Utilization = 60%

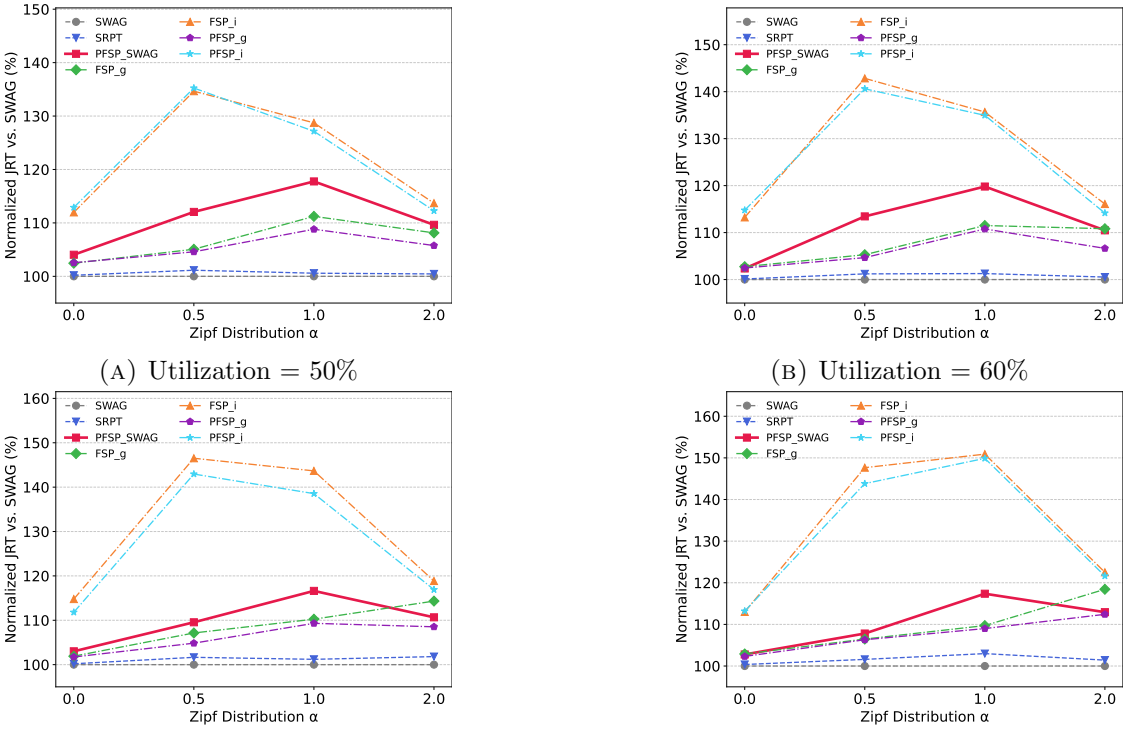
(C) Utilization = 70%

(D) Utilization = 80%

FIGURE 5.2: Average job response time, Alibaba 2017

5.5.4 Experimental Results

We use the average job response time and the number of jobs violating PS deadlines as the performance metrics to evaluate the dual objectives of efficiency and fairness under different scheduling policies, system utilizations and load distributions. The average job response time is defined as the average time span from each job's arrival to its completion, where a lower average job response time indicates higher job processing efficiency of the policy. The number of jobs violating PS deadlines is defined as the number of jobs whose actual completion time are later than their global PS completion times, which indicates the degree of long-term fairness in the system. A larger number of violations means that more jobs fail to finish before their completion times under the distributed PS policy, indicating a higher degree of unfairness/starvation in job scheduling. Because some of the policies have similar performance trends, we only present the results of SWAG, SRPT (efficiency-targeted policies), PFSP_g, FSP_g (policies based on the *GS* scheme), PFSP_i, FSP_i (policies based on the *IS* scheme) and PFSP_SWAG (derived from the PFSP_x framework).



(A) Utilization = 50%

(B) Utilization = 60%

(C) Utilization = 70%

(D) Utilization = 80%

FIGURE 5.3: Average job response time, Alibaba 2018

Figures 5.2 and 5.3 present the average job response time of different scheduling policies under various system settings. To facilitate comparison, we set the optimal average job response time derived from SWAG policy as the 100% baseline and compute the average job response time of other policies as a percentage of it. The results demonstrate that scheduling policies that are efficiency-targeted (SWAG and SRPT) or adopt the *GS* scheme (PFSP_g, FSP_g) achieve better average job response times compared to those adopting the *IS* scheme. In particular, PFSP_g achieves fairly close performance to the SWAG policy, demonstrating its potential in balancing the trade-off between efficiency and long-term fairness. In contrast, policies that adopt the *IS* scheme (PFSP_i, FSP_i, PFSP_SWAG) guarantee strict protectiveness at the expense of worse average job response times, with the performance gap reaching up to approximately 50%. Our proposed PFSP_SWAG algorithm demonstrates the best average job response time among the three policies based on the *IS* scheme, and under certain system configurations when the load distributions of jobs are highly skewed (i.e., $\alpha = 2.0$), it can even outperforms the FSP_g policy based on the *GS* scheme.

Table 5.3: Number of jobs violating PS deadlines, Alibaba 2017

Utilization	α	Strategies			
		SWAG	SRPT	PFSP_g	FSP_g
50%	0.0	44	50	0	0
	0.5	50	68	0	0
	1.0	50	68	0	0
	2.0	37	53	0	0
60%	0.0	41	46	0	0
	0.5	43	57	1	0
	1.0	34	46	0	0
	2.0	25	31	0	0
70%	0.0	36	41	0	0
	0.5	33	40	2	2
	1.0	23	28	0	0
	2.0	14	21	1	0
80%	0.0	29	32	1	1
	0.5	20	19	0	0
	1.0	15	20	0	0
	2.0	12	22	0	0

Table 5.4: Number of jobs violating PS deadlines, Alibaba 2018

Utilization	α	Strategies			
		SWAG	SRPT	PFSP_g	FSP_g
50%	0.0	37	50	0	0
	0.5	57	64	1	0
	1.0	41	47	0	0
	2.0	23	36	0	0
60%	0.0	26	38	1	1
	0.5	40	46	0	0
	1.0	28	28	0	0
	2.0	17	26	0	0
70%	0.0	17	31	2	2
	0.5	32	41	0	0
	1.0	23	25	0	0
	2.0	14	23	0	0
80%	0.0	13	25	1	1
	0.5	32	41	0	0
	1.0	18	18	0	0
	2.0	12	18	0	0

Tables 5.3 and 5.4 present the number of violations to the PS deadline under different scheduling policies. We omit the IS policies (including PFSP_SWAG) from the tables because they guarantee zero violations under all conditions. The results show that both SWAG and SRPT have tens of violations in general, indicating that efficiency-targeted policies do not guarantee protectiveness in the distributed job execution case. In contrast, policies based on the GS scheme have much fewer violations (zero violations in most cases). In particulr, PFSP_g significantly decreases the number of violations while keeping a close average job response time to SWAG and SRPT at all system settings, demonstrating that it achieves a good trade-off between efficiency and long-term fairness. Meanwhile, PFSP_SWAG guarantees zero violations while effectively decreasing the average job response time compared to other policies based on the IS scheme. These empirical results suggest that the PFSP_g and PFSP_SWAG can be selectively applied to different scenarios depending on the system's specific trade-offs between fairness and efficiency.

5.5.5 On Inaccurate Job Load Estimation

All scheduling policies introduced above rely on accurate estimations of job load distributions across sites. Inaccurate estimations can lead to system performance degradation in both average job response time and PS deadline violations. To evaluate the impact of inaccurate estimations, we randomly generate the job load estimates following a uniform distribution between $-x$ and x, where x is the maximum deviation assumed. Based on our evaluation with 10,000 jobs, when the estimated job loads uniformly deviate within $\pm 10\%$ of the ground truth (i.e., ranging from 90% to 110%), the number of jobs violating PS deadlines rises up to hundreds across different scheduling policies. Estimation errors manifest in two distinct forms:

- **Over-estimation** gives a larger estimated load of a job than its actual load, which may deprioritize it in the execution order. In this case, other jobs in the system can either have their execution order advanced or kept unchanged, so generally their completion times will not degrade and the number of jobs violating PS deadlines will increase by one at most. Consequently, over-estimation predominantly impacts the over-estimated job itself, degrading its own completion time.

- **Under-estimation** gives a smaller estimated load of a job than its actual load. As a result, an under-estimated job is not completed and needs to continue execution even when its (estimated) real load drops to 0. This introduces severe systemic risks because with its (estimated) real load being 0, it will be prioritized for execution by all scheduling policies (including efficiency-targeted ones and those based on the GS and IS schemes), inducing serious head-of-line blocking and starving subsequent jobs, which ultimately triggers a surge of PS deadline violations.

To counteract performance degradation under job load under-estimations, we propose a mitigation mechanism, called Processor Sharing Fallback (PSF). PSF allocates a fair capacity share to these under-estimated jobs: Once a job has its (estimated) real load drop to 0 at a site but has not completed execution at this site, it transitions into a fallback PS state. In this state, the job is assigned a fixed equal share ($1/N_{\text{active}}$) of the site capacity, where N_{active} represents the concurrent count of active jobs with remaining real load at the site. The leftover capacity of the site is then used to schedule non-fallback jobs following the original scheduling policy. Note that the PSF mechanism will be activated only when a job's load is underestimated. Consequently, it can be seamlessly integrated as a lightweight add-on module to existing scheduling policies, incurring negligible overhead when load estimations are accurate.

TABLE 5.5: Number of jobs violating PS deadlines with/without PSF, 10% deviation, 70% utilization, Alibaba 2017

	Without PSF				**With PSF**			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	173	300	357	732	69	91	97	160
SRPT	182	322	369	768	82	102	102	164
PFSP_g	217	195	363	577	48	52	57	109
FSP_g	104	83	79	52	29	48	53	110
PFSP_SWAG	543	973	909	888	70	74	65	72
PFSP_i	774	1232	978	981	79	80	62	72
FSP_i	285	547	513	480	71	71	58	67

Tables 5.5 through 5.12 illustrate the impact of the PSF mechanism on PS deadline violations under different deviations of job load estimations (maximum deviation = 10%, 20%, 30% and 40%) for both datasets. Since the performance trends for different system utilizations are similar, we present the representative results for

TABLE 5.6: Number of jobs violating PS deadlines with/without PSF, 20% deviation, 70% utilization, Alibaba 2017

	Without PSF				**With PSF**			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	443	366	556	1185	97	135	136	214
SRPT	451	372	552	1149	108	133	135	226
PFSP_g	395	371	521	791	80	113	110	172
FSP_g	245	167	197	184	78	106	105	167
PFSP_SWAG	970	1509	1606	1519	97	143	98	107
PFSP_i	1314	1976	1623	1662	109	147	102	106
FSP_i	439	920	1039	1000	98	147	98	94

TABLE 5.7: Number of jobs violating PS deadlines with/without PSF, 30% deviation, 70% utilization, Alibaba 2017

	Without PSF				**With PSF**			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	369	607	896	1495	106	146	178	184
SRPT	363	666	889	1526	108	143	780	182
PFSP_g	317	561	580	1476	84	105	118	162
FSP_g	132	191	169	547	84	98	112	161
PFSP_SWAG	632	1499	1486	2157	114	130	134	154
PFSP_i	697	1721	1466	2421	125	136	126	146
FSP_i	293	872	740	1465	116	131	120	142

TABLE 5.8: Number of jobs violating PS deadlines with/without PSF, 40% deviation, 70% utilization, Alibaba 2017

	Without PSF				**With PSF**			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	1143	1351	1925	2826	140	282	306	352
SRPT	1147	1518	1920	2819	141	260	327	326
PFSP_g	1060	1304	1488	2631	120	245	215	296
FSP_g	354	548	597	1572	111	248	205	301
PFSP_SWAG	1640	2353	3338	3375	149	199	215	247
PFSP_i	1903	2911	3648	3765	159	205	237	248
FSP_i	748	1438	2391	2582	144	199	222	249

70% system utilization only. As can be seen, integrating the PSF mechanism significantly reduces violations for all policies, where our proposed protective scheduling policies outperform efficiency-targeted baselines in most cases, especially when the job load distributions are highly skewed. It can also be observed that policies based on the GS scheme (FSP_g, PFSP_g) yield the lowest violation counts. This is because these policies set the global PS completion time of a job as its deadline at all sites for scheduling purposes. The global PS completion time is given by the

TABLE 5.9: Number of jobs violating PS deadlines with/without PSF, 10% deviation, 70% utilization, Alibaba 2018

	Without PSF				With PSF			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	92	98	188	178	44	47	62	76
SRPT	108	91	200	183	55	52	69	81
PFSP_g	81	40	133	149	30	19	27	62
FSP_g	53	27	37	30	29	20	25	65
PFSP_SWAG	394	247	461	276	39	41	31	37
PFSP_i	423	350	513	307	41	37	37	42
FSP_i	196	102	258	137	36	34	31	36

TABLE 5.10: Number of jobs violating PS deadlines with/without PSF, 20% deviation, 70% utilization, Alibaba 2018

	Without PSF				With PSF			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	145	209	425	398	38	63	65	90
SRPT	163	209	394	408	48	64	67	84
PFSP_g	144	216	315	346	34	33	45	70
FSP_g	70	52	99	94	33	32	44	73
PFSP_SWAG	450	564	745	541	45	55	53	51
PFSP_i	498	625	774	536	46	58	51	49
FSP_i	185	293	355	281	44	53	50	48

TABLE 5.11: Number of jobs violating PS deadlines with/without PSF, 30% deviation, 70% utilization, Alibaba 2018

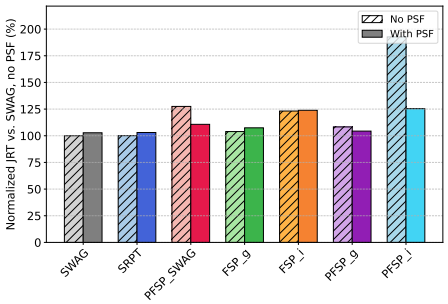
	Without PSF				With PSF			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	880	386	801	746	106	80	161	176
SRPT	884	362	761	760	113	88	150	180
PFSP_g	947	290	597	638	101	59	156	156
FSP_g	243	87	89	218	99	56	153	165
PFSP_SWAG	1666	1078	1785	946	135	75	121	88
PFSP_i	2009	1426	2012	1103	140	80	131	93
FSP_i	767	423	935	514	143	71	126	88

maximum of the site-specific PS completion times across all sites. Thus, it is less affected by under-estimation of job loads at individual sites than the site-specific PS completion times.

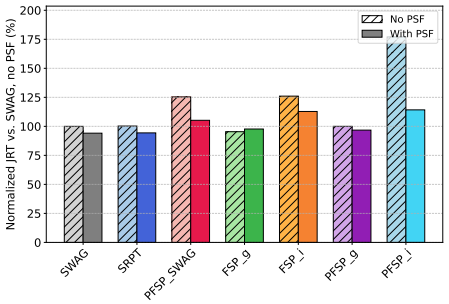
Figures 5.4 through 5.11 illustrate the average job response time under different deviations of job load estimations for both datasets. To facilitate comparison, the response time of the baseline SWAG policy—without the PSF mechanism—is normalized to 100%, with all other policies expressed relative to this baseline. Overall,

TABLE 5.12: Number of jobs violating PS deadlines with/without PSF, 40% deviation, 70% utilization, Alibaba 2018

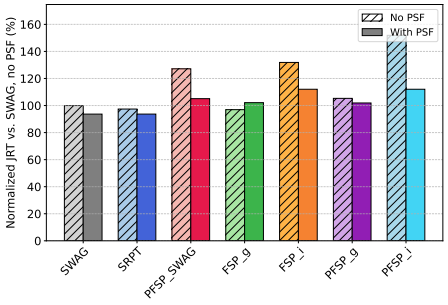
	Without PSF				**With PSF**			
Policy	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$	$\alpha = 0.0$	$\alpha = 0.5$	$\alpha = 1.0$	$\alpha = 2.0$
SWAG	778	369	927	779	134	123	199	151
SRPT	801	408	964	751	142	127	198	152
PFSP_g	727	327	817	624	126	95	176	130
FSP_g	446	191	386	263	123	95	177	139
PFSP_SWAG	1229	903	2053	940	178	124	167	106
PFSP_i	1325	1074	2085	980	173	133	171	107
FSP_i	779	507	1452	419	170	129	166	110



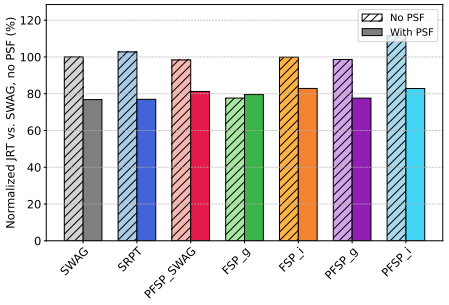
(A) $\alpha = 0.0$



(B) $\alpha = 0.5$



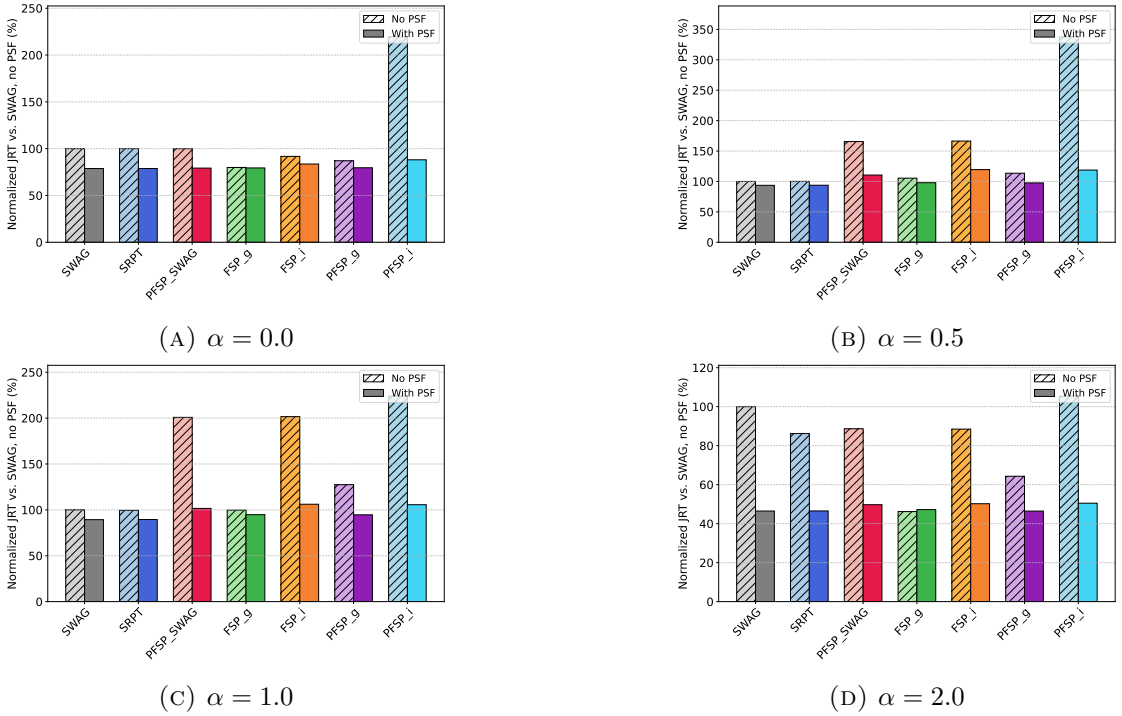
(C) $\alpha = 1.0$



(D) $\alpha = 2.0$

FIGURE 5.4: Average job response time with/without PSF, 10% deviation, 70% utilization, Alibaba 2017

the performance difference between policies with and without the PSF mechanism remains minor across most scenarios. However, in certain cases (e.g., Figure 5.7), integrating the PSF mechanism yields a substantial reduction in average job response time across all evaluated scheduling policies. In conclusion, incorporating the PSF mechanism into the evaluated scheduling policies significantly mitigates PS deadline violations while maintaining comparable or superior average job response times.



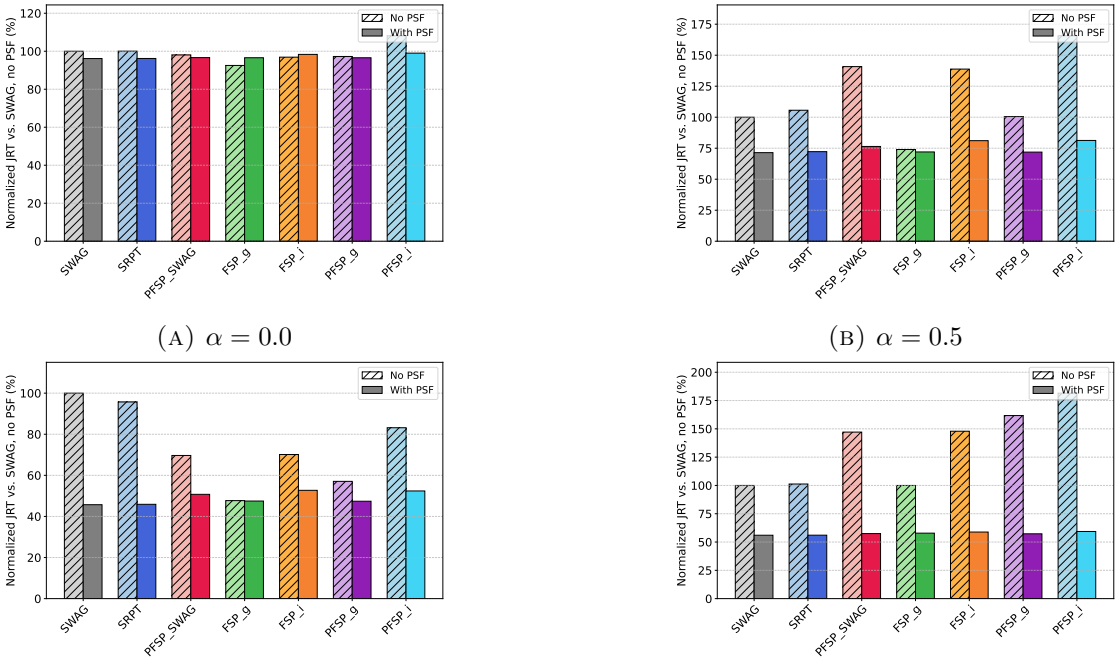
(A) $\alpha = 0.0$ (B) $\alpha = 0.5$

(C) $\alpha = 1.0$ (D) $\alpha = 2.0$

FIGURE 5.5: Average job response time with/without PSF, 20% deviation, 70% utilization, Alibaba 2017

5.6 Summary

In this chapter, we investigated how to achieve the dual objective between efficiency and long-term fairness in distributed job execution by extending the theory of protective scheduling in the single-site case. We established the distributed slack system to help judge the system protectiveness using two perspectives: the Global Slack (GS) scheme and the Independent Slack (IS) scheme. Through theoretical analysis, we proved that dynamic job arrivals inherently prevent global order-based policies from guaranteeing strict protectiveness under the GS scheme. To overcome this dilemma, we proposed PFSP_x, a novel heuristic scheduling framework that adopts the IS scheme to ensure protectiveness while approximating a hint job order derived by any efficiency-targeted policies. We also developed a series of extension policies based on the single-site policies FSP, PFSP and OFSP as the baselines. Extensive trace-driven simulations using Alibaba cluster datasets demonstrated that our proposed PFSP_SWAG policy achieves the highest job processing efficiency among all strictly protective policies evaluated. Furthermore, the extension policy PFSP_g under the GS scheme is shown capable of delivering average job



(A) $\alpha = 0.0$

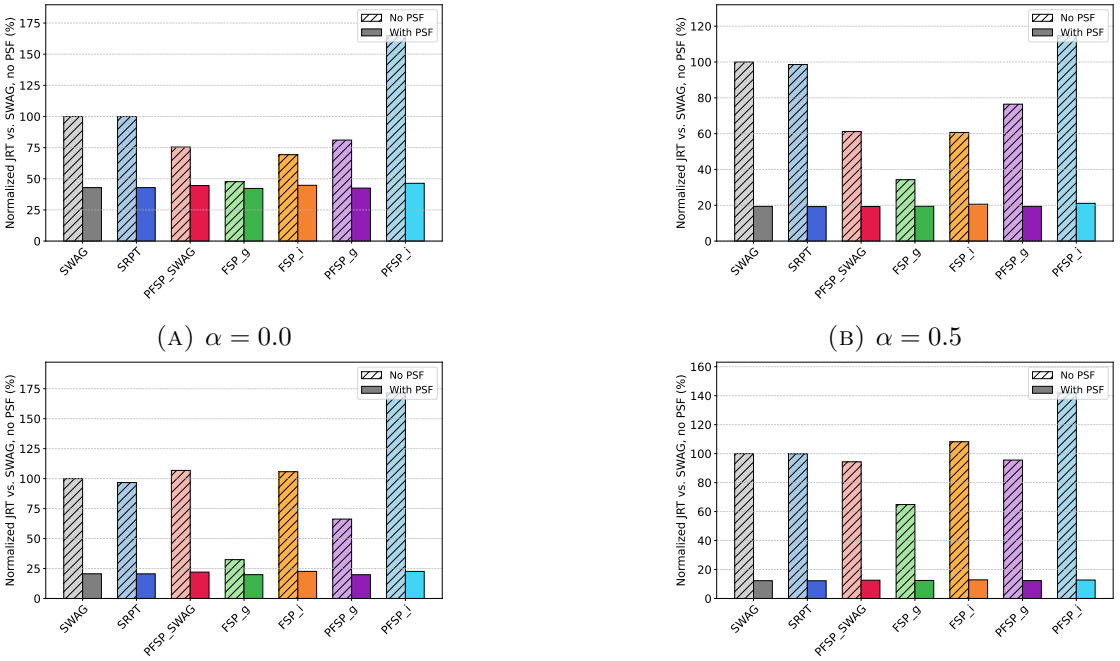
(B) $\alpha = 0.5$

(C) $\alpha = 1.0$

(D) $\alpha = 2.0$

FIGURE 5.6: Average job response time with/without PSF, 30% deviation, 70% utilization, Alibaba 2017

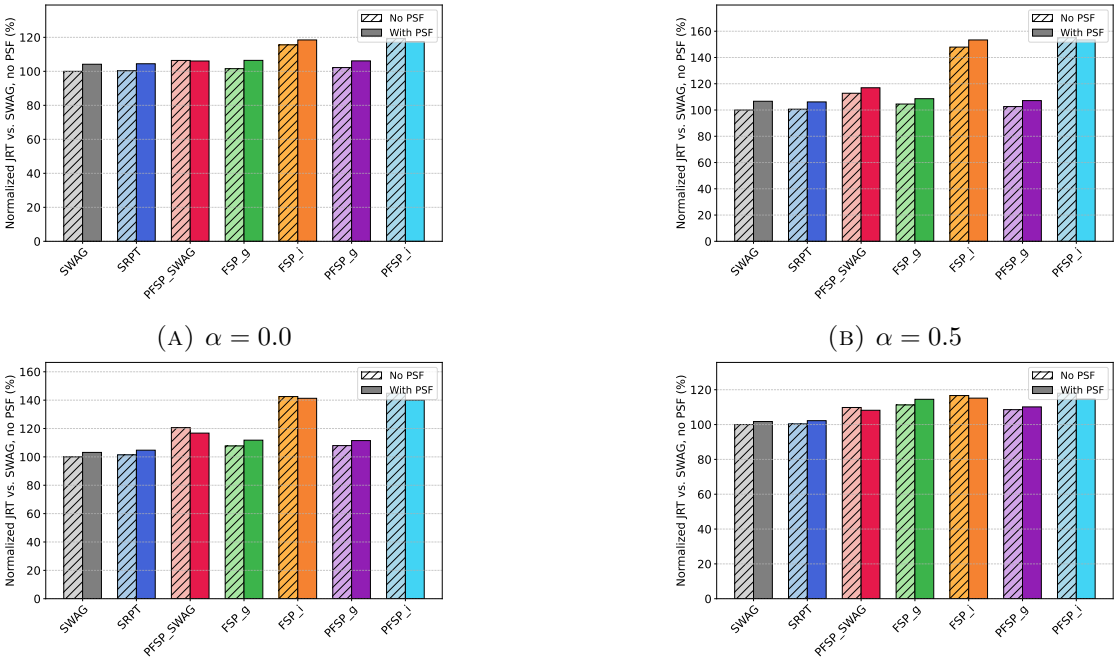
response times comparable to efficiency-targeted policies with negligible fairness violations. Ultimately, these complementary strategies equip distributed systems to flexibly navigate the delicate trade-off between job processing efficiency and long-term fairness.



(A) $\alpha = 0.0$ (B) $\alpha = 0.5$

(C) $\alpha = 1.0$ (D) $\alpha = 2.0$

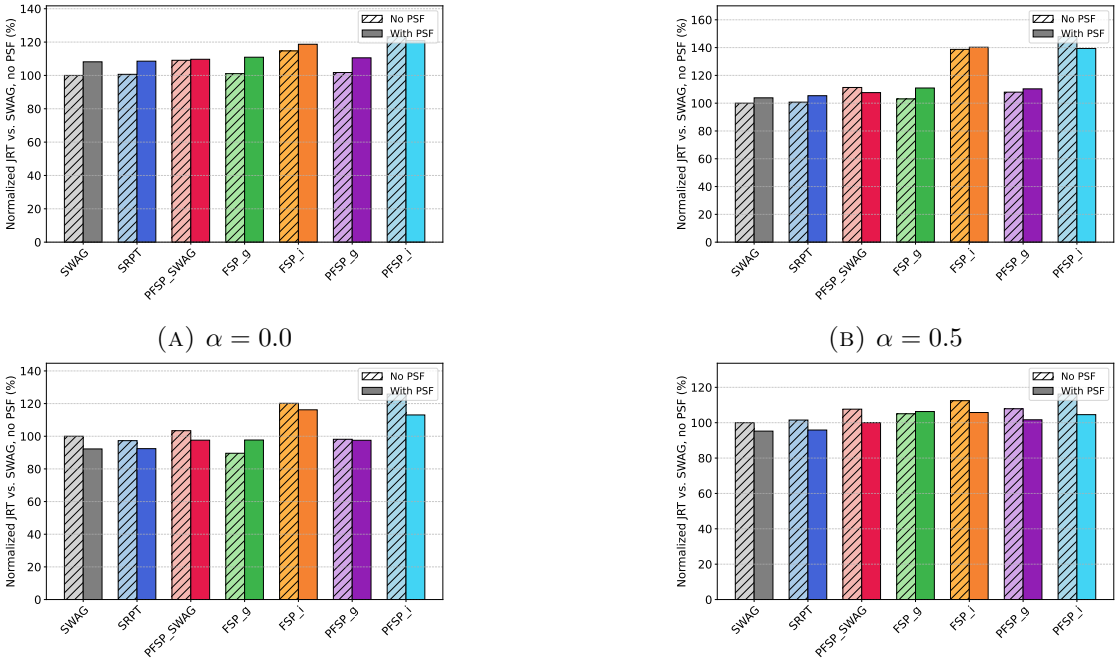
FIGURE 5.7: Average job response time with/without PSF, 40% deviation, 70% utilization, Alibaba 2017



(A) $\alpha = 0.0$ (B) $\alpha = 0.5$

(C) $\alpha = 1.0$ (D) $\alpha = 2.0$

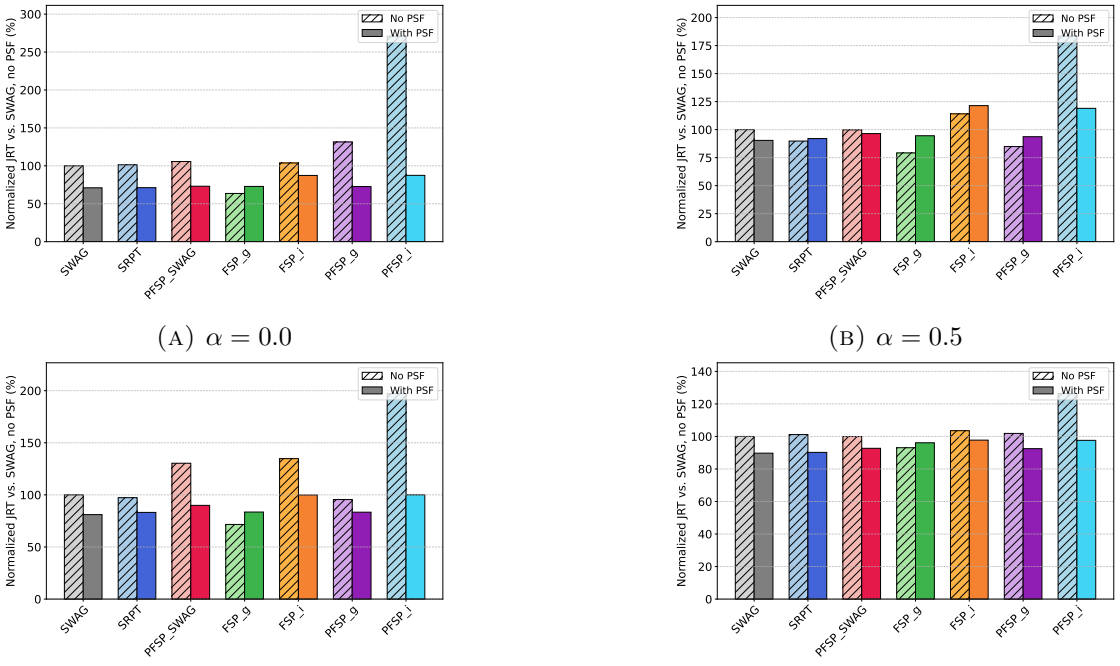
FIGURE 5.8: Average job response time with/without PSF, 10% deviation, 70% utilization, Alibaba 2018



(A) $\alpha = 0.0$ (B) $\alpha = 0.5$

(C) $\alpha = 1.0$ (D) $\alpha = 2.0$

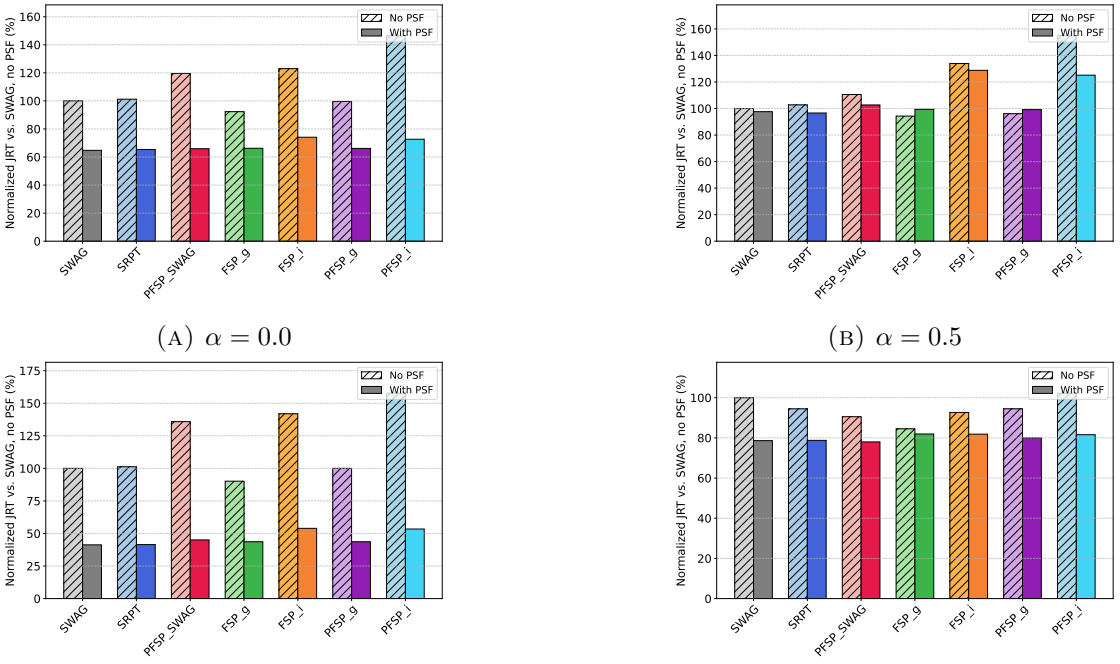
FIGURE 5.9: Average job response time with/without PSF, 20% deviation, 70% utilization, Alibaba 2018



(A) $\alpha = 0.0$ (B) $\alpha = 0.5$

(C) $\alpha = 1.0$ (D) $\alpha = 2.0$

FIGURE 5.10: Average job response time with/without PSF, 30% deviation, 70% utilization, Alibaba 2018



(A) $\alpha = 0.0$

(B) $\alpha = 0.5$

(C) $\alpha = 1.0$

(D) $\alpha = 2.0$

FIGURE 5.11: Average job response time with/without PSF, 40% deviation, 70% utilization, Alibaba 2018

Chapter 6

Conclusion and Future Work

6.1 Conclusion

In this thesis, we have addressed three primary scheduling challenges within the context of distributed job execution: max-min fair resource allocation with task transmission, efficient job scheduling co-optimized with task transmission, and balancing the trade-off between long-term fairness and efficiency. The main contributions of this dissertation encompass defining the Dynamic Aggregate Max-min Fairness (DAMF) policy, developing integrated algorithms for task assignment and transmission to maximize job processing efficiency, and proposing a novel scheduling framework that enhances job processing efficiency while guaranteeing strict completion deadlines.

In Chapter 3, we investigated fair resource allocation policies incorporating task transmission in distributed execution environments. We presented a resource allocation policy named Dynamic Aggregate Max-min Fairness (DAMF), which is theoretically achievable and satisfies widely recognized fairness properties, including Pareto efficiency, envy-freeness, and strategy-proofness. We further proposed comprehensive algorithms to compute the DAMF allocation, alongside a highly efficient approximation method. Experiments utilizing real-world job traces demonstrate that, compared with baseline policies, DAMF and its approximation algorithm significantly reduce the imbalance of resource allocation across jobs and improve average job response times, particularly when spatial workload distributions are highly skewed across sites.

In Chapter 4, we focused on enhancing job processing efficiency through integrated job scheduling and task transmission. Recognizing the intricate inter-dependency between job execution orders and cross-site task migrations, we first targeted the optimization of a single job's response time, transforming the mathematical formulation into multiple minimum-cost flow problems. Building upon this, we developed Workload-Aware Selection with Transmission (WAST), an algorithm that iteratively schedules the job with the earliest potential completion time while simultaneously optimizing its task transmission strategy. Furthermore, we introduced a fast approximation to WAST that significantly reduces computational complexity with marginal performance degradation. Experimental results confirm that WAST and its approximation substantially decrease average job response time compared to conventional baseline policies.

In Chapter 5, we explored the long-term fairness and efficiency trade-off in distributed job execution. Inspired by the notion of protective scheduling in single-site environments, we established a long-term fairness criterion dictating that no job should complete later than its finish time under a baseline distributed Processor Sharing (PS) policy. To operationalize this, we constructed two types of slack systems to evaluate system protectiveness: a global view (GS) scheme and an independent view (IS) scheme. Based on these schemes, we extended classical protective scheduling algorithms (FSP, PFSP, and OFSP) to the distributed job execution case. Crucially, we proved that despite violations being potentially rare, no global order-based scheduling policy is strictly protective in distributed settings. To overcome this fundamental limitation, we introduced PFSP_x, a novel heuristic scheduling framework that adopts the IS scheme to rigorously enforce protectiveness while leveraging heuristic job orderings derived from efficiency-targeted policies to improve efficiency. Our empirical evaluations demonstrate that PFSP_x achieves the highest job processing efficiency among all protective policies evaluated. Meanwhile, PFSP utilizing the GS scheme effectively minimizes PS deadline violations while maintaining processing efficiency comparable to purely efficiency-targeted approaches.

6.2 Future Work

Following the research presented in this dissertation, several promising directions for future work can be outlined. A highly promising avenue is to extend the proposed fair resource allocation and efficient scheduling policies to multi-resource environments in distributed systems. While our current frameworks primarily focus on compute resources (i.e., slots) and treat network bandwidth as an auxiliary facilitator, real-world distributed platforms inherently manage a heterogeneous mix of distinct resource types, including CPU, memory, and disk storage. Since jobs typically submit tasks with multi-dimensional resource requirements, developing a joint multi-resource allocation paradigm targeting fairness and efficiency under network constraints would represent a substantial advancement.

Another limitation of our current work is the assumption of precise a priori knowledge regarding job resource demands, processing times, and systemic network capacities. Although we have evaluated the performance of our proposed algorithms under unstable bandwidths or inaccurate estimations of job workloads, the practical viability of these policies could be further fortified by explicitly incorporating prediction inaccuracies and stochastic job arrival patterns directly into the system modeling phase. Proactively accounting for such estimation errors would enable the development of robust, uncertainty-aware scheduling strategies, thereby bridging the gap between theoretical models and real-world deployments.

Additionally, our current framework for the long-term fairness and efficiency tradeoff does not fully exploit inter-site network resources. Although integrating network transmission into the slack system introduces significant complexity (e.g., cross-site load transfers could inadvertently induce negative slack and violate protectiveness), protective scheduling policies that intelligently utilize task transmissions hold the potential to further enhance processing efficiency while strictly ensuring that each job completes no later than its PS deadline.

Finally, another compelling direction is the integration of learning-based approaches into distributed job execution. While several reinforcement learning (RL) methods have been proposed for resource allocation and job scheduling [37–39, 42], most existing studies optimize efficiency under overly simplified system models (e.g., assuming a single active job or treating transmission costs as negligible). In contrast, our target environment demands that a central controller continuously aggregates

diverse runtime state information—including task-level resource demands, spatial task distributions, site capacities, and dynamic network bandwidths. This holistic view introduces an exponentially large solution space for resource allocation, posing severe scalability challenges. Nevertheless, leveraging historical job arrival and workload patterns in specific geo-distributed systems to train customized, scalable RL agents remains an exciting frontier for resolving this complex optimization problem.

Bibliography

[1] Ninad Hogade and Sudeep Pasricha. A survey on machine learning for geo-distributed cloud data center management. *IEEE Transactions on Sustainable Computing*, 8(1):15–31, 2022. 1

[2] Mohammed Bergui, Said Najah, and Nikola S Nikolov. A survey on bandwidth-aware geo-distributed frameworks for big-data analytics. *Journal of Big Data*, 8(1):40, 2021. 1

[3] Dongyao Wu, Sherif Sakr, Liming Zhu, and Huijun Wu. Towards big data analytics across multiple clusters. In *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 218–227. IEEE, 2017. 1

[4] Kevin Hsieh, Aaron Harlap, Nandita Vijaykumar, Daniel Konomis, Gregory R. Ganger, Phillip B. Gibbons, and Onur Mutlu. Gaia: Geo-Distributed machine learning approaching LAN speeds. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 629–647. USENIX Association, 2017. 1, 9

[5] Zhenheng Tang, Xueze Kang, Yiming Yin, Xinglin Pan, Yuxin Wang, Xin He, Qiang Wang, Rongfei Zeng, Kaiyong Zhao, Shaohuai Shi, et al. Fusionllm: a decentralized llm training system on geo-distributed gpus with adaptive compression. *arXiv preprint arXiv:2410.12707*, 2024. 1

[6] Jinquan Wang, Xiaojian Liao, Xuzhao Liu, Jiashun Suo, Zhisheng Huo, Chenhao Zhang, Xiangrong Xu, Runnan Shen, Xilong Xie, and Limin Xiao. Spantrain: Highly efficient cross-domain model distributed training system under heterogeneous gpus and networks in cee environment. *arXiv e-prints*, pages arXiv–2505, 2025. 1

[7] Bozidar Radunovic and Jean-Yves Le Boudec. A unified framework for max-min and min-max fairness with applications. *IEEE/ACM Transactions on networking*, 15(5):1073–1083, 2007. 7, 15, 25

[8] Jeonghoon Mo and Jean Walrand. Fair end-to-end window-based congestion control. *IEEE/ACM Transactions on networking*, 8(5):556–567, 2000. 7

[9] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of NSDI*, pages 323–336, 2011. 7, 18

[10] Ali Ghodsi, Matei Zaharia, Scott Shenker, and Ion Stoica. Choosy: Max-min fair sharing for datacenter jobs with constraints. In *Proceedings of the 8th ACM European Conference on Computer Systems*, pages 365–378, 2013. 8

[11] Seyed Majid Zahedi and Benjamin C Lee. REF: Resource elasticity fairness with sharing incentives for multiprocessors. In *Proceedings of ACM ASPLOS*, pages 145–160, 2014. 8

[12] Avital Gutman and Noam Nisan. Fair allocation without trade. In *Proceedings of AAMAS*, pages 719–728, 2012. 8, 18

[13] Francesca Fossati, Stephane Rovedakis, and Stefano Secci. Distributed algorithms for multi-resource allocation. *IEEE Transactions on Parallel and Distributed Systems*, 33(10):2524–2539, 2022. 8

[14] Xiulin Li, Li Pan, Shijun Liu, and Xiangxu Meng. A fair and efficient resource allocation algorithm for cloud rendering jobs. *IEEE Transactions on Services Computing*, 2025. 8

[15] Hao Guo and Weidong Li. Dynamic multi-resource fair allocation with elastic demands: H. guo and w. li. *Journal of Grid Computing*, 22(1):35, 2024. 8

[16] Wei Wang, Baochun Li, and Ben Liang. Dominant resource fairness in cloud computing systems with heterogeneous servers. In *IEEE INFOCOM*, pages 583–591, 2014. 8

[17] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of NSDI*, pages 289–304, 2020. 8

[18] Francesca Fossati, Stefano Moretti, Patrice Perny, and Stefano Secci. Multi-resource allocation for network slicing. *IEEE/ACM Transactions on Networking*, 28(3):1311–1324, 2020. 8

[19] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, 2024. 8

[20] Eric J Friedman and Shane G Henderson. Fairness and efficiency in web server protocols. *ACM SIGMETRICS Performance Evaluation Review*, 31(1):229–237, 2003. 8, 75

[21] Eric J Friedman and Gavin Hurley. Protective scheduling. Technical report, Cornell University Operations Research and Industrial Engineering, 2003. 8, 75, 76, 77

[22] Jyoti V Gautam, Harshadkumar B Prajapati, Vipul K Dabhi, and Sanjay Chaudhary. A survey on job scheduling algorithms in big data processing. In *2015 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, pages 1–11. IEEE, 2015. 8

[23] Einollah Jafarnejad Ghomi, Amir Masoud Rahmani, and Nooruldeen Nasih Qader. Load-balancing algorithms in cloud computing: A survey. *Journal of Network and Computer Applications*, 88:50–71, 2017.

[24] Soudabeh Hedayati, Neda Maleki, Tobias Olsson, Fredrik Ahlgren, Mahdi Seyednezhad, and Kamal Berahmand. Mapreduce scheduling algorithms in hadoop: a systematic study. *Journal of Cloud Computing*, 12(1):143, 2023.

[25] Yujian Wu, Shanjiang Tang, Ce Yu, Bin Yang, Chao Sun, Jian Xiao, Hutong Wu, and Jinghua Feng. Task scheduling in geo-distributed computing: a survey. *IEEE Transactions on Parallel and Distributed Systems*, 2025. 8

[26] Chien-Chun Hung, Leana Golubchik, and Minlan Yu. Scheduling jobs across geo-distributed datacenters. In *Proceedings of ACM SOCC*, pages 111–124, 2015. 8, 59, 67, 86, 90

[27] Yitong Guan and Xueyan Tang. On task assignment and scheduling for distributed job execution. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 726–735. IEEE, 2022. 8, 70

[28] Hailiang Zhao, Xueyan Tang, Peng Chen, Jianwei Yin, and Shuiguang Deng. Data-locality-aware task assignment and scheduling for distributed job executions. *IEEE Transactions on Services Computing*, 18(5):2701–2713, 2025. 8, 70

[29] Zhiming Hu, Baochun Li, and Jun Luo. Flutter: Scheduling tasks closer to data across geo-distributed datacenters. In *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*, pages 1–9. IEEE, 2016. 9

[30] Qifan Pu, Ganesh Ananthanarayanan, Peter Bodik, Srikanth Kandula, Aditya Akella, Paramvir Bahl, and Ion Stoica. Low latency geo-distributed data analytics. *ACM SIGCOMM Computer Communication Review*, 45(4):421–434, 2015. 9

[31] Chien-Chun Hung, Ganesh Ananthanarayanan, Leana Golubchik, Minlan Yu, and Mingyang Zhang. Wide-area analytics with multiple resources. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–16, 2018. 9

[32] Moïse W Convolbo, Jerry Chou, Ching-Hsien Hsu, and Yeh Ching Chung. Geodis: towards the optimization of data locality-aware job scheduling in geo-distributed data centers. *Computing*, 100(1):21–46, 2018. 9, 67

[33] Chunlin Li, Jianhang Tang, Tao Ma, Xihao Yang, and Youlong Luo. Load balance based workflow job scheduling algorithm in distributed cloud. *Journal of Network and Computer Applications*, 152:102518, 2020. 9

[34] Xiaoping Li, Fuchao Chen, Ruben Ruiz, and Jie Zhu. Mapreduce task scheduling in heterogeneous geo-distributed data centers. *IEEE Transactions on Services Computing*, 15(6):3317–3329, 2021. 9

[35] Xu Chen. Decentralized computation offloading game for mobile cloud computing. *IEEE Transactions on Parallel and Distributed Systems*, 26(4):974–983, 2014. 9

[36] Ninad Hogade, Sudeep Pasricha, and Howard Jay Siegel. Energy and network aware workload management for geographically distributed data centers. *IEEE Transactions on Sustainable Computing*, 7(2):400–413, 2022. 9

[37] Chao Wang, Lei Liu, Chunxiao Jiang, Shangguang Wang, Peiying Zhang, and Shigen Shen. Incorporating distributed drl into storage resource optimization of space-air-ground integrated wireless communication network. *IEEE Journal of Selected Topics in Signal Processing*, 16(3):434–446, 2021. 9, 107

[38] Haiyuan Li, Karcius Day R Assis, Shuangyi Yan, and Dimitra Simeonidou. Drl-based long-term resource planning for task offloading policies in multi-server edge computing networks. *IEEE Transactions on Network and Service Management*, 19(4):4151–4164, 2022. 9

[39] Amin Mohajer, Javad Hajipour, and Victor CM Leung. Dynamic offloading in mobile edge computing with traffic-aware network slicing and adaptive td3 strategy. *IEEE Communications Letters*, 2024. 9, 107

[40] Xianglai Liao, Xin Hu, Zhijun Liu, Shijun Ma, Lexi Xu, Xiuhua Li, Weidong Wang, and Fadhel M Ghannouchi. Distributed intelligence: A verification for multi-agent drl-based multibeam satellite resource allocation. *IEEE Communications Letters*, 24(12):2785–2789, 2020. 9

[41] Xiaonan Li, Haijun Zhang, Huan Zhou, Ning Wang, Keping Long, Saba Al-Rubaye, and George K Karagiannidis. Multi-agent drl for resource allocation and cache design in terrestrial-satellite networks. *IEEE Transactions on Wireless Communications*, 22(8):5031–5042, 2022. 9

[42] Wenting Wei, Huaxi Gu, Kun Wang, Jianjia Li, Xuan Zhang, and Ning Wang. Multi-dimensional resource allocation in distributed data centers using deep reinforcement learning. *IEEE Transactions on Network and Service Management*, 20(2):1817–1829, 2022. 9, 107

[43] Li Chen, Shuhao Liu, and Baochun Li. Optimizing network transfers for data analytic jobs across geo-distributed datacenters. *IEEE Transactions on Parallel and Distributed Systems*, 33(2):403–414, 2021. 9

[44] Jiuchen Shi, Kan Fu, Jiaqi Wang, Qiang Chen, Dan Zeng, and Minyi Guo. Adaptive qos-aware microservice deployment with excessive loads via intra- and inter-datacenter scheduling. *IEEE Transactions on Parallel and Distributed Systems*, 35(9):1565–1582, 2024. 9

[45] Laiping Zhao, Yanan Yang, Ali Munir, Alex X Liu, Yue Li, and Wenyu Qu. Optimizing geo-distributed data analytics with coordinated task scheduling and routing. *IEEE Transactions on Parallel and Distributed Systems*, 31(2): 279–293, 2019. 9

[46] Yitong Guan, Chuanyou Li, and Xueyan Tang. On max-min fair resource allocation for distributed job execution. In *Proceedings of the 48th International Conference on Parallel Processing*, pages 1–10, 2019. 9, 15, 33

[47] Li Chen, Shuhao Liu, Baochun Li, and Bo Li. Scheduling jobs across geo-distributed datacenters with max-min fairness. *IEEE Transactions on Network Science and Engineering*, 6(3):488–500, 2018. 9

[48] Erfan Meskar and Ben Liang. Fair multi-resource allocation in heterogeneous servers with an external resource type. *IEEE/ACM Transactions on Networking*, 31(3):1244–1262, 2022. 10

[49] Xingxing Li, Guangqin Hu, Weidong Li, and Xuejie Zhang. Fair multiresource allocation with access constraint in cloud–edge systems. *Future Generation Computer Systems*, 159:395–410, 2024. 10

[50] Huangshi Tian, Yunchuan Zheng, and Wei Wang. Characterizing and synthesizing task dependencies of data-parallel jobs in alibaba cloud. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 139–151, 2019. 10

[51] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of the ACM symposium on cloud computing*, pages 412–426, 2021. 10

[52] Sara Migliorini, Alberto Belussi, Elisa Quintarelli, and Damiano Carra. Copart: a context-based partitioning technique for big data. *Journal of Big Data*, 8:1–28, 2021. 13

[53] James Renegar. Linear programming, complexity theory and elementary functional analysis. *Mathematical Programming*, 70(1-3):279–351, 1995. 27

[54] Alibaba. Alibaba cluster trace program, 2017. URL `https://github.com/alibaba/clusterdata`. 34, 64, 88

[55] Google. Google clusterdata 2019 traces, 2019. URL `https://github.com/google/cluster-data`. 34

[56] IBM. IBM ILOG CPLEX optimization studio, 2024. URL `https://www.ibm.com/products/ilog-cplex-optimization-studio`. 35

[57] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010. 51

[58] James G Shanahan and Laing Dai. Large scale distributed data science using apache spark. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pages 2323–2324, 2015. 51

[59] Akshay Jajoo, Y Charlie Hu, Xiaojun Lin, and Nan Deng. Slearn: A case for task sampling based learning for cluster job scheduling. *IEEE Transactions on Cloud Computing*, 11(3):2664–2680, 2022. 51

[60] Eugene L Lawler. *Combinatorial optimization: networks and matroids*. Courier Corporation, 2001. 58

[61] Péter Kovács. Minimum-cost flow algorithms: an experimental evaluation. *Optimization Methods and Software*, 30(1):94–127, 2015. 58, 63

[62] Laurent Perron and Vincent Furnon. Or-tools. `https://developers.google.com/optimization/`, 2023. Google Optimization Tools. 67